CSCI3360 Introduction to Computational Complexity (2026 Fall)

Xiao Liang

https://xiao-liang.github.io

Department of Computer Science and Engineering
The Chinese University of Hong Kong

(Updated on October 26, 2026)

Disclaimer: These lecture notes originate from the 2026-Fall offering of the course. They have been lightly edited for accuracy and completeness, and may still contain typographical or other errors.

Contents

1 The Model of Computation: Turing Machines **1**
1.1 Before Turing Machines: Restricted Models of Computation . . . 1
1.1.1 Finite Automata . . . 1
1.1.2 Pushdown Automata . . . 2
1.2 Turing Machines . . . 4
1.2.1 Informal Description . . . 4
1.2.2 Formal Definition . . . 5
1.2.3 A Turing Machine for Palindromes . . . 8
1.2.4 Running Example: $x = \texttt{radar}$ . . . 8
1.2.5 Going Beyond Pushdown Automata . . . 10
1.3 Robustness of the Turing-Machine Model . . . 12
1.3.1 Three Tapes Versus One Tape . . . 12
1.3.2 One-Way Versus Bidirectional Tapes . . . 14
1.4 Universal Turing Machines . . . 16
1.4.1 Machines as Data . . . 16
1.4.2 Definition and Basic Simulation . . . 17
1.4.3 Efficient Universal Simulation . . . 17
1.4.4 *Proof of Theorem 1.4.2 (Part 1/2) . . . 19
1.4.4.1 Example 1: Computing powers by repeated squaring . . . 20
1.4.4.2 Example 2: Incrementing a binary counter . . . 21
1.4.5 *Proof of Theorem 1.4.2 (Part 2/2) . . . 22
1.4.5.1 The main idea: move the contents, not the simulated heads . . . 22
1.4.5.2 Buffers and their meaning . . . 23
1.4.5.3 Simulating one transition . . . 24
1.4.5.4 Simulating a move to the right . . . 24
1.4.5.5 The cost of one rearrangement . . . 25
1.4.5.6 Why large rearrangements are rare . . . 26
1.4.5.7 Summing the costs . . . 27
1.5 The Church–Turing Thesis . . . 27
1.5.1 The Standard Church–Turing Thesis . . . 27
1.5.2 The Extended Church–Turing Thesis . . . 28
1.5.3 Quantum Computing and the Extended Thesis . . . 29

1.5.4 The Turing Machine as a Final Model . . . 29
1.6 Closing Remarks . . . 30

2 Deterministic Polynomial Time: P **31**
2.1 Polynomial Time and the Class P . . . 31
2.1.1 Why Polynomial Time? . . . 31
2.1.2 The Formal Definition . . . 32
2.1.3 Input Encodings and Bit Complexity . . . 33
2.2 Representative Decision Problems in P . . . 34
2.2.1 Relative Primality . . . 34
2.2.2 Directed Reachability . . . 35
2.2.3 Bipartite Matching with a Size Threshold . . . 37
2.3 Decision and Search . . . 41
2.3.1 Witnesses and Search Problems . . . 41
2.3.2 Polynomial-Time Search-to-Decision Reductions . . . 42
2.4 When the Natural Existence Question Is Not Enough . . . 45
2.4.1 The Maximum-Cut Problem . . . 46
2.4.2 Factorization versus Compositeness . . . 47
2.4.3 Further Discussion of Primality Testing . . . 48
2.5 Randomized Algorithms and the Limits of P . . . 53

3 Non-Deterministic Polynomial Time: NP **55**
3.1 From Efficient Solution to Efficient Verification . . . 55
3.2 Witnesses, Polynomial-Time Verifiers, and the Definition of NP . . . 56
3.3 Examples of Problems in NP . . . 58
3.3.1 Boolean Satisfiability . . . 59
3.3.2 Hamiltonian Path . . . 66
3.3.3 Clique . . . 67
3.3.4 Independent Set . . . 68
3.3.5 Graph Three-Coloring . . . 69
3.3.6 Knapsack . . . 70
3.3.7 Subset Sum . . . 71
3.3.8 Composite Integers . . . 72
3.4 The Relationship Between P and NP . . . 73
3.4.1 The P Versus NP Question . . . 74
3.5 Nondeterministic Turing Machines . . . 74
3.5.1 Nondeterministic Computation . . . 74
3.5.2 The Guess-and-Check View . . . 76
3.5.3 The Nondeterministic Definition of NP . . . 76
3.6 Equivalence of the Two Definitions of NP . . . 77
3.6.1 From Verifiers to Nondeterministic Machines . . . 78
3.6.2 From Nondeterministic Machines to Verifiers . . . 78

4 Reducibility and NP-Completeness **80**
4.1 Polynomial-Time Reducibility . . . 80
4.1.1 Polynomial-Time Computable Functions . . . 80
4.1.2 Polynomial-Time Reductions . . . 81
4.1.3 Using a Reduction to Obtain an Algorithm . . . 81
4.1.4 Composition of Reductions . . . 82
4.2 NP-Hardness and NP-Completeness . . . 83
4.3 Example: Reduce 3SAT to CLIQUE . . . 84
4.3.1 The Language 3SAT . . . 84

4.3.2 The Target Problem . . . 85
4.4 Warm-Up: Describing Computations with CNF Formulas . . . 87
4.4.1 Clauses as Constraints . . . 87
4.4.2 Forbidding a Local Assignment . . . 88
4.4.3 Encoding the Contents of One Cell . . . 89
4.4.4 Local Views of Turing-Machine Computations . . . 89
4.5 The Cook–Levin Theorem . . . 90
4.5.1 Proof of the Cook–Levin Theorem . . . 91
4.5.1.1 Model, Normalization, and Computation Tableau . . . 91
4.5.1.2 Constraints as Boolean Formulas . . . 93
4.5.1.3 Correctness of the Construction . . . 95
4.5.1.4 Size and Construction Time . . . 96
4.6 Consequences of the Cook–Levin Theorem . . . 97
4.6.1 3SAT Is NP-Complete . . . 97
4.6.2 The 2SAT problem . . . 101
4.6.3 CLIQUE Is NP-Complete . . . 102

5 Exemplary NP-Complete Problems 103
5.1 Independent Set . . . 103
5.2 Vertex Cover . . . 104
5.2.1 Minimum Vertex Cover: From Decision to Search . . . 105
5.2.2 Maximum Matching and Minimum Vertex Cover . . . 105
5.3 Graph 3-Coloring . . . 108
5.3.1 Three Colors, Four Colors, and a Mathematical Mystery . . . 109
5.4 Hamiltonian Path and Subset Sum . . . 111

Bibliography 112

Chapter 1

The Model of Computation: Turing Machines

Computational complexity asks not only whether a problem can be solved, but also how many computational resources are required to solve it. Before we can measure resources such as time and memory, however, we must specify what counts as a computation. In other words, we need a mathematical model of an algorithm.

We begin by recalling two models from formal-language theory: finite automata and pushdown automata, which were already covered in **CSCI3130 Formal Languages and Automata Theory**. These models are useful precisely because their memory is restricted, but the same restrictions prevent them from expressing general computation. We then introduce the Turing machine, work through algorithms for recognizing palindromes and a non-context-free language, and examine the robustness of the model. Finally, we introduce universal Turing machines and discuss the Church–Turing thesis and its complexity-theoretic extension.

To study the complexity of computation, we must first define a model of computation. The model must be precise enough for mathematical proofs and expressive enough to represent general algorithms.

1.1 Before Turing Machines: Restricted Models of Computation

Nothing from Section 1.1 will be tested on quizzes or exams!

1.1.1 Finite Automata

Recall that a deterministic finite automaton has a finite set of states and reads its input from left to right. At every step, its next state is determined by its current state and the next input symbol.

Definition 1.1.1 (Deterministic Finite Automaton)**.** *A* deterministic finite automaton*, or DFA, is a tuple*

$$A = (Q, \Sigma, \delta, q_0, F),$$

where:

- Q *is a finite set of states;*
- Σ *is a finite input alphabet;*
- $\delta : Q \times \Sigma \to Q$ *is the transition function;*
- $q_0 \in Q$ *is the start state; and*
- $F \subseteq Q$ *is the set of accepting states.*

The automaton reads its input only once, from left to right. It accepts an input $x \in \Sigma^*$ if the state reached after reading all of x belongs to F. ◊

The current state of a DFA contains all the information that the machine retains about the portion of the input that it has already read. Since Q is finite, a DFA has only a constant amount of memory, independent of the input length.

Example 1.1.1 (A Language Recognized by a DFA)**.** *Consider the language*

$$L_{\text{even}} \coloneqq \{x \in \{0,1\}^* : x \text{ contains an even number of } 1\text{'s}\}.$$

A two-state DFA recognizes this language. One state records that the number of 1*'s seen so far is even, and the other records that it is odd. Reading a* 1 *switches states, while reading a* 0 *leaves the state unchanged. This is illustrated in Figure 1.1.*

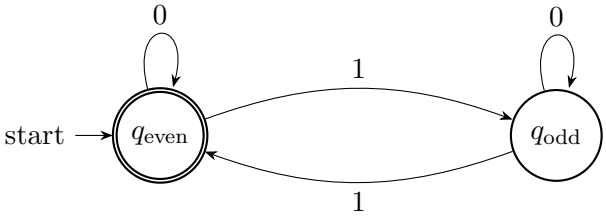


Figure 1.1: A DFA recognizing binary strings containing an even number of 1's.

The boundary of finite-state computation

Finite automata recognize exactly the *regular languages*. They are well suited to problems in which only a bounded amount of historical information is needed.

For example, finite automata are used in the lexical-analysis stage of a compiler. A lexer may recognize identifiers, integer literals, punctuation, and keywords using regular expressions and finite-state machinery.

The limitation is that a DFA cannot store an unbounded counter. Consequently, it cannot recognize the language

$$L_{\text{eq}} \coloneqq \{0^n 1^n : n \geq 0\}. \tag{1.1}$$

To recognize L_{eq}, a machine must compare an arbitrarily large number of 0's with an arbitrarily large number of 1's.

A finite set of states cannot remember the required number for all possible input lengths.

1.1.2 Pushdown Automata

A pushdown automaton extends a finite automaton by providing it with a stack. The stack can grow with the input, so a pushdown automaton can remember an unbounded amount of information. However, that information can be accessed only in last-in, first-out order.

Definition 1.1.2 (Deterministic Pushdown Automaton)**.** *A* deterministic pushdown automaton *(DPDA) is a* 6*-tuple*

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F),$$

where:

1. *Q is a finite set of states;*
2. *Σ is the finite input alphabet;*

3. Γ *is the finite stack alphabet;*

4. $\delta\colon Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightharpoonup Q \times \Gamma_\varepsilon$ *is the transition function; Here,*

$$\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\} \qquad \textit{and} \qquad \Gamma_\varepsilon = \Gamma \cup \{\varepsilon\},$$

where ε denotes the empty string.

5. $q_0 \in Q$ *is the start state; and*

6. $F \subseteq Q$ *is the set of accept states.*

The automaton reads its input only once, from left to right. It accepts an input $x \in \Sigma^*$ if the state reached after reading all of x belongs to F.

$\Diamond$

Remark 1.1.1 (On Nondeterminism)**.** *There is a more powerful version called a* nondeterministic pushdown automaton *(NPDA), in which the transition function is allowed to return a set of possible next states and stack symbols. Deterministic and nondeterministic pushdown automata do not have the same expressive power. Every language recognized by a DPDA can be recognized by an NPDA, but the converse does not hold. More precisely,*

$$\text{DCFL} \subsetneq \text{CFL}.$$

Thus, NPDAs recognize all context-free languages, whereas DPDAs recognize only the proper subclass of deterministic context-free languages.

Think of the stack as restricted working memory. The machine can access the most recently stored symbol, but it cannot directly inspect a symbol buried in the middle of the stack.

Example 1.1.2 (Recognizing 0^n1^n with a Stack)**.** *A DPDA can recognize L_{eq} from Equation (1.1) as follows:*

1. *For each* 0 *in the first part of the input, push one marker onto the stack.*

2. *When the first* 1 *is encountered, begin popping one marker for each* 1*.*

3. *Accept if the input ends exactly when the stack returns to its initial marker.*

Such a DPDA is shown in Figure 1.2. For the input 000111*, the stack first grows by three markers and then shrinks by three markers. For* 00111*, the machine attempts to pop a third marker after only two have been stored, so it rejects.*

Nondeterministic pushdown automata recognize exactly the *context-free languages*. In compiler design, they provide the mathematical basis for parsing nested syntactic structures, such as balanced parentheses, arithmetic expressions, and recursively nested blocks.

The boundary of stack-based computation

Although a stack is more powerful than finite-state memory, one stack does not provide general-purpose random-access memory. A standard example beyond the power of pushdown automata is

$$L_{\text{three}} \coloneqq \{0^n1^n2^n : n \geq 0\}. \tag{1.2}$$

A machine recognizing this language must verify that three independently delimited blocks have the same length.

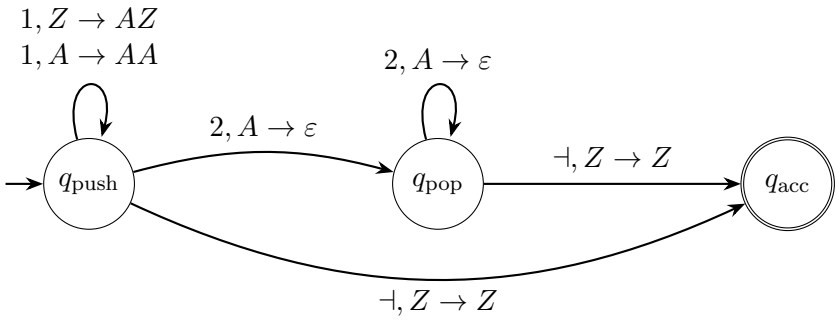


Figure 1.2: A deterministic pushdown automaton for $L = \{1^n 2^n \mid n \geq 0\}$. Each transition has the form "input, stack top $\to$ replacement." The automaton pushes one A for every 1 and pops one A for every 2. Here, Z is the bottom-of-stack symbol, ε means that the top symbol is removed, and $\dashv$ marks the end of the input. Thus, $\dashv, Z \to Z$ means that the input has ended, only Z remains on the stack, and Z is left unchanged.

A stack can conveniently match the 0's against the 1's by pushing for the first block and popping for the second. After those comparisons, however, the stored count has been destroyed, leaving no copy with which to check the 2's.

Fact 1.1.1 (Pushdown-Automaton Boundary)**.** *The language L_{three} in Equation (1.2) is not context-free. Therefore, no pushdown automaton recognizes it.*

The statement in Fact 1.1.1 can be proved using the pumping lemma for context-free languages or Ogden's lemma. The important point for us is conceptual: the stack provides unbounded but highly structured memory, and that structure imposes a genuine limit on the languages a PDA can recognize.

Takeaway

Finite automata and pushdown automata remain important because their restrictions make them suitable for particular applications and allow precise classifications of languages. They are not, however, expressive enough to represent every algorithm that we would normally regard as executable by a computer. This motivates the Turing-machine model.

1.2 Turing Machines

Our next goal is to formalize a general-purpose model of computation. To this end, we introduce *Turing machines*, which may be viewed roughly as finite automata augmented with unbounded memory and the ability to move back and forth over their input, rather than being restricted to a single left-to-right pass.

1.2.1 Informal Description

An informal algorithm consists of a fixed collection of mechanical rules that can be applied repeatedly. A typical step may perform the following operations:

- read symbols from the input or working memory;
- write symbols to working memory;
- update a finite control state;

- move to nearby memory locations; and
- halt with an output.

A Turing machine turns this description into a precise mathematical object. Each tape is divided into cells, and each cell stores one symbol from a finite alphabet. A tape head reads the symbol in the current cell and may move one position left or right.

The tapes in our definition are unbounded to the right and have a leftmost cell. This is a mathematical abstraction: an execution lasting T steps can visit at most $T + 1$ cells on any tape, so every finite computation uses only a finite portion of each tape.

1.2.2 Formal Definition

Definition 1.2.1 (Three-Tape Turing Machine)**.** *A* three-tape Turing machine *is a tuple*

$$M = (\Gamma, Q, \delta),$$

with the following components:

- Γ *is a finite set of symbols that M's tape can contain. We call it the alphabet.*

  We assume that it contains a blank symbol $\sqcup$*, a left-end marker* $\triangleright$*, and a set of some characters*[1] *(e.g., the English alphabet, or the binary bits* $\{0, 1\}$*).*
- Q *is a finite set of states containing a start state* q_{start} *and a halting state* q_{halt}*.*
- *The transition function is*

$$\delta : Q \times \Gamma^3 \longrightarrow Q \times \Gamma^2 \times \{L, S, R\}^3. \tag{1.3}$$

The first tape is a read-only input tape. The second tape is a read/write work tape, and the third tape is a read/write output tape. ♢

Let us unpack the transition function in Equation (1.3). Suppose

$$\delta(q, a, b, c) = (q', b', c', d_1, d_2, d_3).$$

This transition instructs the machine to:

1. change its state from q to q';
2. leave the input symbol a unchanged; (recall that the input tape is read-only;)
3. replace the work-tape symbol b by b';
4. replace the output-tape symbol c by c'; and
5. move the three heads according to

$$d_1, d_2, d_3 \in \{L, S, R\},$$

   where L, S, and R mean move left, stay in place, and move right, respectively.

If a head is at the left-end marker and is instructed to move left, it remains in place.

[1] Note that the choice of characters is arbitrary. But it is without loss of generality that we only consider binary alphabets, because any finite alphabet can be encoded as a binary string. On potential issue is the size of encoding may grow. But as we will see, this does not affect the computational power of Turing machines, at least to the extent that this course is concerned.

Definition 1.2.2 (Start Configuration)**.** *On input $x = x_1 x_2 \cdots x_n \in \Gamma^*$, the* start configuration *of a three-tape Turing machine is defined as follows:*

- *the input tape contains*
$$\triangleright x_1 x_2 \cdots x_n \sqcup \sqcup \cdots ;$$
- *the work and output tapes each contain*
$$\triangleright \sqcup \sqcup \sqcup \cdots ;$$
- *all three heads are placed at their left-end markers; and*
- *the finite control is in state q_{start}.*

$\Diamond$

The machine proceeds by repeatedly applying its transition function. If it enters q_{halt}, its computation stops. The finite string written immediately to the right of the output tape's left-end marker is interpreted as the machine's output.

Definition 1.2.3 (Computing a Function and Running Time)**.** *Let $f : \{0,1\}^* \to \{0,1\}^*$, and let $T : \mathbb{N} \to \mathbb{N}$. A Turing machine M* computes *f* in time *$T(n)$ if, for every input $x \in \{0,1\}^*$, the machine:*

1. *starts in the start configuration on x;*
2. *halts after at most $T(|x|)$ transitions; and*
3. *has $f(x)$ written on its output tape when it halts.*

The running time of M is the maximum number of transitions it performs over all inputs of a given length. $\Diamond$

The quantity $n = |x|$ is the input length. Complexity theory studies how resource usage grows as a function of n, rather than how long a machine takes on only one particular input.

Definition 1.2.4 (Deciding a Language)**.** *Let $L \subseteq \{0,1\}^*$. A Turing machine M* decides *L if it halts on every input x and outputs*

$$M(x) = \begin{cases} 1, & x \in L, \\ 0, & x \notin L. \end{cases} \tag{1.4}$$

Equivalently, M computes the characteristic function of L. $\Diamond$

Remark 1.2.1. *In complexity theory, we normally study machines that halt on every input. A machine that accepts members of a language but may run forever on nonmembers is called a* recognizer *rather than a* decider*. Unless stated otherwise, the algorithms in these notes are deciders.*

Notice that Definition 1.2.4 use the concept of a language to formalize the notion of a decision problem. Do not worry if you are not familiar with the concept of a language. Let us explain it in the following example.

Example 1.2.1 (Representing a Property as a Language)**.** *A language is a set of strings satisfying a specified property. For example, consider the property*

"The number of 1*'s in the binary string is a multiple of* 3*."*

This property determines the language

$$L_{\text{mult-3}} \coloneqq \{x \in \{0,1\}^* : \textit{the number of } 1\textit{'s in } x \textit{ is a multiple of } 3\}\,.$$

If $\#_1(x)$ *denotes the number of occurrences of* 1 *in* x*, then the same language can be written more concisely as*

$$L_{\text{mult-3}} = \{x \in \{0,1\}^* : \#_1(x) \equiv 0 \pmod 3\}\,.$$

For instance,

$$\varepsilon,\ 0,\ 000,\ 111,\ 10101 \in L_{\text{mult-3}},$$

because these strings contain either zero or three occurrences of 1*. On the other hand,*

$$1,\ 10,\ 110,\ 1111 \notin L_{\text{mult-3}}.$$

Notice that 0 *is a multiple of* 3*, so every string containing no* 1*'s, including the empty string* ε*, belongs to the language.*

This language corresponds to the following decision problem:

Input: A binary string x.

Question: Is the number of 1's in x a multiple of 3?

The answer is "yes" exactly when $x \in L_{\text{mult-3}}$, and it is "no" exactly when $x \notin L_{\text{mult-3}}$.

Remark 1.2.2 (Decision Problems as Languages)**.** *Every decision problem whose instances have finite descriptions can be formalized as a language. First, encode each problem instance* I *as a binary string* $\langle I \rangle$*. The language associated with the problem is then*

$$L = \{\langle I \rangle \in \{0,1\}^* : \textit{the answer for instance } I \textit{ is "yes"}\}\,.$$

Thus, solving the decision problem is equivalent to deciding whether the encoding $\langle I \rangle$ *belongs to* L*.*

Strings in L *are called* yes-instances*, while strings not in* L *are called* no-instances*.*

Why Turing Machines?

The Turing machine provides a simple but powerful mathematical model of general computation. Its finite control represents the machine's fixed program: the set of states and transition rules is chosen in advance and does not depend on the particular input. The work tape represents memory whose size may grow as the computation proceeds. Thus, unlike a finite automaton, a Turing machine can store an amount of information that is not bounded by a constant.

Despite having potentially unbounded memory, the machine operates locally. During one transition, it can inspect only the three currently scanned symbols, overwrite only the scanned tape cells, and move each head by at most one cell. A complicated computation must therefore be carried out as a sequence of elementary local operations. Consequently, the number of transitions gives a meaningful measure of sequential running time, while the number of tape cells used gives a meaningful measure of memory consumption.

This model is also sufficiently general to express ordinary algorithms. Data structures, intermediate results, and program configurations can all be encoded as strings on the tapes. Although such encodings may be less convenient than the data types and instructions of a programming language, they allow a Turing machine to simulate the essential steps of a computer

program. In particular, a universal Turing machine can receive the description of another machine together with its input and simulate that machine, capturing the idea of a stored-program computer.

Finally, the simplicity of the model makes mathematical analysis possible. Questions about whether a problem is computable can be stated precisely in terms of whether some Turing machine decides the corresponding language. Questions about efficiency can likewise be studied by counting the machine's transitions and tape usage. Moreover, standard variants of the model—such as machines with one tape, several tapes, or different tape alphabets—compute the same class of languages and can simulate one another with controlled overhead. The Turing machine therefore abstracts away implementation details while retaining the fundamental notions of algorithm, memory, running time, and computability.

1.2.3 A Turing Machine for Palindromes

We now use the formal model to design a familiar algorithm. The purpose of this example is not to argue that palindrome testing is difficult. Rather, it illustrates how an ordinary algorithm can be implemented using tapes, head movements, and a finite collection of states.

Definition 1.2.5 (Palindrome Language)**.** *For an alphabet Σ, define*

$$\mathrm{PAL}_\Sigma := \{x \in \Sigma^* : x = x^R\},$$

where x^R denotes the reversal of x.

For example,

$$\texttt{radar}^R = \texttt{radar}, \qquad \texttt{complexity}^R = \texttt{ytixelpmoc}.$$

Thus, $\texttt{radar} \in \mathrm{PAL}_\Sigma$*, while* $\texttt{complexity} \notin \mathrm{PAL}_\Sigma$*.* ◊

Algorithm 1.2.1: Three-Tape Palindrome Algorithm

On input $x = x_1 x_2 \cdots x_n$:

1. Scan the input from left to right, copying each symbol to the work tape.
2. When the input head reaches the first blank after x, move the work head left so that it scans x_n.
3. Return the input head to the beginning of x, while leaving the work head in place.
4. Repeatedly compare the input symbols from left to right with the work-tape symbols from right to left.
5. If any pair differs, write 0 on the output tape and halt.
6. If all pairs agree, write 1 on the output tape and halt.

1.2.4 Running Example: $x = \texttt{radar}$

Let us follow the machine through its major phases.

Phase 1: Copy the input. Initially, the relevant portions of the tapes are

$$\begin{aligned} \text{input} &: \ \triangleright\ \texttt{r}\ \texttt{a}\ \texttt{d}\ \texttt{a}\ \texttt{r}\ \sqcup, \\ \text{work} &: \ \triangleright\ \sqcup\ \sqcup\ \sqcup\ \sqcup\ \sqcup\ \sqcup, \\ \text{output} &: \ \triangleright\ \sqcup\,. \end{aligned}$$

where the red color indicates the current position of each head.

After the copying phase, the work tape contains a copy of the input:

$$\begin{aligned} \text{input} &: \triangleright\, \texttt{r}\ \texttt{a}\ \texttt{d}\ \texttt{a}\ \texttt{r}\ \sqcup, \\ \text{work} &: \triangleright\, \texttt{r}\ \texttt{a}\ \texttt{d}\ \texttt{a}\ \texttt{r}\ \sqcup, \\ \text{output} &: \triangleright\ \sqcup\,. \end{aligned}$$

At this point, both heads have reached the blank immediately following the word.

Phase 2: Position the two heads. The work head moves left once and therefore scans the last symbol, `r`. The input head moves left until it reaches $\triangleright$, and then moves right once to the first symbol. The heads now scan opposite ends of the word:

$$\begin{aligned} \text{input} &: \triangleright\, \texttt{r}\ \texttt{a}\ \texttt{d}\ \texttt{a}\ \texttt{r}\ \sqcup, \\ \text{work} &: \triangleright\, \texttt{r}\ \texttt{a}\ \texttt{d}\ \texttt{a}\ \texttt{r}\ \sqcup, \\ \text{output} &: \triangleright\ \sqcup\,. \end{aligned}$$

Phase 3: Compare in opposite directions. The sequence of comparisons is shown in Table 1.1.

Comparison	Input-tape symbol	Work-tape symbol	Result
1	`r`	`r`	Match
2	`a`	`a`	Match
3	`d`	`d`	Match
4	`a`	`a`	Match
5	`r`	`r`	Match

Table 1.1: The comparison phase of the palindrome machine on input `radar`. The input head moves right while the work head moves left.

After the fifth successful comparison, the input head reaches the blank following the input and the work head reaches the left-end marker. The machine writes 1 to the output tape and halts.

Theorem 1.2.1 (Correctness and Running Time of Palindrome Testing)**.** *The machine in Algorithm 1.2.1 decides* PAL_Σ *in time* $3n + O(1)$*, where* n *is the input length.* ◊

Proof. Let the input be $x = x_1 x_2 \cdots x_n$.

The copying phase writes x_i into the i-th work-tape cell for every $i \in \{1, \ldots, n\}$. Therefore, after copying, the work tape contains exactly $x_1 x_2 \cdots x_n$.

The machine then positions the input head over x_1 and the work head over x_n. After $i - 1$ successful comparisons, the input head scans x_i, while the work head scans x_{n-i+1}. Thus, the i-th comparison checks whether

$$x_i = x_{n-i+1}.$$

The machine outputs 1 exactly when this equality holds for every i. This condition is equivalent to $x = x^R$, so the algorithm is correct.

The machine makes $3n + O(1)$ transitions in total:

- $n + O(1)$ transitions to copy the input;
- $n + O(1)$ transitions to return the input head to the beginning; and
- $n + O(1)$ transitions to compare the two copies.

In each phase, the n term accounts for the transitions that process or traverse the n input symbols. The $O(1)$ term accounts for a fixed number of additional transitions, such as detecting an endmarker, changing states, repositioning a head by a fixed number of cells, and entering an accepting or rejecting state. It is not hard to see that the number of these additional transitions is bounded by a constant independent of n.

■

Remark 1.2.3. *We remark that palindromes can actually be recognized by a* nondeterministic *pushdown automaton. Therefore, palindrome recognition is not being used here to separate Turing machines from all pushdown automata. It is used as a simple example of Turing-machine design. The separation from pushdown automata will instead use* L_{three}.

Takeaway

The work tape allows the machine to store a copy of its input and revisit that information in a different order. This is an elementary example of a general pattern: a Turing machine can reorganize and repeatedly access information rather than being restricted to one left-to-right pass or one stack discipline.

1.2.5 Going Beyond Pushdown Automata

We now return to the language

$$L_{\text{three}} = \{0^n 1^n 2^n : n \geq 0\},$$

which no pushdown automaton recognizes. We show that a Turing machine can decide this language.

Algorithm 1.2.2: Deciding L_{three}

On input x:

1. Verify that x has the form $0^*1^*2^*$. Reject if the symbols appear in any other order.
2. While scanning the block of 0's, write one marker symbol "X" on the work tape for each 0.
3. At the beginning of the block of 1's, return the work head to its first marker.
4. For each input symbol 1, move the work head right by one marker. Reject if the input has more 1's than there are markers.
5. At the end of the 1-block, reject unless the work head has passed exactly all the markers.
6. Return the work head to its first marker.
7. For each input symbol 2, move the work head right by one marker. Reject if the input has more 2's than there are markers.
8. At the end of the input, accept if and only if the work head has passed exactly all the markers.

Although Algorithm 1.2.2 is stated at a high level, each instruction can be expanded into ordinary Turing-machine transitions. We illustrate this for two representative steps. Let $\triangleright$ denote a left-endmarker and let $\sqcup$ denote the blank symbol.

- First, the machine can verify that the input has the form $0^*1^*2^*$ using only its finite control. It uses three states, corresponding to the three possible blocks. In the first state it reads 0's; upon seeing the first 1, it enters the second state; and upon seeing the first 2, it enters the third state. A 0 seen in either the second or third state, or a 1 seen in the third state, causes immediate rejection. Reaching the end of the input without such a violation confirms that the symbols occur in the required order.

- Second, consider Steps 3–5. After Step 2, the relevant part of the work tape has the form

$$\triangleright X^n \sqcup \sqcup \cdots .$$

  To return to the first marker, the work head repeatedly moves left until it sees $\triangleright$, and then moves one cell to the right. For each input symbol 1, the machine checks that the work head is scanning an X and then moves the work head one cell to the right. If it sees $\sqcup$ while another 1 remains, then there are more 1's than 0's, so it rejects. When the input head reaches the first 2, the machine checks the work-tape symbol currently under the work head. If that symbol is X, then some markers remain and there are fewer 1's than 0's; if it is $\sqcup$, then exactly all n markers have been traversed. Thus, these elementary tests implement the high-level instruction that the numbers of 0's and 1's must be equal. The comparison with the 2's is implemented in exactly the same way: the work head first returns to the left-endmarker and then traverses one X-cell for each 2.

These examples illustrate a general principle: phrases such as "verify the order," "return to the first marker," and "check that all markers have been passed" abbreviate finite sequences of transitions that inspect one symbol, change state, and move a tape head by one cell at a time.

Example 1.2.2 (Input 000111222)**.** *The machine first records three markers:*

$$\triangleright XXX \sqcup \sqcup \cdots .$$

It then uses these markers to count the three 1*'s. After returning to the beginning of the markers, it uses the same three markers again to count the three* 2*'s. Both comparisons succeed, so the machine accepts.*

For the input 0011122*, the machine encounters a third* 1 *after it has already passed its two stored markers. It therefore rejects.*

Proposition 1.2.1. *Algorithm 1.2.2 is an $O(n)$-time (i.e., linear-time) Turing machine that decides L_{three}.*

Proof. The algorithm in Algorithm 1.2.2 accepts only strings of the form $0^a1^b2^c$. It writes exactly a markers and accepts only if both the 1-block and the 2-block contain exactly one symbol per marker. Therefore, it accepts exactly when

$$a = b = c.$$

This is precisely the condition for membership in L_{three}.

The algorithm is in linear time: each input symbol is scanned a constant number of times, and the work-tape head makes only a constant number of complete traversals of the n stored markers.

■

Turing machines are strictly more expressive than finite automata and pushdown automata: they can decide languages, such as $\{0^n 1^n 2^n : n \geq 0\}$, that neither of the earlier models can recognize.

1.3 Robustness of the Turing-Machine Model

Many details of Definition 1.2.1 may appear arbitrary. Why three tapes rather than one? Why are tapes unbounded only to the right? Why is the input tape read-only? We now show that several natural variations produce the same class of computable functions. The simulations may change running time, but they do not change what can ultimately be computed.

1.3.1 Three Tapes Versus One Tape

Definition 1.3.1 (Single-Tape Turing Machine)**.** *A* single-tape Turing machine *has one read/write tape that initially contains*

$$\triangleright x \sqcup \sqcup \cdots .$$

The same tape is used for the input, working memory, and output. ◊

Lemma 1.3.1 (A Three-Tape Machine Simulates a Single-Tape Machine)**.** *If a single-tape Turing machine M computes a function in time $T(n)$, then a three-tape Turing machine computes the same function in time $O(T(n))$.* ◊

Proof. The three-tape machine simply uses the work tape exactly as M uses its single tape, storing M's simulated state in its own finite control.

Each transition of M is simulated by a constant number of transitions. When M halts, the simulator copies the designated output to its output tape. Thus, apart from the initial and final linear-time copies, the simulation has constant overhead per transition. ■

The reverse direction requires more work because one physical tape must represent three independently moving tapes.

Lemma 1.3.2 (A Single-Tape Machine Simulates a Three-Tape Machine)**.** *Suppose a three-tape Turing machine M runs in time $T(n)$. There is a single-tape Turing machine $\widetilde{M}$ that computes the same function in time*

$$O(T(n)^2).$$

◊

Proof. Let Γ be the tape alphabet of M, including the blank symbol $\sqcup$, and write $g = |\Gamma|$. We assume that tape 1 initially contains the input, tapes 2 and 3 are initially blank, and all three heads begin at their designated starting cells.

Encoding the three tapes. The machine $\widetilde{M}$ stores the three simulated tapes using its single physical tape. An encoded physical cell has the form

$$\big((a_1, h_1), (a_2, h_2), (a_3, h_3)\big),$$

where $a_i \in \Gamma$ is the symbol stored in that cell of simulated tape i, and

$$h_i \in \{0, 1\}$$

indicates whether the head of tape i is currently scanning that cell.

Each component (a_i, h_i) within the cell $\big((a_1, h_1), (a_2, h_2), (a_3, h_3)\big)$ is called a *track*. Conceptually, the three tracks are three aligned sequences laid on top of one another, with the i-th track recording the contents and head position of the i-th simulated tape.

Thus, for example, the encoded cell

$$\big((a_1, 0), (a_2, 1), (a_3, 0)\big)$$

stores the symbols $(a_1, 0)$, $(a_2, 1)$, $(a_3, 0)$ on the first, second, and third tracks, respectively. The head bits indicate that the second simulated head is located at this position, whereas the first and third simulated heads are not.

For each track there are $2g$ possible symbol-marker pairs:

$$|\Gamma \times \{0, 1\}| = 2g.$$

Consequently, the number of possible three-track (i.e., one-cell) symbols is

$$(2g)^3 = 8g^3.$$

The tape alphabet may also retain a separate copy of the original input alphabet for input cells that have not yet been encoded, adding at most g symbols. In addition, the simulator may use a constant number of special symbols for bookkeeping. Hence $\widetilde{M}$ can be given a tape alphabet of size at most

$$8g^3 + g + O(1).$$

In particular, the simulated alphabet remains finite. Notice that the head markers must be attached independently to the three components; merely adding one marked copy of each tuple would not suffice, since several simulated heads may occupy the same physical position.

At every point in the simulation, exactly one cell has $h_i = 1$ for each $i \in \{1, 2, 3\}$. The finite control of $\widetilde{M}$ also stores the current state of M.

To simulate one transition of M, the machine $\widetilde{M}$ performs two sweeps:

1. It sweeps from left to right, finds the three marked symbols, and stores their values in its finite control (see Remark 1.3.1). Once it has the current state and the three scanned symbols, it can determine the next state, the symbols to write, and the directions to move each head; this is done by invoking M's transition function.

2. It then sweeps back from right to left, updating the three symbols and moving the three head markers as required.

The main take way is: to simulate one transition of M, the machine $\widetilde{M}$ must sweep across the entire encoded region of its tape twice. Since M is a $T(n)$-time machine, the encoded region has length $O(T(n))$. Therefore, simulating one transition of M takes $O(T(n))$ time. Simulating all $T(n)$ transitions consequently takes

$$T(n) \cdot O(T(n)) = O(T(n)^2).$$

Remark 1.3.1 (On the Finite Control of a Turing Machine)**.** *The* finite control *of a Turing machine refers to its finite set of states. Thus, saying that $\widetilde{M}$ stores information in its finite control means that this information can be incorporated into its current state. In the simulation above, the finite control does not store the positions of the three simulated heads, since these positions may be arbitrarily large. Instead, the head positions are recorded on the tape by the marker bits h_1, h_2, h_3. The finite control stores only the current state q of M and the three scanned symbols a_1, a_2, a_3. This is possible because*

$$(q, a_1, a_2, a_3) \in Q_M \times \Gamma^3,$$

which is a finite set. Equivalently, $\widetilde{M}$ may have a distinct state for each possible tuple (q, a_1, a_2, a_3). More generally, storing information in finite control is possible whenever that information ranges over a fixed finite set independent of the input length; unbounded information, such as arbitrary head positions, must instead be stored on the tape.

■

Combining Lemma 1.3.1 and Lemma 1.3.2 shows that the two models compute the same functions, up to a polynomial slowdown:

Theorem 1.3.1 (Tape-Number Robustness)**.** *A function is computable by a three-tape Turing machine if and only if it is computable by a single-tape Turing machine. The efficiency is up to a quadratic relationship in running time.* ◊

Remark 1.3.2. *The quadratic overhead in Lemma 1.3.2 is an upper bound for a straightforward simulation, not a claim that every multi-tape computation inherently requires a quadratic slowdown. More sophisticated simulations can achieve smaller overhead.*

1.3.2 One-Way Versus Bidirectional Tapes

Definition 1.3.2 (Bidirectional Turing Machine)**.** *A* bidirectional Turing machine *is a Turing machine whose tape cells are indexed by all integers:*

$$\ldots, -3, -2, -1, 0, 1, 2, 3, \ldots.$$

Thus, its tape is unbounded in both directions. ◊

A standard one-way tape has cells indexed by $0, 1, 2, \ldots$. To simulate a bidirectional tape, we fold it around position 0. See Figure 1.3 for an illustration.

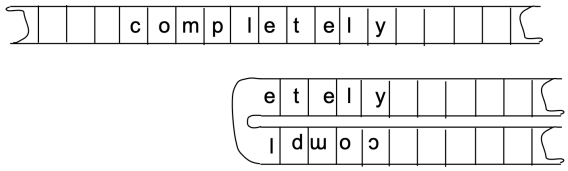


Figure 1.3: Folding a bidirectional tape between positions -1 and 0 to simulate it using a one-way-infinite tape. Adapted from [AB09].

Lemma 1.3.3 (Bidirectional Tapes Can Be Folded)**.** *Let M be a Turing machine whose tapes are infinite in both directions. If M computes a function f in time $T(n)$, then there is a standard Turing machine $\widetilde{M}$, whose tapes are infinite only to the right, that computes f in time $O(T(n))$.* ◊

Proof. We describe how to replace each bidirectional tape of M with a standard tape that has a left endpoint. Because the number of tapes is fixed, it suffices to explain the construction for one tape.

Index the cells of M's bidirectional tape by the integers:

$$\ldots, -3, -2, -1, 0, 1, 2, 3, \ldots,$$

with the tape head initially at cell 0. We "fold" this tape between cells -1 and 0. Cell $j \geq 0$ of

the corresponding standard tape stores the pair

$$(a_j, a_{-j-1}),$$

where a_i denotes the symbol in cell i of M's tape. Thus, the first few physical cells of $\widetilde{M}$'s tape represent

Cell of $\widetilde{M}$	0	1	2	3
First component	a_0	a_1	a_2	a_3
Second component	a_{-1}	a_{-2}	a_{-3}	a_{-4}.

The first components encode the nonnegative half of M's tape in its usual order, while the second components encode the negative half in reverse order. Again, see Figure 1.3 for an illustration.

If M has tape alphabet Γ, then each encoded cell contains an element of

$$\Gamma \times \Gamma.$$

The simulator therefore uses Γ^2, together with a constant number of special bookkeeping symbols, as part of its tape alphabet. This is still a finite alphabet.

The finite control of $\widetilde{M}$ records, for each simulated tape, which component is currently active:

$$\mathsf{P} \quad \text{or} \quad \mathsf{N}.$$

Mode P means that the simulated head is on the nonnegative half of the tape and is therefore reading the first component. Mode N means that it is on the negative half and is reading the second component. The physical head of $\widetilde{M}$ always scans the cell containing the currently simulated tape cell.

Suppose first that the simulator is in mode P and its physical head is at position j. It is then simulating the head of M at position j. Its movements are translated as follows:

Move by M	Move by $\widetilde{M}$	New mode
R	R	P
$L,\ j > 0$	L	P
$L,\ j = 0$	remain at cell 0	N.

The last line represents crossing the fold: moving left from simulated cell 0 leads to simulated cell -1, and both cells are represented in physical cell 0.

Now suppose that the simulator is in mode N and its physical head is at position j. It is then simulating the head of M at position $-j-1$. Since the negative half was stored in reverse order, left and right movements are interchanged:

Move by M	Move by $\widetilde{M}$	New mode
L	R	N
$R,\ j > 0$	L	N
$R,\ j = 0$	remain at cell 0	P.

The final line represents crossing the fold from simulated cell -1 to simulated cell 0.

The leftmost physical cell can be given a specially marked version of the paired alphabet, so $\widetilde{M}$ can recognize when its head is at the fold.

In summary, on the nonnegative half, head movements are simulated normally; On the negative half, left and right movements are reversed because that half is stored in reverse order. When the simulated head crosses between positions 0 and -1, the physical head remains at the leftmost cell, and the simulator switches between the two components.

If M has tape alphabet Γ, the simulator uses paired symbols from Γ^2. Each transition of

M can therefore be simulated using only a constant number of transitions: the simulator reads and updates the appropriate component of the pair and moves its head as described above. The same construction is applied independently to each tape of M. Therefore, a $T(n)$-step computation of M is simulated in $O(T(n))$ time. Hence, the resulting one-way-tape machine computes the same function as M.

■

What robustness does and does not say

These simulations establish that the models compute the same functions. They do not say that every choice of model gives exactly the same running time. For example, a direct three-tape palindrome algorithm runs in linear time, while some restricted single-tape implementations may require substantially more time.

Reasonable variants of the Turing-machine model agree on computability and usually agree on broad complexity distinctions such as polynomial versus exponential time. This robustness is one reason Turing machines are useful as a foundation for complexity theory.

1.4 Universal Turing Machines

1.4.1 Machines as Data

A Turing machine has a finite description: it has a finite alphabet, finitely many states, and a finite transition table.[2] Therefore, the description of a machine can itself be encoded as a binary string. This simple observation allows one machine to receive another machine as input and simulate it.

Let M be a Turing machine. We write

$$\langle M \rangle \in \{0,1\}^*$$

for a binary encoding of M. The encoding contains enough information to recover:

- the tape alphabet and an encoding of each tape symbol;
- the states, including the start and halting states;
- the number and roles of the tapes; and
- every entry of the transition function.

Because all these sets are finite, such an encoding can be constructed using ordinary binary representations and delimiters.

Definition 1.4.1 (Machine Represented by a String)**.** *For a binary string α, let M_α denote the Turing machine represented by α. Strings that are not valid encodings may be assigned to a fixed default machine, such as a machine that immediately outputs* 0 *and halts.* ◊

This convention ensures that every binary string denotes some machine. It is also convenient to permit harmless padding, so one machine may have more than one valid encoding.

[2]Note that the infinite tape is not part of the *description*; only the sentence that "a Turing machine has an infinite tape" is part of the description; this sentence has finite number of words, and thus can be encoded as a finite string.

Remark 1.4.1 (Programs as Data)**.** *The distinction between program and data is not absolute. A machine description is data when it is read by another machine, but it specifies a program when interpreted as a transition table. This principle underlies interpreters, compilers, stored-program computers, and many later arguments in complexity theory.*

1.4.2 Definition and Basic Simulation

Definition 1.4.2 (Universal Turing Machine)**.** *A* universal Turing machine *is a fixed Turing machine U such that, for every machine description α and every input x,*

$$U(\alpha, x) = M_\alpha(x), \tag{1.5}$$

whenever $M_\alpha(x)$ halts. If $M_\alpha(x)$ does not halt, then the simulation by U also need not halt. $\Diamond$

The notation $U(\alpha, x)$ means that U receives an encoding of the pair (α, x). A standard delimiter or self-delimiting encoding can be used to distinguish the two components.

The universal machine maintains encoded versions of:

- the simulated machine's transition table;
- the simulated machine's current state;
- the contents of its tapes; and
- the positions of its tape heads.

To simulate one transition, U reads the simulated state and scanned symbols, looks up the appropriate transition in the encoded table, and updates the simulated configuration.

Analogy: Virtual Machines

A useful real-world analogy is a *virtual machine*. A virtual machine (e.g., a VMWare® instance) may behave like a complete computer, with its own processor, memory, disks, and operating system. Nevertheless, everything needed to specify it—its configuration, virtual hardware, and software—can be stored as finite data on a physical computer.

A physical computer can read this description and simulate the virtual computer step by step. The same physical computer can simulate many different virtual machines; which machine it simulates depends on the description it is given. Similarly, a universal Turing machine can take an encoding $\langle M \rangle$ as input and simulate the particular Turing machine M described by that encoding.

This analogy also illustrates the idea of *programs as data*. To the host computer, a virtual-machine image is simply a file containing bits. When those bits are interpreted by virtualization software, however, they describe a computer and determine its behavior. In the same way, $\langle M \rangle$ is an ordinary binary string when treated as input, but it becomes a program when interpreted as the description of a Turing machine.

1.4.3 Efficient Universal Simulation

The existence of a universal Turing machine does not immediately imply that the simulation is efficient. In this subsection, we discuss the accurate cost of simulating a machine M_α on input x using a universal machine U. We will first show that a naive simulation may introduce quadratic overhead, and then we will present a more efficient simulation that incurs only logarithmic overhead.

Theorem 1.4.1 (Quadratic Universal Turing Machine)**.** *There exists a fixed Turing machine U such that, for every $\alpha, x \in \{0,1\}^*$,*

$$U(\alpha, x) = M_\alpha(x)$$

whenever $M_\alpha(x)$ halts.

Furthermore, if M_α halts on x within T steps, then U halts within

$$C_\alpha T^2 \tag{1.6}$$

steps, where C_α depends on the fixed machine M_α, including its number of states, alphabet size, and number of tapes, but does not depend on x or T. ◊

Before proving the theorem, let us emphasize its main strength: the input machine M_α is completely arbitrary. For example, M_α may have any fixed number of tapes and may use any finite tape alphabet. In contrast, the universal machine U is fixed once and for all; indeed, U may be taken to be a standard three-tape Turing machine with binary tape alphabet. Nevertheless, U can simulate every machine M_α with at most quadratic overhead in its running time. Thus, although the constants involved may depend on the description α, the quadratic form of the bound is uniform over all input machines.

Proof of Theorem 1.4.1. We construct the required universal Turing machine U. In fact, U will be a standard three-tape Turing machine with binary tape alphabet $\{0,1\}$.

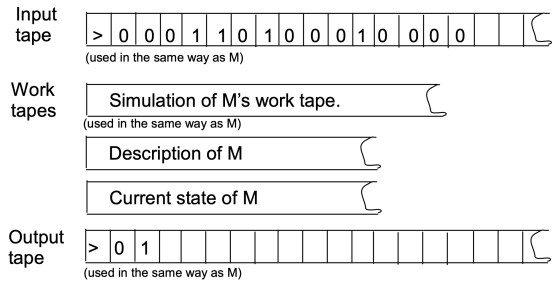


Figure 1.4: A fixed universal Turing machine U simulating an arbitrary machine M_α. The machine U is a standard three-tape Turing machine with binary tape alphabet, whereas M_α may have any fixed number of tapes and any finite alphabet. Adapted from [AB09, Figure 1.6].

The basic idea is straightforward. Given the description α of a Turing machine M_α and an input x, the machine U maintains an encoding of the current configuration of M_α. It then repeatedly consults α to determine the next transition and updates the encoded configuration accordingly.

This process is illustrated in Figure 1.4, where the "Description of M" and "Current state of M" segments can be placed at the beginning of the work tape (immediately preceding the "Simulation of M's work tape" segment), and occupy only a constant number of cells.

The main point is that this simulation remains efficient even though M_α may have multiple tapes and an arbitrary finite alphabet. This follows from two observations:

1. Let Γ be the tape alphabet of M_α. Each symbol of Γ can be represented by a binary block of length

   $$\lceil \log_2 |\Gamma| \rceil .$$

   Since Γ is fixed once M_α is fixed, operations on these blocks incur only constant overhead relative to the running time of M_α.

2. Suppose that M_α has k tapes. Its k tapes can be encoded on a single tape by placing their contents on separate tracks and marking the locations of their heads. As in the proof of Lemma 1.3.2, one transition of M_α can be simulated by scanning the portion of the encoded tape visited so far. After t steps, this portion has length $O(t)$, so simulating the first T transitions requires

$$\sum_{t=1}^{T} O(t) = O(T^2)$$

time. Although k may be arbitrarily large across different machines, it is a constant for each fixed machine M_α.

The universal machine U performs these operations directly from the binary description α. Consequently, if M_α halts on x within T steps, then U simulates the computation and produces the same output in $O(T^2)$ time, with constants that may depend on α.

This finishes the proof of Theorem 1.4.1.

■

By a more involved construction, one can achieve a more efficient simulation of a universal Turing machine. The following Theorem 1.4.2 shows that the overhead can be reduced to $O(T \log T)$ time. However, the proof of this theorem is much more technical and out of the scope of this undergraduate-level course. We present the proof in Sections 1.4.4 and 1.4.5 as optional readings for interested students.

Theorem 1.4.2 (Efficient Universal Turing Machine)**.** *There exists a fixed Turing machine U such that, for every $\alpha, x \in \{0,1\}^*$,*

$$U(\alpha, x) = M_\alpha(x)$$

whenever $M_\alpha(x)$ halts.

Furthermore, if M_α halts on x within $T \geq 2$ steps, then U halts within

$$C_\alpha T \log T \tag{1.7}$$

steps, where C_α depends on the fixed machine M_α, including its number of states, alphabet size, and number of tapes, but does not depend on x or T. ◊

1.4.4 *Proof of Theorem 1.4.2 (Part 1/2)

Nothing from Section 1.4.4 will be tested on quizzes or exams!

Before studying the $O(T \log T)$-time universal Turing machine simulation, we examine two simpler examples involving powers of two. They illustrate two related but distinct ideas:

- Organizing a computation into exponentially increasing scales produces only logarithmically many levels.
- An operation can be expensive occasionally without making a long sequence of operations expensive on average.

Repeated squaring illustrates the first idea. A binary counter illustrates the second.

These ideas will become crucial in the real proof as we will show in Section 1.4.5, where we use them in a more complicated way than in these examples. Thus, we believe it is helpful to understand them in a simpler context first, as we will do now in the current subsection.

1.4.4.1 Example 1: Computing powers by repeated squaring

Suppose we want to compute a^n, where n is a nonnegative integer. Multiplying a by itself repeatedly takes $n-1$ multiplications when $n \geq 1$. Can we use fewer multiplications?

The key identities are

$$a^{2m} = (a^m)^2, \qquad a^{2m+1} = a(a^m)^2.$$

Thus, to compute a^n, we can first compute $a^{\lfloor n/2 \rfloor}$, square the result, and multiply by a once more if n is odd.

More explicitly, the following recursive procedure computes a^n:

1. If $n = 0$, return 1.
2. If $n = 1$, return a.
3. Recursively compute
$$y = a^{\lfloor n/2 \rfloor}.$$
4. If n is even, return $y \cdot y$. If n is odd, return $a \cdot (y \cdot y)$.

The important point is that we compute y only once and then reuse it.

For example, to compute a^{13}, we can calculate

$$a^2 = a \cdot a, \qquad a^3 = a \cdot a^2, \qquad a^6 = a^3 \cdot a^3, \qquad a^{12} = a^6 \cdot a^6, \qquad a^{13} = a \cdot a^{12}.$$

This uses five multiplications rather than twelve.

Why the number of multiplications is logarithmic. Each recursive call roughly halves the exponent:

$$n, \quad \left\lfloor \frac{n}{2} \right\rfloor, \quad \left\lfloor \frac{n}{4} \right\rfloor, \quad \ldots, \quad 1.$$

For $n \geq 1$, there are $\lfloor \log_2 n \rfloor$ reductions before reaching 1, and each reduction contributes at most two multiplications. Hence the procedure uses

$$O(\log(n+1))$$

multiplications.

An equivalent way to understand the method is through the binary representation of the exponent. For instance,

$$13 = 8 + 4 + 1, \qquad \text{so} \qquad a^{13} = a^8 a^4 a.$$

Repeated squaring constructs

$$a, \quad a^2, \quad a^4, \quad a^8, \quad \ldots,$$

and the binary digits of n tell us which powers to multiply together.

A warning about the cost model. This is a bound on the *number of multiplications*, not necessarily on the number of Turing machine steps. Multiplication is not a constant-time operation when its operands are arbitrarily long integers. For example, if $a \geq 2$ is a fixed integer, the binary representation of a^n already has $\Theta(n)$ bits.

The lesson we want here is structural:

Doubling the scale at each level gives logarithmically many levels.

1.4.4.2 Example 2: Incrementing a binary counter

Consider a binary counter initially equal to 0. We increment it T times, where $T \geq 1$. Allocate

$$b = \lceil \log_2(T+1) \rceil$$

bits, which is enough to represent every value from 0 through T.

To increment the counter, start at the least significant bit:

1. Change each consecutive 1 into a 0, carrying to the next bit.
2. Change the first 0 into a 1, and stop.

For example,

$$0111 \longrightarrow 1000$$

changes four bits, whereas

$$1000 \longrightarrow 1001$$

changes only one.

We measure the cost of an increment by the number of bits it changes.

A worst-case bound is not the whole story. One increment can change all $b = O(\log(T+1))$ bits. Multiplying this worst-case cost by the number of increments gives the valid upper bound

$$O(T \log(T+1)).$$

However, this overestimates the total cost: increments with long carry chains are rare.

To obtain a sharper bound, we count how often each bit changes over the entire sequence.

Number the bits $0, 1, \ldots, b-1$, starting with the least significant bit. Then:

- Bit 0 changes on every increment.
- Bit 1 changes on every second increment.
- Bit 2 changes on every fourth increment.
- In general, bit i changes on every 2^i-th increment.

Consequently, the total number of bit changes is

$$\begin{aligned}\sum_{i=0}^{b-1} \left\lfloor \frac{T}{2^i} \right\rfloor &\leq T \sum_{i=0}^{b-1} \frac{1}{2^i} \\ &< T \sum_{i=0}^{\infty} \frac{1}{2^i} \\ &= 2T.\end{aligned}$$

Thus all T increments together cost $O(T)$, even though a single increment can cost $\Theta(\log(T+1))$.

What amortized analysis means. We say that incrementing the counter has $O(1)$ *amortized cost*: the total cost of T increments, starting from zero, is $O(T)$.

This is not a probabilistic statement. We are not assuming that the counter values are random, nor that every increment is cheap. Instead, we prove a bound on the total cost of the entire sequence.

The useful accounting principle is

$$\boxed{\text{Total cost} = \sum_{\text{levels}} \big(\text{cost per event}\big)\big(\text{number of events}\big).}$$

For the counter, a level-i event is a change to bit i. It costs 1 and occurs at most $T/2^i$ times.

Connecting these examples to tape simulation. As we will see in Section 1.4.5, the universal Turing machine simulation uses buffers whose sizes grow geometrically:

$$2, \quad 4, \quad 8, \quad 16, \quad \ldots.$$

As in repeated squaring, there are only $O(\log(T+1))$ relevant levels during a computation of T steps.

As with a binary counter, activity at high levels is infrequent. The buffer construction will ensure that reorganizing level i costs $O(2^i)$, but occurs only $O(T/2^i)$ times. Therefore, each level contributes at most

$$O(2^i) \cdot O(T/2^i) = O(T)$$

work. Summing over all relevant levels gives

$$O(T) \cdot O(\log(T+1)) = O(T \log(T+1)).$$

Notice the difference from the binary counter: changing a high-order bit costs only 1, whereas reorganizing a large buffer costs proportionally to its size. This is why the counter has $O(T)$ total cost, while the buffer simulation has an $O(T \log(T+1))$ bound.

The proof that follows makes this accounting argument precise by constructing the buffers and showing why a large rearrangement cannot happen again until sufficiently many simulated head moves have occurred.

1.4.5 *Proof of Theorem 1.4.2 (Part 2/2)

Nothing from Section 1.4.5 will be tested on quizzes or exams!

We now proceed to the real proof of Theorem 1.4.2.

We use the input/output-tape convention of the theorem: U simulates M's input and output tapes directly. The main task is to simulate M's k work tapes using a fixed number of work tapes.

1.4.5.1 The main idea: move the contents, not the simulated heads

Let Γ be the work-tape alphabet of M. We view one work tape of U as having k parallel *tracks*. A cell of this tape stores one symbol for each track, so it represents a tuple of k symbols.

These tracks are not independent physical tapes: they share a single physical head. Nevertheless, we will arrange that, between simulated steps, the symbol under each of M's work-tape heads is stored at position 0 of its corresponding track.

Thus, when a simulated head moves, we move the represented tape contents relative to position 0:

- A simulated move to the right brings the next symbol on the right into position 0.
- A simulated move to the left brings the next symbol on the left into position 0.

If we physically shifted an entire track after every move, one simulated step could cost $O(T)$, giving an $O(T^2)$ simulation. Instead, we introduce unused space into each track and use it as a collection of buffers.

Implementation convention. For readability, we describe cells as storing tuples from an alphabet that depends on M. The actual universal machine has a fixed alphabet: it encodes each such tuple by a block of fixed-alphabet symbols. For a fixed M, the block length is constant, so this changes the running time by only a factor depending on M.

1.4.5.2 Buffers and their meaning

Introduce a padding symbol $\diamond$, distinct from every symbol of Γ, including the ordinary tape blank.

The distinction is important:

- An ordinary blank represents an actual cell of M's tape.
- A $\diamond$ represents unused storage and is ignored.

For example, the strings

$$010, \qquad 0 \diamond 10, \qquad 0 \diamond \diamond 1 \diamond 0$$

represent the same sequence of simulated tape symbols.

We use a two-sided physical tape, with positions indexed by the integers. Position 0 is the *home position*. For each $i \geq 1$, define a right zone and a left zone by

$$R_i = \{2^i - 1, \ldots, 2^{i+1} - 2\}, \qquad L_i = \{-p : p \in R_i\}.$$

Both zones have 2^i cells. The layout is therefore

$$\cdots \; L_3 \; L_2 \; L_1 \; \boxed{0} \; R_1 \; R_2 \; R_3 \; \cdots,$$

where the zone sizes, moving outward from the center, are

$$2, \; 4, \; 8, \; 16, \; \ldots.$$

Starting the indexing at 1 ensures that every zone has an integer half-capacity.

For each track separately, we maintain these invariants:

1. Position 0 contains the symbol currently under the corresponding head of M.
2. Reading the track and ignoring all $\diamond$'s gives the simulated tape contents, in the correct order.
3. Each zone of size 2^i is either empty, half full, or full of non-padding symbols.
4. The total number of non-padding symbols in L_i and R_i is exactly 2^i.

The last two conditions mean that the occupancies of L_i and R_i must be one of

$$(0, 2^i), \qquad (2^{i-1}, 2^{i-1}), \qquad (2^i, 0).$$

Initially, every zone is conceptually half full of ordinary blank symbols, and position 0 also contains a blank. There is no need to initialize infinitely many cells: a zone is initialized only when it is first reached. Beyond the initialized region, all simulated symbols are still blank.

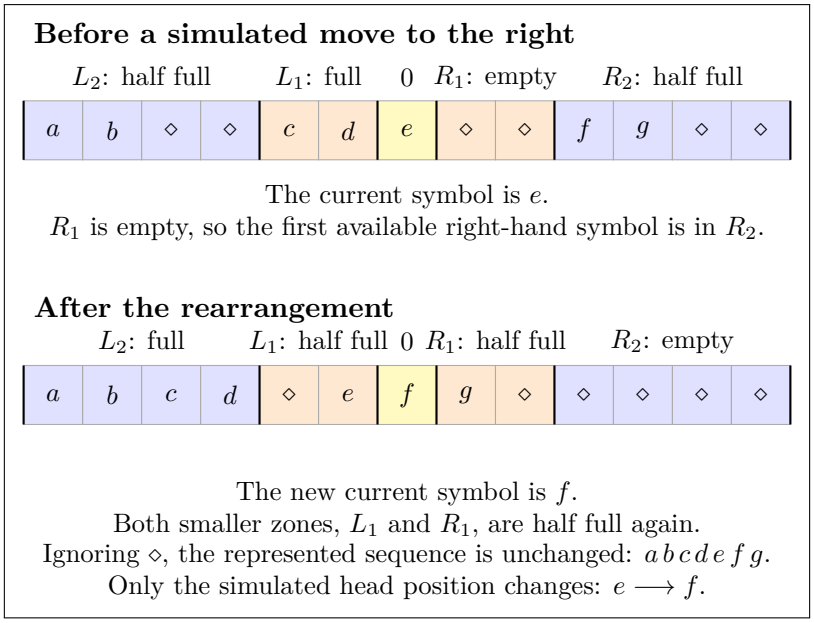


Figure 1.5: A rightward head move on one track, assuming the transition does not change the symbol e. The simulator brings f to position 0, places g in R_1, moves c, d into L_2, and leaves e in L_1. The symbol $\diamond$ denotes unused storage, not an ordinary tape blank. Only zones through level 2 are rearranged; all farther zones are unchanged.

1.4.5.3 Simulating one transition

At the beginning of a simulated step, U's physical head is at position 0. It reads the tuple of current work-tape symbols, consults M's transition table, and writes the updated symbols at position 0.

It then handles the prescribed head movement on each track separately. A stationary head requires no rearrangement. We explain a move to the right (see also Figure 1.5 for an illustration); a move to the left is symmetric.

1.4.5.4 Simulating a move to the right

A move to the right must bring the nearest represented symbol on the right into position 0, while placing the old position-0 symbol on the left.

Find the smallest i such that R_i is nonempty. Then

$$R_1, \ldots, R_{i-1} \quad \text{are empty,}$$

and, by the paired-occupancy invariant,

$$L_1, \ldots, L_{i-1} \quad \text{are full.}$$

Write

$$h = 2^{i-1}.$$

The total capacity of the smaller zones on either side is

$$\sum_{j=1}^{i-1} 2^j = 2^i - 2 = 2h - 2,$$

and their total half-capacity is

$$\sum_{j=1}^{i-1} 2^{j-1} = h - 1.$$

We now rearrange only

$$L_i, \ldots, L_1,\ 0,\ R_1, \ldots, R_i.$$

Right side. Take the first h non-padding symbols of R_i, in order starting from the end nearest position 0.

- Put the first symbol at position 0.
- Distribute the remaining $h - 1$ symbols among $R_1, \ldots, R_{i-1}$, making each of these zones half full.

Preserve the order of the represented symbols. The occupancy of R_i decreases by h.

Left side. The smaller left zones contain $2h - 2$ symbols. Together with the old symbol at position 0, there are $2h - 1$ symbols to place on the left.

- Move the h symbols farthest from the new head position into L_i, placing them after its existing symbols in left-to-right order.
- Distribute the remaining $h - 1$ symbols among $L_1, \ldots, L_{i-1}$, making each of these zones half full.

Again, preserve the order of all represented symbols. The occupancy of L_i increases by h.

Why the invariants are preserved. Before the rearrangement, the occupancies of (L_i, R_i) were either

$$(h, h) \quad \text{or} \quad (0, 2h),$$

because R_i was nonempty. Afterward, they are respectively

$$(2h, 0) \quad \text{or} \quad (h, h).$$

Thus the allowed occupancies and their sum are preserved.

All smaller zones are now half full. Position 0 contains the next simulated tape symbol, and the old position-0 symbol is immediately to its left in the represented sequence. Consequently, this rearrangement correctly simulates one rightward head move.

For $i = 1$, there are no smaller zones. The operation simply transfers the old central symbol into L_1 and the nearest symbol of R_1 into the center.

1.4.5.5 The cost of one rearrangement

Call an operation whose outermost affected zones are L_i, R_i a *level-i rearrangement*.

Its entire affected region has length

$$1 + 2\sum_{j=1}^{i} 2^j = O(2^i).$$

Using a scratch tape, U can copy the represented symbols in order and redistribute them with a constant number of scans of this region. Zone boundaries and half-capacities can be marked on auxiliary tracks.

Therefore, a level-i rearrangement costs

$$O_M(2^i).$$

This includes returning the physical head to position 0. Initializing a newly reached zone also fits within this bound.

The crucial point is that expensive rearrangements occur rarely.

1.4.5.6 Why large rearrangements are rare

Fix one track and a level i. Consider any rearrangement that reaches level i or farther.

Immediately afterward, every smaller zone

$$L_1, R_1, \ldots, L_{i-1}, R_{i-1}$$

is half full. Hence the smaller zones contain exactly

$$s_i = \sum_{j=1}^{i-1} 2^{j-1} = 2^{i-1} - 1$$

represented symbols on each side of position 0.

Until another rearrangement reaches level i, each rightward move decreases the number of symbols in the smaller right zones by one, and each leftward move increases it by one. The analogous statement holds on the left.

To reach level i again, all smaller zones on one side must first be empty. Starting with s_i symbols on each side, this requires at least s_i moves, followed by the move that actually reaches level i. Thus two rearrangements reaching level i or farther are separated by at least

$$s_i + 1 = 2^{i-1}$$

head moves.

The same lower bound applies before the first such rearrangement, because initially all zones are half full.

Since a work-tape head makes at most T moves during T steps, the number of rearrangements reaching level i or farther is at most

$$\left\lfloor \frac{T}{2^{i-1}} \right\rfloor.$$

In particular, this also bounds the number of level-i rearrangements.

Moreover, a level can be reached only if $2^{i-1} \leq T$. Thus the largest possible level is

$$1 + \lfloor \log_2 T \rfloor.$$

1.4.5.7 Summing the costs

For one track, the total rearrangement time is at most

$$\begin{aligned}\sum_{i=1}^{1+\lfloor \log_2 T \rfloor} O_M(2^i) \left\lfloor \frac{T}{2^{i-1}} \right\rfloor &\le O_M\left(\sum_{i=1}^{1+\lfloor \log_2 T \rfloor} 2^i \frac{T}{2^{i-1}} \right) \\ &= O_M\left(\sum_{i=1}^{1+\lfloor \log_2 T \rfloor} 2T \right) \\ &= O_M\big(T \log(T+1)\big).\end{aligned}$$

There are k tracks, where k depends only on M, so their combined cost has the same asymptotic bound. Reading the transition table, updating the state, and handling the input and output tapes add only $O_M(T)$ time.

Therefore U correctly simulates M within

$$O_M\big(T \log(T+1)\big)$$

steps.

The idea to remember. A buffer at level i costs $O(2^i)$ to reorganize, but it is involved only once per $\Omega(2^i)$ simulated head moves. Each level therefore contributes only $O(T)$ total work. There are $O(\log T)$ levels, giving $O(T \log T)$ time overall.

This finishes the proof of Theorem 1.4.2.

Takeaway

A single fixed machine can simulate every other Turing machine with only modest overhead. Thus, computational universality does not require building a different physical device for every algorithm; the algorithm itself can be supplied as encoded data.

1.5 The Church–Turing Thesis

We have shown that several formal models can simulate one another, but a broader question remains: does the Turing-machine model capture every computation that could be carried out by any effective mechanical procedure? The Church–Turing thesis proposes that it does.

1.5.1 The Standard Church–Turing Thesis

Church–Turing thesis: Every function that can be computed by a physically realizable computational device can be computed by a Turing machine.

The phrase *a physically realizable computational device* refers informally to a finite collection of unambiguous rules that can be followed step by step without requiring creativity, insight, or infinitely precise information.

Two important requirements of an effective mechanical procedure are *precision* and *executability*.

- *Precision:* Every instruction must specify exactly what is to be done. For example, a cooking recipe may instruct the reader to add "a pinch of salt" or "sugar to taste." Such instructions may be perfectly adequate for cooking, but they are not mechanically precise:

different people may interpret them differently. By contrast, an instruction such as "add 5 grams of salt" specifies a definite quantity.

- *Executability:* Every instruction must also describe an operation that can actually be carried out by the intended computational agent. Consider the familiar joke:

  > *How do you put an elephant into a refrigerator?*
  > Answer: In three steps: (1) Open the door, (2) put the elephant inside, and (3) close the door.

  The instructions are short and appear unambiguous, but the crucial step—putting the elephant into the refrigerator—may be physically impossible. Merely describing an action does not make that action executable.

Thus, an effective mechanical procedure must consist of instructions that are both sufficiently precise to determine each step uniquely and sufficiently elementary to be carried out mechanically.

It is only a conjecture! The Church–Turing thesis is not a theorem. One side of the thesis refers to the informal notion of an effective procedure, so the statement cannot be proved using only formal mathematics. It is a hypothesis connecting an informal concept to a formal model.

Evidence for the thesis includes the following:

- Many independently proposed formal models of computation, including Turing machines, lambda calculus, and general recursive functions, define the same class of computable functions.
- Standard programming languages can simulate Turing machines, and Turing machines can simulate programs written in standard programming languages.
- Natural changes to the details of the Turing-machine model do not change the class of computable functions.
- No generally accepted effective procedure has been discovered that computes a function uncomputable by a Turing machine.

Implications for the limits of computation

If we accept the standard Church–Turing thesis, then a proof that no Turing machine computes a particular function establishes a fundamental limit. The conclusion is not merely that present computers lack sufficient memory or that current programmers have not found the correct algorithm. Rather, no algorithmic procedure can compute that function on every input.

This distinction will become important when we study undecidability. Some problems are difficult because known algorithms use many resources. Other problems are uncomputable: no Turing-machine algorithm exists at all.

1.5.2 The Extended Church–Turing Thesis

The standard thesis concerns what is computable, without requiring the simulation to be efficient. Complexity theory motivates a stronger statement.

Extended Church–Turing thesis: Every computation carried out by a physically realizable computational device can be simulated efficiently, usually meaning with at most polynomial overhead, by a probabilistic Turing machine.

The addition of "efficiently" is crucial. An exponential-time simulation still supports the standard Church–Turing thesis, because the same function remains computable. It may nevertheless contradict the extended thesis because exponential overhead is not considered efficient.

Thesis	**Main question**	**Required simulation**
Standard	What can be computed?	Any finite overhead is allowed
Extended	What can be computed efficiently?	Polynomial overhead

Table 1.2: The standard and extended Church–Turing theses make different claims. The extended thesis is strictly stronger because it imposes an efficiency requirement.

1.5.3 Quantum Computing and the Extended Thesis

Quantum computers are physical models whose evolution is governed by quantum mechanics. They can be formalized by quantum circuits or quantum Turing machines.

A classical Turing machine can simulate any finite quantum computation to any required finite precision. The classical simulation may, however, require exponential time because a general state of m quantum bits is described using 2^m complex amplitudes.

Let:

- BPP denote the class of decision problems solvable in polynomial time by a "probabilistic" (we will discuss this in later chapters) classical Turing machine with bounded error; and
- BQP denote the class of decision problems solvable in polynomial time by a quantum computer with bounded error.

If quantum computers can efficiently solve a problem that no probabilistic classical Turing machine can solve efficiently, then

$$\mathsf{BQP} \not\subseteq \mathsf{BPP}.$$

Such a separation would contradict the classical probabilistic version of the extended Church–Turing thesis.

It has not been proved that $\mathsf{BQP} \not\subseteq \mathsf{BPP}$. Quantum algorithms provide strong evidence that quantum computation may violate the extended Church–Turing thesis, but this conclusion remains conditional on unresolved complexity-theoretic questions.

Quantum computing does not, according to the standard formal models, violate the standard Church–Turing thesis. A quantum computation can be simulated by a classical Turing machine, although possibly with exponential overhead. It changes our understanding of efficient computation, not the class of functions that are computable in principle.

Remark 1.5.1 (What Quantum Computing Challenges)**.** *Quantum computing challenges the claim that every reasonable physical computation has an efficient classical simulation. It does not provide a method for solving Turing-uncomputable problems. Thus, it is evidence against some formulations of the extended thesis, not against the standard thesis.*

1.5.4 The Turing Machine as a Final Model

If we accept the standard Church–Turing thesis, then introducing a more realistic programming language, a different processor, more tapes, random-access memory, parallel hardware, or quantum mechanics does not enlarge the class of computable functions beyond Turing computability.

This does not mean that all models are equally convenient or equally efficient:

- Different programming languages may make different algorithms easier to express.
- Different machine models may change running time by polynomial or even exponential factors.
- Randomness, parallelism, and quantum computation may change which problems can be solved efficiently.
- Restricted models such as finite automata remain valuable when limited memory is part of the problem being studied.

The main point is more fundamental: under the standard Church–Turing thesis, changing to another physically realizable model does not allow us to compute a genuinely uncomputable function.

If the standard Church-Turing thesis is correct, then the Turing machine is an eventual final model of computability: future models may alter convenience or efficiency, but they will not enlarge the set of functions that are computable by effective procedures.

1.6 Closing Remarks

We emphasize that Turing machines are not the only mathematical model of computation. Other models include the lambda calculus, recursive functions, register machines, and various idealized programming languages. Although these models describe computation in different ways, they are *equivalent* at the level of computability: each can compute exactly the functions computable by a Turing machine. Thus, the Turing machine should not be regarded as the uniquely correct description of computation, but as one particularly useful formalization among several equivalent ones.

We focus on Turing machines because their operation is concrete and easy to visualize: a machine reads and writes symbols, moves its tape heads, and changes its finite state. These elementary actions make it possible to describe algorithms precisely and to measure resources such as running time and memory usage. Moreover, the distinction between the finite control and the unbounded tape closely reflects the familiar distinction between a fixed program and its working memory. Turing machines therefore provide a simple, rigorous, and convenient foundation for studying computability and computational complexity, even though other equivalent models could serve the same fundamental purpose.

We also emphasize that we do not expect you to memorize every detail of the formal definition of a Turing machine or its transition rules. Rather, the purpose of studying Turing machines is to see how computation can be described with mathematical precision by reducing it to a small collection of elementary operations. Much like building with LEGO bricks, each natural step of an algorithm can be decomposed into finitely many simple Turing-machine steps. Despite its simplicity, this model has proved remarkably robust: every standard, reasonable model of general-purpose computation developed so far can compute the same class of functions as a Turing machine. This observation provides the main evidence for the Church-Turing thesis, which asserts that anything computable by an effective algorithm can be computed by a Turing machine. Thus, Turing machines give us a simple and implementation-independent foundation for reasoning rigorously about what can be computed and how efficiently it can be computed.

Chapter 2
Deterministic Polynomial Time: P

Our goal in this chapter is to make precise one of the central ideas of computational complexity: a problem is efficiently solvable if it has a polynomial-time algorithm. Building on our computational model and the notion of running time, we will define the class P and establish that several graph and number-theoretic decision problems belong to it.

We will then examine why complexity theory places so much emphasis on decision problems, even though applications often ask us to find actual solutions. For many natural problems, suitable decision queries allow us to construct a solution. However, this relationship depends on exactly which decision problem we are allowed to ask.

2.1 Polynomial Time and the Class P

Let us begin by connecting running-time analysis to a class of decision problems. The main issues are the choice of polynomial time as a benchmark, the role of input encodings, and the requirement that the algorithm be deterministic.

2.1.1 Why Polynomial Time?

A useful dividing line

An algorithm runs in polynomial time if its worst-case running time is bounded by a polynomial in the input length. Typical polynomial bounds include $O(n)$, $O(n^2)$, and $O(n^5)$, where n denotes the number of symbols in the input.

Polynomial and exponential functions behave very differently as the input length grows. For example, when $n = 100$,

$$n^3 = 1{,}000{,}000, \qquad 2^n \approx 1.27 \times 10^{30}.$$

This comparison does not tell us the actual running time of a particular implementation. It illustrates why replacing exhaustive search by a polynomial-time method can fundamentally change which input sizes are accessible.

Exponential running times often arise from searching through all possible solutions. If an instance involves n independent binary choices, there are 2^n possible combinations. An efficient algorithm must frequently avoid examining them one by one.

Why not choose a particular polynomial?

Why do we group n^2 and n^5 together, instead of defining efficiency using one particular exponent?

One reason is that polynomial time is stable under several basic operations.

- Running polynomially many polynomial-time subroutines still takes polynomial time, provided their inputs have polynomial length.
- A polynomial slowdown in simulating one computational model by another preserves polynomial-time solvability.

- Changing between encodings that can be translated into each other in polynomial time preserves polynomial-time solvability.

For example, standard deterministic single-tape and multitape Turing machines can simulate one another with polynomial overhead. Their detailed running-time bounds may differ, but they define the same class of polynomial-time decision problems.

Polynomial time is a theoretical benchmark, not a guarantee of practical speed. A running time of n^{100} is polynomial but usually impractical. Conversely, an exponential-time algorithm can be useful on small inputs or restricted families of instances. We use polynomial time to study a robust structural question, not to replace detailed algorithm analysis.

2.1.2 The Formal Definition

Recall that a *decision problem* asks for a yes/no answer. We represent such a problem by a language (see Definition 1.2.4): the set of encodings of its yes-instances.

Definition 2.1.1 (The class P)**.** *The class* P *consists of all languages decidable in polynomial time by a deterministic Turing machine. Equivalently,*

$$\mathsf{P} := \bigcup_{k \geq 1} \mathrm{TIME}(n^k), \tag{2.1}$$

where k *ranges over positive integers and* $\mathrm{TIME}(n^k)$ *denotes the languages decidable by deterministic machines in time* $O(n^k)$*.* ◊

Let us unpack Definition 2.1.1. A language L belongs to P if there exist a deterministic machine M, a constant $C > 0$, and a fixed positive integer k such that, for every input string x,

- M accepts x if and only if $x \in L$;
- M halts within $C(|x| + 1)^k$ steps.

Here $|x|$ is the length of x. The constants C and k may depend on the machine, but they must not depend on the particular input.

There must be one algorithm that works for every input. We cannot choose a different algorithm or a different exponent separately for each instance.

Example 2.1.1 (Recognizing a polynomial bound)**.** *Suppose a deterministic decider has running time*

$$T(n) \leq 5n^3 + 12n + 7$$

for inputs of length n*. For* $n \geq 1$*,*

$$\begin{aligned} T(n) &\leq 5n^3 + 12n^3 + 7n^3 \\ &= 24n^3. \end{aligned}$$

The finitely many shorter inputs can be absorbed into the constant in the running-time bound. Therefore, the language decided by this machine belongs to P*.*

The class P contains decision problems for which there exists a deterministic algorithm that is correct on every input and runs in worst-case polynomial time.

Problems versus algorithms. A language is not itself deterministic or randomized; those words describe algorithms. Saying that $L \in \mathsf{P}$ means that at least one deterministic polynomial-time decider for L exists. The same language might also have other algorithms, including slower algorithms or randomized algorithms.

2.1.3 Input Encodings and Bit Complexity

An algorithm's running time is measured against the length of its encoded input, not against the numerical value represented by that input. Consequently, changing the encoding can change whether a running-time bound is polynomial in the input length.

To see this, consider two encodings of the same positive integer N:

- **Binary encoding:** Write N in base two. The input length is

$$\ell(N) \coloneqq \lfloor \log_2 N \rfloor + 1. \tag{2.2}$$

- **Unary encoding:** Write N as a string of N copies of the symbol 1. The input length is N.

Both strings represent the same integer, but their lengths can differ exponentially.

Example 2.1.2 (The same loop under two encodings)**.** *Consider an algorithm that performs a loop exactly N times. For*

$$N = 2^{20} = 1{,}048{,}576,$$

the binary input has only 21 *symbols, whereas the unary input has* 1,048,576 *symbols. The loop performs the same number of iterations in either case, but we compare that number against very different input lengths.*

More generally, let n denote the length of the chosen encoding.

- *With **unary encoding**, $n = N$, so the loop performs n iterations. If each iteration and the remaining computation take polynomial time in n, the algorithm runs in polynomial time.*
- *With **binary encoding**, $n = \lfloor \log_2 N \rfloor + 1$, so*

$$2^{n-1} \le N < 2^n.$$

The loop therefore performs exponentially many iterations in n. Even if each iteration takes only one step, this algorithm is not polynomial-time.

Thus, a loop that is polynomially bounded for unary input can be exponentially long for binary input.

Changing to unary does not make the loop perform less work. It makes the input much longer, so the benchmark against which we measure that work changes. Nor does this example rule out a different, faster algorithm for the binary-encoded problem.

Which encoding changes preserve polynomial time? Polynomial-time solvability is preserved when two encodings can be translated into each other in polynomial time. Binary and unary do not satisfy this condition: converting a binary integer N to unary requires writing N symbols, which can take exponential time in the binary input length.

For graphs, we use standard explicit representations, such as adjacency lists that include all vertices or adjacency matrices. These representations can be converted into one another in polynomial time, so choosing between them does not change whether a graph decision problem

belongs to P. In both representations, the numbers of vertices and edges are bounded by polynomials in the input length.

We do not use compressed graph representations that can describe exponentially many vertices with a short string. Such an encoding could change the complexity measured in terms of input length, just as binary encoding does in comparison with unary encoding.

2.2 Representative Decision Problems in P

We now apply the definition to three problems: relative primality, directed reachability, and bipartite matching with a size threshold. The algorithms use different ideas, but the proof structure is the same: correctness together with a polynomial running-time bound.

2.2.1 Relative Primality

For positive integers d and x, we write $d \mid x$ if $x = dq$ for some integer q. In this case, d is a *divisor* of x.

The greatest common divisor $\mathsf{gcd}(x, y)$ is the largest positive integer that divides both x and y. The integers x and y are *relatively prime* if $\mathsf{gcd}(x, y) = 1$.

Relative primality is a property of a pair of integers. Neither integer must itself be prime: for example, 10 and 21 are relatively prime.

Definition 2.2.1 (The language RELPRIME)**.** *For positive integers encoded in binary, define*

$$\text{RELPRIME} \coloneqq \{\langle x, y\rangle : \mathsf{gcd}(x, y) = 1\}. \tag{2.3}$$

◊

Testing every possible common divisor would take too long in the worst case, because the values of x and y can be exponential in their bit lengths. Instead, we use Euclid's algorithm.

Lemma 2.2.1 (The Euclidean identity)**.** *For integers $a \geq 0$ and $b > 0$,*

$$\mathsf{gcd}(a, b) = \mathsf{gcd}(b, a \bmod b). \tag{2.4}$$

Here $a \bmod b$ is the remainder when a is divided by b. ◊

Proof. Write $a = qb + r$, where $0 \leq r < b$.

If an integer divides both a and b, it divides $a - qb = r$. Conversely, if it divides both b and r, it divides $qb + r = a$. Thus, the pairs (a, b) and (b, r) have exactly the same common divisors, so their greatest common divisors are equal. ■

Algorithm 2.2.1: Euclid's algorithm

On input positive integers x and y:

1. Set $a \coloneqq x$ and $b \coloneqq y$. If $a < b$, exchange them.
2. While $b \neq 0$, replace the pair (a, b) simultaneously by
$$(a, b) \coloneqq (b, a \bmod b).$$
3. Output a.

Theorem 2.2.1. RELPRIME $\in \mathsf{P}$. ◊

Proof. Run Algorithm 2.2.1 and accept exactly when its output is 1.

By Lemma 2.2.1, every iteration preserves the gcd. The second component strictly decreases until it reaches zero. Since $\mathsf{gcd}(a, 0) = a$ for $a > 0$, the algorithm outputs the original gcd. This proves correctness.

To bound the number of iterations, consider a state with $a \geq b > 0$. Its remainder must satisfy:

$$a \bmod b < \frac{a}{2}.$$

This is because:

- if $b \leq a/2$, the remainder is smaller than b and hence smaller than $a/2$.
- If $b > a/2$, the quotient is 1, so the remainder is $a - b < a/2$.

After two iterations, if the algorithm has not already terminated, the first component is this remainder. Thus, the larger component decreases by at least a factor of two every two iterations. The total number of iterations is $O(\log \max\{x, y\})$, which is $O(n)$ for total input length n.

All intermediate values have at most n bits. Binary division with remainder can be performed in polynomial time; for example, a grade-school implementation uses $O(n^2)$ bit operations per division. This gives a sufficient polynomial bound of $O(n^3)$ for the complete computation. ■

Example 2.2.1 (A yes-instance and a no-instance)**.** *For* $(x, y) = (55, 34)$*, the successive divisions are*

$$\begin{aligned}
55 &= 1 \cdot 34 + 21, \\
34 &= 1 \cdot 21 + 13, \\
21 &= 1 \cdot 13 + 8, \\
13 &= 1 \cdot 8 + 5, \\
8 &= 1 \cdot 5 + 3, \\
5 &= 1 \cdot 3 + 2, \\
3 &= 1 \cdot 2 + 1, \\
2 &= 2 \cdot 1 + 0.
\end{aligned}$$

The last nonzero remainder is 1*, so* $\mathsf{gcd}(55, 34) = 1$ *and the decision algorithm accepts.*

For $(x, y) = (24, 18)$*,*

$$\begin{aligned}
24 &= 1 \cdot 18 + 6, \\
18 &= 3 \cdot 6 + 0.
\end{aligned}$$

The gcd is 6*, so the decision algorithm rejects.*

Takeaway

Computing a gcd is a function problem: the output is an integer. The language RELPRIME is a decision problem: the output is yes or no. We established its membership in P by using the numerical computation as a subroutine.

2.2.2 Directed Reachability

A directed graph $G = (V, E)$ consists of a finite vertex set V and a set of ordered pairs $E \subseteq V \times V$. An edge (u, w) allows movement from u to w, but not necessarily from w to u.

A directed path from s to t is a sequence of distinct vertices

$$s = u_0, u_1, \ldots, u_r = t$$

such that $(u_{i-1}, u_i) \in E$ for every $1 \leq i \leq r$. Its length is r, the number of edges. We allow a path of length zero from a vertex to itself.

Definition 2.2.2 (The language PATH)**.** *The directed reachability language is*

$$\text{PATH} \coloneqq \left\{ \langle G, s, t \rangle : \begin{array}{l} G \text{ is a directed graph, } s, t \in V(G), \\ G \text{ has a directed path from } s \text{ to } t \end{array} \right\}. \tag{2.5}$$

◊

Reachability models questions such as whether one state can be reached from another by a sequence of allowed transitions.

Why not enumerate paths? If a directed route repeats a vertex, we can remove the portion between two occurrences of that vertex. Repeating this process produces a path with no repeated vertices. Thus, whenever t is reachable from s, there is a path of length at most $v - 1$.

However, even sequences of at most v vertices can form a very large (i.e., exponential) search space. The marking algorithm avoids enumerating these sequences.

Algorithm 2.2.2: Directed reachability by marking

On input $\langle G, s, t \rangle$:

1. Set $R \coloneqq \{s\}$.
2. Repeat the following:
   (a) Set $R' \coloneqq R$.
   (b) Scan every edge $(u, w) \in E$. If $u \in R$, add w to R'.
   (c) If $R' = R$, stop the repetition.
   (d) Otherwise, replace R by R'.
3. Accept if $t \in R$; otherwise, reject.

During a scan, membership tests use the old set R. Newly added vertices are used as starting points only in the next scan.

Theorem 2.2.2. PATH $\in \mathsf{P}$. ◊

Proof. The correctness of Algorithm 2.2.2 is obvious.

For the running time, write $v = |V|$ and $m = |E|$. Associate a Boolean mark with each vertex, indicating whether it belongs to R. Reading or setting a vertex's mark takes constant time. Initially, setting these marks takes $O(v)$ operations.

During a scan, the algorithm examines each edge (u, w) at most once. For each edge, it reads the marks of u and w and, if necessary, sets the mark of w and records that a vertex was added. Thus, each edge requires only a constant number of basic operations. Using adjacency lists, traversing the vertices and their outgoing edges therefore costs $O(v + m)$ operations per scan.

Each scan either marks at least one new vertex or terminates the repetition. Since s is marked initially and there are only $v-1$ other vertices, at most $v-1$ scans add vertices, followed

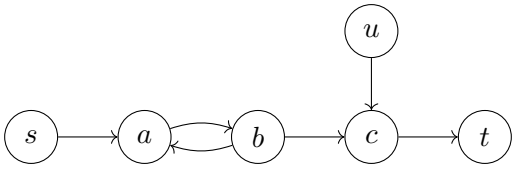


Figure 2.1: A directed reachability instance. The cycle between a and b does not prevent the marking algorithm from terminating. The vertex u is not reachable from s.

by at most one final scan. Hence the total number of operations, including initialization, is

$$O(v) + v \cdot O(v + m) = O\big(v(v + m)\big).$$

The final test of whether t is marked takes constant time. Because v and m are polynomially bounded in the length of the explicit graph encoding, the algorithm runs in polynomial time. ■

Example 2.2.2 (Tracing reachability)**.** *Consider the directed graph in Figure 2.1, whose edge set is*

$$E = \{(s, a), (a, b), (b, a), (b, c), (c, t), (u, c)\}.$$

Let R_i be the marked set after i productive scans. The algorithm produces

$$\begin{aligned} R_0 &= \{s\}, \\ R_1 &= \{s, a\}, \\ R_2 &= \{s, a, b\}, \\ R_3 &= \{s, a, b, c\}, \\ R_4 &= \{s, a, b, c, t\}. \end{aligned}$$

The next scan adds nothing.

The instance $\langle G, s, t\rangle$ is accepted. The instance $\langle G, s, u\rangle$ is rejected: although u has an edge leading to c, there is no directed path from s to u.

Connection to breadth-first search

Breadth-first search explores the same reachable set using a queue. With adjacency lists, it requires $O(v + m)$ elementary graph operations. This familiar bound need not be the exact running time on a single-tape Turing machine; polynomial simulation overhead is enough to establish membership in P.

Takeaway

An exponentially large collection of candidate paths does not imply that reachability requires exponential time. Marking vertices captures the information we need without enumerating all paths.

2.2.3 Bipartite Matching with a Size Threshold

Suppose we have applicants and jobs. An applicant may be compatible with some jobs but not others. We want to assign applicants to jobs so that each applicant receives at most one job and each job is assigned to at most one applicant.

A *bipartite graph* represents this situation. Its vertices are partitioned into disjoint sets U and W, and every edge has one endpoint in each set. We write $G = (U, W, E)$, with an edge (u, w) representing compatibility.

A *matching* is a set of edges no two of which share an endpoint. The size of a matching is its number of edges.

Definition 2.2.3 (Bipartite matching decision problem)**.** *Define*

$$\text{BIPARTITE-MATCHING} \coloneqq \left\{ \langle G, k \rangle : \begin{array}{l} G = (U, W, E) \text{ is bipartite, } k \geq 0, \\ G \text{ has a matching of size at least } k \end{array} \right\}, \tag{2.6}$$

where k is encoded in binary and the partition (U, W) is included in the graph representation. ◊

Maximum versus maximal. A *maximum* matching has the largest possible size. A *maximal* matching is one to which no single edge can be added while retaining the matching property.

These notions are different. A maximal matching may fail to be maximum because increasing its size can require replacing some selected edges.

Example 2.2.3 (Why a greedy choice may need to be revised)**.** *Let*

$$U = \{a, b\}, \qquad W = \{1, 2\}, \qquad E = \{(a, 1), (a, 2), (b, 1)\}.$$

If we first select $(a, 1)$, neither remaining edge can simply be added. Thus, $\{(a, 1)\}$ is maximal.

However,

$$\{(a, 2), (b, 1)\}$$

is a matching of size 2. The first matching is not maximum. For this graph, the threshold $k = 2$ gives a yes-instance, whereas $k = 3$ gives a no-instance.

Augmenting paths

Relative to a matching M, a vertex is *unmatched* if it is not incident to an edge of M.

An *augmenting path* is a simple path whose endpoints are unmatched and whose edges alternate between edges outside M and edges in M, starting and ending with edges outside M.

Then, we can characterize maximum matchings in terms of augmenting paths:

Theorem 2.2.3 (Berge's Theorem [Ber57])**.** *A matching is maximum if and only if it has no augmenting path.* ◊

Proof. We prove the two directions separately.

First, suppose that M has an augmenting path P. By definition, the endpoints of P are unmatched by M, and the edges of P alternate between edges not in M and edges in M. Since P starts and ends with edges outside M, it has one more edge outside M than inside M.

Now form a new set of edges M' by "flipping" the edges along P: remove from M all edges of P that are in M, and add all edges of P that are not in M. In symbols,

$$M' = M \triangle E(P),$$

where $\triangle$ denotes symmetric difference and $E(P)$ is the set of edges contained in P.

We claim that M' is again a matching. Indeed, every internal vertex of P was incident to exactly one selected edge before the flip, and it is still incident to exactly one selected edge after the flip. The two endpoints of P, which were unmatched by M, become matched after the flip. All vertices not on P are unaffected. Thus no vertex is incident to more than one edge of M', so M' is a matching.

Moreover, since P contains one more edge outside M than inside M, the flip increases the size of the matching by one:

$$|M'| = |M| + 1.$$

Therefore, if M has an augmenting path, then M is not maximum.

Conversely, suppose that M is not maximum. Then there exists a matching M^* with

$$|M^*| > |M|.$$

We compare M and M^* by looking at the graph whose edge set is their symmetric difference:

$$M \triangle M^* = (M \setminus M^*) \cup (M^* \setminus M).$$

That is, we keep exactly those edges that belong to one of the two matchings, but not both.

In this graph, every vertex has degree at most two. This is because a vertex is incident to at most one edge of M and at most one edge of M^*. Hence each connected component of this graph is either a path, a cycle, or an isolated vertex. Ignoring isolated vertices, every component is a path or a cycle.

Furthermore, along each nontrivial component, the edges must alternate between edges of M and edges of M^*. This follows from the matching property: two distinct edges of M cannot be adjacent, and two distinct edges of M^* cannot be adjacent.

Now consider the number of edges from M and from M^* in each component. Any cycle must contain the same number of edges from M and M^*, since the edges alternate around the cycle. A path may have the same number of edges from the two matchings, or it may have one more edge from one matching than from the other.

Since $|M^*| > |M|$, across all components there are more edges from M^* than from M. Therefore, at least one path component must contain more edges from M^* than from M.

Let P be such a path component. Because the edges of P alternate between M and M^*, and because P has more edges from M^* than from M, the path P must start and end with edges of M^*. Thus P has the form

$$M^*, M, M^*, M, \ldots, M^*.$$

It remains to check that the endpoints of P are unmatched by M. Consider one endpoint v of P. The unique edge of P incident to v lies in M^*. If v were also incident to an edge of M, then that M-edge would belong to the symmetric difference and would be adjacent to the M^*-edge at v. Hence it would also be part of the same component, contradicting the fact that v is an endpoint of the path. Therefore v is unmatched by M. The same argument applies to the other endpoint.

Thus P is a simple path whose endpoints are unmatched by M, and whose edges alternate between edges outside M and edges in M, starting and ending with edges outside M. Hence P is an augmenting path for M.

We have shown that if M is not maximum, then M has an augmenting path. Combining this with the first direction proves the theorem. ■

In a bipartite graph, an augmenting path can be found by a directed reachability search. Temporarily orient:

- edges outside M from U to W;
- edges in M from W to U.

Search from the unmatched vertices in U for an unmatched vertex in W. A directed path found this way is an augmenting path.

Algorithm 2.2.3: Deciding bipartite matching

On input $\langle G, k\rangle$, where $G = (U, W, E)$:

1. If $k = 0$, accept. If $k > \min\{|U|, |W|\}$, reject.
2. Initialize $M := \varnothing$.
3. While $|M| < k$:
   (a) Search for an augmenting path relative to M.
   (b) If no augmenting path exists, reject.
   (c) Otherwise, replace the matching edges along the path by its nonmatching edges.
4. Accept.

Theorem 2.2.4. BIPARTITE-MATCHING $\in$ P. ◊

Proof. Every augmentation preserves the matching property and increases the size by one. Thus, acceptance certifies the existence of a matching of size k.

If the algorithm rejects because no augmenting path exists, then Theorem 2.2.3 implies that the current matching is maximum. Its size is less than k, so rejection is correct.

There are at most $\min\{|U|, |W|\}$ augmentations. Each search is a polynomial-time graph search. In the usual adjacency-list model, each search and update uses $O(v + m)$ graph operations, giving $O(v(v + m))$ operations overall. Accounting for representation and simulation costs still gives a deterministic polynomial-time bound. ■

Returning to Example 2.2.3, start with $M = \{(a, 1)\}$. The path

$$b,\ 1,\ a,\ 2$$

is augmenting: its edges are outside M, inside M, and outside M, respectively. Removing $(a, 1)$ and inserting $(b, 1)$ and $(a, 2)$ produces the matching of size 2.

Further Discussion on Mathching Algorithms

The algorithm described above is the generic *augmenting-path algorithm* for bipartite matching. But improved algorithms exist, such as theHopcroft–Karp algorithm. The Hopcroft–Karp algorithm improves Algorithm 2.2.3 by proceeding in phases. In each phase, it finds a maximal collection of vertex-disjoint shortest augmenting paths and augments the matching along all of them simultaneously. Its running time is

$$O\big((|E| + |V|)\sqrt{|V|}\big),$$

which is commonly written as $O(|E|\sqrt{|V|})$ after isolated vertices have been discarded [HK73].

Polynomial-time matching algorithms are not limited to bipartite graphs. Both the decision and search versions of maximum matching are polynomial-time solvable for general graphs. Odd cycles create an additional difficulty because they give rise to structures called *blossoms*, which prevent the straightforward bipartite augmenting-path search from working. Edmonds's blossom algorithm handles this difficulty by contracting blossoms and finds a maximum matching in polynomial time [Edm65]. The later Micali–Vazirani algorithm finds a maximum matching in a general graph $G = (V, E)$ in $O\big(|E|\sqrt{|V|}\big)$ time [MV80].

2.3 Decision and Search

Applications often require an explicit solution rather than a yes/no answer. In this section, we formalize that distinction and show how a decision procedure can be used to construct a actual solution to the problem. We will use the bipartite matching problem (discussed in Section 2.2.3) as a running example.

The key point is not that the previously described matching algorithm already produces a matching. We will treat the decision procedure as a subroutine that reveals only one bit per query.

2.3.1 Witnesses and Search Problems

Two formulations of the same task

Recall that a *matching* in a graph $G = (V, E)$ is a subset $M \subseteq E$ such that no two edges in M share an endpoint. Equivalently, each vertex in V is incident to at most one edge in M. A matching is *perfect* if every vertex in V is incident to exactly one edge in M.

For a bipartite graph G, compare:

- **Decision:** Does G have a matching of size at least k?
- **Search:** Find such a matching, or report that none exists.

A matching demonstrating that the answer is yes is called a *witness*. For PATH, a witness is a path from s to t.

Definition 2.3.1 (Witness relation and associated search problem)**.** *A witness relation is a relation*

$$R \subseteq \{0,1\}^* \times \{0,1\}^*,$$

where $R(x, w)$ means that w is a valid solution for instance x.

In the examples considered here:

- *validity can be checked deterministically in time polynomial in $|x| + |w|$;*
- *there is a polynomial q such that every valid witness satisfies $|w| \leq q(|x|)$.*

The associated decision language is

$$L_R := \{x : \text{there exists } w \text{ such that } R(x, w)\}. \tag{2.7}$$

The associated search problem asks, on input x, for any w satisfying $R(x, w)$, or for the failure symbol $\perp$ if no such w exists. ◊

The symbol $\perp$ is not a witness. It means that the instance has no valid solution.

The polynomial bound on witness length matters because an algorithm needs time to write its output. We do not ask a polynomial-time algorithm to print an exponentially long object.

Example 2.3.1 (A matching witness)**.** *For the graph in Example 2.2.3, take*

$$x = \langle G, 2\rangle, \qquad w = \langle\{(a, 2), (b, 1)\}\rangle.$$

To verify $R(x, w)$, check that:

1. *both listed edges belong to G;*

2. *no two listed edges share an endpoint;*

3. *the list contains at least two edges.*

All three checks succeed, so w certifies that x is a yes-instance. A general matching witness contains at most $v/2$ edges and therefore has polynomial encoding length.

Search gives decision. A correct search algorithm immediately gives a decision algorithm: run it and accept precisely when it outputs a witness rather than $\perp$.

This argument assumes that the search algorithm halts and correctly reports nonexistence on no-instances. An algorithm guaranteed to work only when a solution exists does not automatically give a decider.

The converse requires an argument. A decision algorithm supplies only one bit. It does not directly reveal which edges form a matching or which vertices form a path. To recover a witness, we need a systematic way to ask additional decision questions.

2.3.2 Polynomial-Time Search-to-Decision Reductions

Using a decision procedure as an oracle

An *oracle* for a language L is an idealized subroutine that, on a query string y, answers whether $y \in L$.

The reduction must construct every query and process every answer. The oracle abstraction simply separates that work from the task of answering the decision question itself.

Definition 2.3.2 (Polynomial-time search-to-decision reduction)**.** *A polynomial-time search-to-decision reduction from a search problem to a decision language L is a deterministic algorithm that solves the search problem using an oracle for L, such that:*

- *the number of oracle calls is polynomial in the original input length;*
- *every oracle query has polynomial length;*
- *the computation outside the oracle calls takes polynomial time.*

Later queries may depend on earlier oracle answers. ◊

Such a reduction is a polynomial-time *Turing reduction*, also called an oracle reduction. It is not required to replace the entire task by a single decision instance. This distinction will matter when we study other types of reductions later.

Proposition 2.3.1 (Replacing an oracle by an algorithm)**.** *If a search problem has a polynomial-time search-to-decision reduction to a language $L \in \mathsf{P}$, then the search problem has a deterministic polynomial-time algorithm.*

Proof. Suppose the reduction makes at most $p(n)$ calls, each of length at most $q(n)$, and uses at most $p(n)$ steps outside those calls, for polynomials p and q.

Since $L \in \mathsf{P}$, its oracle can be replaced by a deterministic decider with running time at most $C(t+1)^d$ on queries of length t, for fixed C and d.

The resulting running time is bounded by

$$p(n) + p(n)\, C(q(n)+1)^d, \tag{2.8}$$

which is polynomial in n. Every oracle answer is reproduced exactly, so correctness is preserved. ■

Recovering a matching from decision queries

Suppose the only information our subroutine provides is

$$D(G,k) = \begin{cases} 1, & \text{if } G \text{ has a matching of size at least } k, \\ 0, & \text{otherwise.} \end{cases}$$

We are not allowed to inspect how D computes its answer.

To decide whether an edge (u, w) can be included in a matching of size r, delete its endpoints (and all edges incident to them) and ask whether the remaining graph has a matching of size at least $r - 1$.

Let $G - \{u, w\}$ denote the graph obtained by deleting u, w, **and all edges incident to them**. Then, it is obvious that

$$\begin{aligned} &\text{There is a matching of size at least } r \text{ containing } (u,w) \\ &\quad \Longleftrightarrow D(G - \{u,w\}, r-1) = 1. \end{aligned} \tag{2.9}$$

Algorithm 2.3.1: Finding a Size-k matching using a decision oracle

On input $\langle G, k\rangle$, where G is bipartite:

1. Query $D(G, k)$. If the answer is 0, output $\perp$ and halt.
2. Set
$$H \coloneqq G, \qquad M \coloneqq \varnothing, \qquad r \coloneqq k.$$
3. While $r > 0$:
   (a) Examine the edges $e = (u, w)$ of H in a fixed order. For each such edge, query
   $$D\big(H - \{u,w\}, r-1\big).$$
   (b) If the answer is 0, leave H, M, and r unchanged and continue to the next edge.
   (c) On the first answer of 1, add e to M, replace H by $H - \{u, w\}$, decrease r by one, and begin the next iteration of the while-loop.
4. Output M.

Lemma 2.3.1. *Algorithm 2.3.1 is a polynomial-time search-to-decision reduction for*

BIPARTITE-MATCHING.

◊

Proof. If the initial oracle query returns 0, then G has no matching of size at least k, so the algorithm correctly outputs $\perp$. Suppose, therefore, that the initial query returns 1.

We prove correctness using the following invariant, which holds at the beginning of every iteration of the while-loop:

1. M is a matching in the original graph G;

2. no vertex of H is an endpoint of an edge in M;

3. $|M| + r = k$;

4. H contains a matching of size at least r.

The invariant holds initially because $M = \varnothing$, $H = G$, $r = k$, and $D(G, k) = 1$.

We first prove that the invariant is preserved in each iteration. Suppose that the algorithm selects an edge $e = (u, w)$ after receiving a yes-answer. By Equation (2.9), the graph $H - \{u, w\}$ contains a matching of size at least $r - 1$. Moreover, the second invariant condition guarantees that e shares no endpoint with any edge already in M. Therefore, after adding e to M, replacing H by $H - \{u, w\}$, and decreasing r by one, all four conditions of the invariant continue to hold.

Next, we prove that the while-loop eventually terminates. Suppose that the invariant holds and that $r > 0$. By the fourth condition, H contains a matching N of size r. Let $e = (u, w)$ be any edge of N. Removing e from N leaves a matching of size $r - 1$, none of whose edges is incident to u or w. Consequently,

$$D\big(H - \{u, w\}, r - 1\big) = 1.$$

Thus, at least one edge examined during the current iteration produces a yes-answer, so the algorithm cannot become stuck.

Each successful iteration decreases r by one. Hence, after exactly k iterations, the algorithm reaches $r = 0$. The third invariant condition then gives

$$|M| = k,$$

and the first condition guarantees that M is a matching in G. Thus, the algorithm correctly outputs a matching of size k.

Running Time. It remains to analyze the running time. If the initial query returns 1, it must hold that

$$k \leq \frac{|V|}{2}.$$

This is because if the initial query $D(G, k)$ returns 1, then G contains a matching of size at least k. Because no two edges in a matching share an endpoint, each of the k edges requires two distinct vertices. Consequently, a matching of size k requires at least $2k$ vertices. Since G has only $|V|$ vertices, we must have

$$2k \leq |V|.$$

Equivalently,

$$k \leq \left\lfloor \frac{|V|}{2} \right\rfloor \leq \frac{|V|}{2}.$$

The algorithm performs k iterations, and each iteration examines at most $|E|$ edges. Including the initial query, it therefore makes at most

$$1 + k|E| \leq 1 + \frac{|V||E|}{2}$$

oracle queries. Also, all computation outside the oracle can be carried out in polynomial time. Hence the algorithm is a polynomial-time search-to-decision reduction for BIPARTITE-MATCHING.

This finishes the proof for Lemma 2.3.1.

■

Example 2.3.2 (Reconstructing the matching)**.** *Use the graph from Example 2.2.3 and request $k = 2$. The initial query returns $D(G, 2) = 1$.*

Suppose the first candidate edge is $(a, 1)$*. Deleting* a *and* 1 *leaves the vertices* b *and* 2 *with no edge, so*

$$D(G - \{a, 1\}, 1) = 0.$$

We do not select $(a, 1)$*.*

Next consider $(a, 2)$*. Deleting* a *and* 2 *leaves the edge* $(b, 1)$*, so*

$$D(G - \{a, 2\}, 1) = 1.$$

We select $(a, 2)$*.*

In the remaining graph, selecting $(b, 1)$ *leaves a request for a matching of size zero, whose decision answer is yes. The algorithm outputs*

$$M = \{(a, 2), (b, 1)\}.$$

The witness was reconstructed using only yes/no answers.

> A decision procedure can be enough to construct a solution when we can efficiently express the question: "Can this partial choice be extended to a complete solution?"

Scope of the principle. Many natural problems allow reductions of this kind. However, this is not a theorem asserting that every search problem reduces to its associated existence language. Nor does one yes/no answer necessarily determine a witness. The useful information comes from polynomially many carefully chosen queries.

The next section explains why the exact meaning of the decision query is essential.

2.4 When the Natural Existence Question Is Not Enough

The matching reduction works because deleting endpoints expresses the effect of committing to an edge. But it is totally possible that there are some other search problems that do not have a known reduction to the corresponding decision questions.

We will examine two examples. In both, the associated decision language is in P, but a deterministic polynomial-time algorithm for the search task is not known.

> The examples in this section are not claims of proved search lower bounds. "No polynomial-time algorithm is known" and "no polynomial-time algorithm exists" are different statements. We will also specify the target decision language whenever discussing a search-to-decision reduction.

Remark 2.4.1 (On Example Selection)**.** *We emphasize that the examples selected for this course were chosen deliberately to avoid overlap with other related courses such as CSCI3160, CSCI3130, and CSCI2100. Therefore, please do not panic if you find some of these examples unfamiliar. This is intentional: the goal is to broaden your exposure to different types of algorithmic and complexity-theoretical problems.*

2.4.1 The Maximum-Cut Problem

Let $G = (V, E)$ be an undirected graph with nonnegative integer edge weights w_e, encoded in binary. A subset $S \subseteq V$ specifies a cut, whose weight is

$$W_G(S) := \sum_{\substack{e \in E \\ |e \cap S| = 1}} w_e.$$

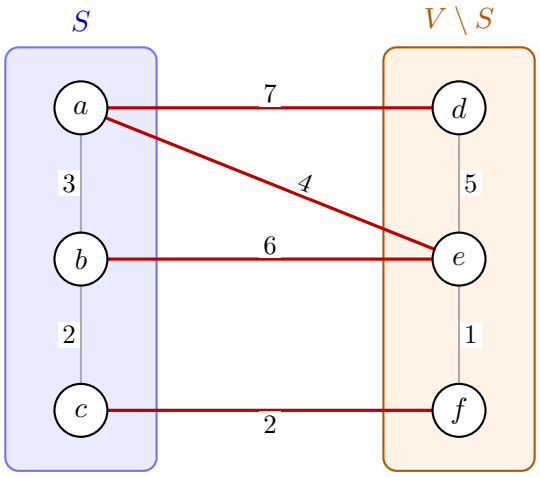
Thus, an edge contributes to the sum exactly when its endpoints lie on opposite sides of the cut. We allow either side to be empty.

Figure 2.2 illustrates the definition. The vertices are partitioned into S and $V \setminus S$. Exactly the edges drawn in red cross this partition, so the cut weight is the sum of their weights.

A *maximum cut* is a subset $S^\star \subseteq V$ such that

$$W_G(S^\star) \geq W_G(T) \qquad \text{for every } T \subseteq V.$$

The search task is to find such a subset $S^\star$.



Crossing edges contribute to the cut weight: $W_G(S) = 7 + 4 + 6 + 2 = 19$.

Figure 2.2: A weighted cut specified by $S = \{a, b, c\}$. The thick red edges have one endpoint in S and one endpoint in $V \setminus S$, so their weights contribute to $W_G(S)$. The gray edges lie entirely on one side and therefore do not contribute.

Existence is immediate

Every finite weighted graph has a maximum cut: there are finitely many subsets of V, and at least one attains the largest cut weight. Consequently, the existence language

$$\text{MAXIMUM-CUT-EXISTS} := \left\{ \langle G, w \rangle : \begin{array}{l} (G, w) \text{ is a valid weighted graph} \\ \text{having a maximum cut} \end{array} \right\}$$

contains every valid instance. A decider need only check that the input is a valid encoding, so this language belongs to P.

This existence argument also suggests a construction: enumerate all $2^{|V|}$ subsets, evaluate their cut weights, and return one with the largest weight. That algorithm terminates and returns a maximum cut, but its exhaustive enumeration is not a polynomial-time procedure.

Thus, we have established existence and given an algorithm, but we have not thereby given an efficient algorithm.

The Search Problem

We define the search problem as follows:

- Given a weighted graph (G, w), find the cut $S^\star \subseteq V$ with maximum weight $W_G(S^\star)$. (Where there are multiple maximum cuts, any one of them is acceptable.)

This problem is not known to be solvable in polynomial time. In fact, it is widely believed that no polynomial-time algorithm exists for this problem.

Therefore, it is also widely believed that no polynomial-time search-to-decision reduction exists from the maximum-cut search problem to the MAXIMUM-CUT-EXISTS decision problem.

Takeaway

An existence theorem and an efficient construction are different guarantees. Finiteness ensures that a maximum cut exists, but it does not tell us how to find one in polynomial time. This example illustrates that distinction; it does not prove that efficient construction is impossible or that search-to-decision reductions cannot exist.

2.4.2 Factorization versus Compositeness

Two different questions about an integer

A positive integer $N \geq 2$ is *prime* if its only positive divisors are 1 and N. Otherwise, it is *composite*.

Consider the following tasks for a binary-encoded integer $N \geq 2$:

- **Compositeness decision:** Does there exist an integer d with $1 < d < N$ and $d \mid N$?
- **Factor search:** Find such a d, or report that N is prime.

The corresponding decision languages are

$$\text{PRIMES} \coloneqq \{\langle N \rangle : N \geq 2 \text{ is prime}\}, \tag{2.10}$$

$$\text{COMPOSITE} \coloneqq \{\langle N \rangle : N \geq 2 \text{ is composite}\}. \tag{2.11}$$

Theorem 2.4.1 (Agrawal–Kayal–Saxena [AKS04])**.** $\text{PRIMES} \in \mathsf{P}$. ◊

We use Theorem 2.4.1 as a known result. Its proof and the AKS algorithm require number theory beyond the scope of this chapter.

Corollary 2.4.1. $\text{COMPOSITE} \in \mathsf{P}$. ◊

Proof. Check that the input encodes an integer $N \geq 2$. On such an input, run the deterministic primality algorithm and reverse its answer. Every integer at least 2 is either prime or composite. ■

Example 2.4.1 (Knowing that a factor exists)**.** *For $N = 91$, a compositeness decider returns yes. A factor-search algorithm must instead output an integer such as 7 or 13, because*

$$91 = 7 \cdot 13.$$

The yes-answer certifies existence, but it does not itself identify either factor.

What is not known

No deterministic polynomial-time algorithm is known for finding a nontrivial factor of every composite binary integer.

Consequently, no polynomial-time search-to-decision reduction from this factor-search task to COMPOSITE is known. Such a reduction, combined with Corollary 2.4.1 and Proposition 2.3.1, would yield a deterministic polynomial-time factor-search algorithm.

This does not prove that compositeness queries can never help. It identifies the consequence that a successful polynomial-time reduction would have.

A different decision question does suffice

Now consider a more informative decision language:

$$\text{BOUNDED-DIVISOR} := \left\{ \langle N, b \rangle : \begin{array}{l} N \geq 2,\ 2 \leq b < N, \\ \text{there exists } d \text{ with } 2 \leq d \leq b \text{ and } d \mid N \end{array} \right\}. \tag{2.12}$$

For a fixed composite N, these answers are monotone in b: once the answer becomes yes, it remains yes for larger admissible thresholds.

An oracle for this language lets us find a factor by binary search. For $N \geq 3$, first query the threshold $N - 1$. A no-answer means that N is prime. A yes-answer lets us search the interval $[2, N - 1]$ for the first threshold with a yes-answer. That threshold is the smallest nontrivial divisor. The special case $N = 2$ is handled directly.

Example 2.4.2 (Finding a divisor by threshold queries)**.** *For $N = 91$, first establish that the threshold 90 gives a yes-answer. Maintain an interval $[\ell, h]$ containing the smallest nontrivial divisor, initially $[2, 90]$.*

At each step, query $b = \lfloor (\ell + h)/2 \rfloor$. On yes, set $h = b$; on no, set $\ell = b + 1$. The resulting steps are

query threshold b	*answer*	*new interval*
46	*yes*	$[2, 46]$
24	*yes*	$[2, 24]$
13	*yes*	$[2, 13]$
7	*yes*	$[2, 7]$
4	*no*	$[5, 7]$
6	*no*	$[7, 7]$

The final interval identifies the divisor 7.

Binary search makes $O(\log N)$ queries, each of length $O(\log N)$, so this is a polynomial-time oracle reduction. It does not give an unconditional polynomial-time factoring algorithm, because BOUNDED-DIVISOR is not known to belong to P.

Takeaway

It is inaccurate to say simply that "factoring does not reduce to decision." Factor search reduces to bounded-divisor decision. What is not known is a polynomial-time reduction to the easier compositeness decision problem.

2.4.3 Further Discussion of Primality Testing

Nothing from Section 2.4.3 will be tested on quizzes or exams!

How difficult is it to recognize a prime number? This question connects more than two thousand years of mathematics with the modern theory of computation. The sieve of Eratosthenes already provided a method for finding primes in ancient Greece. However, having an algorithm is not the same as having an efficient algorithm: for an integer N given in binary, we seek a running time polynomial in its bit length $\ell(N)$, not in its numerical value N.

The modern study of primality testing involved many mathematicians and computer scientists, including Pratt, Miller, Rabin, Solovay, Strassen, Adleman, Pomerance, and Rumely. A major breakthrough came with Miller's work, published in 1976 [Mil75]: assuming the Extended Riemann Hypothesis, he obtained a deterministic polynomial-time primality test. Rabin's 1980 paper [Rab80] gave an unconditional randomized polynomial-time test, now called the *Miller–Rabin test*. For an overview of these developments, see the historical discussion in the AKS paper. To appreciate what AKS later accomplished, we first examine this randomized test.

Number Theory behind Miller–Rabin

We start with a number-theoretic lemma that underlies the Miller–Rabin test. We present it Lemma 2.4.1. We emphasize that it is a necessary condition for primality, but it is **not sufficient**: there are composite numbers that satisfy this condition as well .

Lemma 2.4.1 (Miller–Rabin condition for primes)**.** *Let $N \geq 3$ be an odd prime, and write*

$$N - 1 = 2^s d, \qquad s \geq 1, \quad d \text{ odd}.$$

For every integer $a \in \{1, \ldots, N-1\}$,

$$a^d \equiv 1 \pmod{N} \quad \text{or} \quad a^{2^j d} \equiv -1 \pmod{N} \text{ for some } 0 \leq j < s. \tag{2.13}$$

$\Diamond$

Proof. The key observation is that we can repeatedly apply the difference-of-squares identity

$$u^2 - 1 = (u-1)(u+1).$$

Since $N - 1 = 2^s d$, for $s \geq 2$ this gives

$$\begin{aligned} x^{N-1} - 1 &= x^{2^s d} - 1 \\ &= \left(x^{2^{s-1} d}\right)^2 - 1 \\ &= \left(x^{2^{s-1} d} - 1\right)\left(x^{2^{s-1} d} + 1\right) \\ &= \left(x^{2^{s-2} d} - 1\right)\left(x^{2^{s-2} d} + 1\right)\left(x^{2^{s-1} d} + 1\right) \\ &\;\;\vdots \\ &= (x^d - 1)(x^d + 1)(x^{2d} + 1) \cdots \left(x^{2^{s-1} d} + 1\right) \\ &= (x^d - 1) \prod_{j=0}^{s-1} \left(x^{2^j d} + 1\right). \end{aligned}$$

At each step, we factor the remaining expression of the form $x^{2^r d} - 1$, halving its exponent and producing a new factor $x^{2^{r-1} d} + 1$. After s steps, the remaining factor is $x^d - 1$. For $s = 1$, the same identity is simply

$$x^{2d} - 1 = (x^d - 1)(x^d + 1).$$

Now substitute $x = a$. Since N is prime and $1 \leq a < N$, Fermat's little theorem implies

$$0 \equiv a^{N-1} - 1 = (a^d - 1) \prod_{j=0}^{s-1} (a^{2^j d} + 1) \pmod{N}.$$

A prime dividing a product must divide at least one of its factors. Therefore,

$$a^d - 1 \equiv 0 \pmod{N} \quad \text{or} \quad a^{2^j d} + 1 \equiv 0 \pmod{N} \text{ for some } 0 \leq j < s,$$

which is precisely Equation (2.13).

■

Witnesses and Nonwitnesses

Definition 2.4.1 (Miller–Rabin witness and nonwitness)**.** *Let $N \geq 3$ be odd, write*

$$N - 1 = 2^s d$$

with d odd, and let $a \in \{1, \ldots, N-1\}$. We call a a Miller–Rabin witness for N *if*

$$a^d \not\equiv 1 \pmod{N} \quad \textit{and} \quad a^{2^j d} \not\equiv -1 \pmod{N} \textit{ for every } 0 \leq j < s.$$

Thus, a witness is a base for which Equation (2.13) is false.

We call a a Miller–Rabin nonwitness for N *if*

$$a^d \equiv 1 \pmod{N} \quad \textit{or} \quad a^{2^j d} \equiv -1 \pmod{N} \textit{ for some } 0 \leq j < s.$$

In this case, we also say that N passes the test to base a. ◊

Here, "witness" means a witness of *compositeness*, not of primality. By the contrapositive of Lemma 2.4.1, finding even one Miller–Rabin witness proves that N is composite. In particular, a prime has no Miller–Rabin witnesses.

It is important that, Equation (2.13) is a *necessary but not sufficient* condition for primality. Namely,

$$\text{All } a \in \{1, \ldots, N-1\} \text{ are Miller–Rabin witnesses for } N \implies N \text{ is composite,}$$

but the reverse implication need not hold. A composite number can have both witnesses and nonwitnesses, as demonstrated in the following Example 2.4.3.

Example 2.4.3 (A composite number with witnesses and nonwitnesses)**.** *Let $N = 9$. Since $N - 1 = 8 = 2^3 \cdot 1$, we have $s = 3$ and $d = 1$. The Miller–Rabin sequence for a base a is*

$$(a, a^2, a^4) \pmod{9}.$$

For $a = 2$, this sequence is

$$(2, 4, 7) \pmod{9}.$$

Its first term is not 1, and none of its terms is $-1 \equiv 8 \pmod{9}$. Thus 2 is a Miller–Rabin witness for 9, proving that 9 is composite.

For $a = 8$, however, the sequence is

$$(8, 1, 1) \pmod{9}.$$

Its first term is $-1 \pmod 9$*, so* 8 *is a Miller–Rabin nonwitness for* 9*. Therefore, the composite number* 9 *has both witnesses and nonwitnesses: it fails the test to base* 2 *but passes the test to base* 8*.*

Efficient Falsifiability of (Non)witnesses. We notice that for any $a \in \{1, \ldots, N-1\}$, there is an efficient way to check whether a is a witness. To do that, compute the *Miller–Rabin sequence*

$$a^d,\ a^{2d},\ a^{4d},\ \ldots,\ a^{2^{s-1}d} \pmod N.$$

Each term is obtained by squaring the preceding term modulo N. The base is a nonwitness exactly when the first term is 1 or some term is -1. Otherwise, it is a witness.

For example, let $N = 49$. Since $49 - 1 = 2^4 \cdot 3$, the sequence generated by $a = 2$ is

$$(2^3, 2^6, 2^{12}, 2^{24}) \equiv (8, 15, 29, 8) \pmod{49}.$$

The first term is not 1, and no term is -1 modulo 49, so 2 is a witness for the compositeness of 49. In contrast,

$$18^3 = 5832 = 119 \cdot 49 + 1 \equiv 1 \pmod{49}.$$

Thus 18 is a nonwitness: 49 passes the test to base 18, even though $49 = 7^2$ is composite. This explicitly shows why passing one test does not establish primality.

The Nonwitness Bound

The effectiveness of the test rests on the following counting theorem. We state it without proof, since its proof is beyond the scope of this course. Interested students can find a proof in standard references, e.g., Keith Conrad's notes on the Miller–Rabin test.

Theorem 2.4.2 (Miller–Rabin nonwitness bound)**.** *Let* $N \geq 3$ *be an odd composite integer, and let*

$$L_N = \{a \in \{1, \ldots, N-1\} : a \text{ is a Miller–Rabin nonwitness for } N\}.$$

Then

$$|L_N| \leq \frac{N-1}{4}.$$

Consequently, a uniformly random base in $\{1, \ldots, N-1\}$ *is a nonwitness with probability at most* $1/4$*. A uniformly random base in* $\{2, \ldots, N-2\}$ *is a nonwitness with probability strictly less than* $1/4$*.* ◊

To see the last assertion, observe that 1 and $N-1$ are always nonwitnesses: $1^d \equiv 1 \pmod N$ and, since d is odd, $(N-1)^d \equiv -1 \pmod N$. Removing these two bases gives

$$\frac{|L_N| - 2}{N-3} \leq \frac{(N-1)/4 - 2}{N-3} = \frac{N-9}{4(N-3)} < \frac{1}{4}.$$

Thus we avoid the two uninformative bases by sampling from $\{2, \ldots, N-2\}$.

The Miller–Rabin Test

Here is the Miller–Rabin test for deciding whether an odd integer $N \geq 5$ is prime.

Algorithm 2.4.1: Miller–Rabin Primality Test

Write $N-1=2^s d$ with d odd. The error bound in the last step follows from Theorem 2.4.2.

1. Pick an integer $k \geq 1$ to be the number of trials.
2. Choose an integer a uniformly at random from $\{2, \ldots, N-2\}$, independently of all previous choices. Sampling is with replacement.
3. If a is a Miller–Rabin witness for N, stop the test and output COMPOSITE. This conclusion is certainly correct.
4. If a is a nonwitness and fewer than k trials have been completed, return to Step 2.
5. If all k trials are completed without finding a witness, output PROBABLY PRIME. For every fixed composite input, the probability of reaching this output is at most 4^{-k}.

For general integer inputs $N \geq 2$, first handle the small and even cases: output PROBABLY PRIME immediately if $N=2$ or $N=3$, and output COMPOSITE if $N>2$ is even. The remaining inputs are odd and at least 5, so the sampling range above is nonempty.

The error guarantee

Theorem 2.4.3 (Error guarantee for the repeated test)**.** *For every fixed integer $N \geq 2$ and every integer $k \geq 1$, the k-trial Miller–Rabin test satisfies:*

1. *If N is prime, it outputs PROBABLY PRIME with probability 1.*
2. *If N is composite, it outputs PROBABLY PRIME with probability at most 4^{-k}.*

The probabilities are over the independent, uniformly random bases chosen by the algorithm. $\Diamond$

Proof. The primes 2 and 3 are handled immediately. For any other prime, Lemma 2.4.1 shows that every base is a nonwitness, so all trials pass.

An even composite input is rejected immediately.

For a fixed odd composite input N, let

$$p_N = \frac{|L_N \cap \{2, \ldots, N-2\}|}{N-3} = \frac{|L_N|-2}{N-3}.$$

By Theorem 2.4.2, $p_N < 1/4$. The algorithm outputs PROBABLY PRIME exactly when all k independently chosen bases are nonwitnesses. Therefore,

$$\Pr(\text{output PROBABLY PRIME}) = p_N^k \leq \left(\frac{1}{4}\right)^k.$$

■

For example, 20 independent trials give the error bound

$$4^{-20} = 2^{-40} < 10^{-12}.$$

This is a *one-sided error* guarantee: COMPOSITE is always correct, whereas PROBABLY PRIME allows a small probability of error.

More precisely, for each fixed composite input, the probability of detecting its compositeness is at least $1 - 4^{-k}$. This does *not* mean that, after an output of PROBABLY PRIME, the input is prime with probability at least $1 - 4^{-k}$. Such a posterior probability requires additional assumptions about how the input was selected.

Why the test is efficient

Let $n = \ell(N)$. Repeated squaring computes $a^d \bmod N$ using $O(n)$ modular multiplications, and the remaining checks require at most $s - 1 < n$ modular squarings. All stored residues have at most n bits, and a product of two such residues has at most $2n$ bits. Using ordinary multiplication and long division, each modular multiplication takes $O(n^2)$ bit operations. Thus, the arithmetic in one round takes $O(n^3)$ bit operations, and k rounds take $O(kn^3)$. See also the repeated-squaring implementation in these notes on randomized primality testing.

In particular, for any fixed number of rounds, the arithmetic cost is polynomial in the binary input length. Increasing k reduces the error exponentially while increasing this cost only linearly.

The AKS breakthrough

Miller–Rabin showed that primality testing could be efficient and extremely reliable without any unproved hypothesis. But a fundamental question remained: was there a deterministic algorithm with a polynomial running-time bound and no possibility of error?

In August 2002, more than twenty years after Rabin's paper, Manindra Agrawal, Neeraj Kayal, and Nitin Saxena answered this question. Remarkably, Kayal and Saxena had just completed their undergraduate degrees at IIT Kanpur, where they had worked with Agrawal. Their result was published in 2004.

The significance of Theorem 2.4.1 is not simply that primality can be tested efficiently: randomized polynomial-time tests had been known for decades. AKS established that neither randomness nor unproved number-theoretic assumptions are needed. It settled a longstanding problem at the intersection of mathematics and computer science and showed that, for primality testing, deterministic computation can also achieve polynomial time. This is a statement about polynomial-time solvability, not a claim that AKS is faster than Miller–Rabin. Nor does it establish that randomness can be eliminated efficiently from every randomized algorithm.

We use Theorem 2.4.1 as a known result. Its proof and the AKS algorithm require number theory beyond the scope of this chapter.

2.5 Randomized Algorithms and the Limits of P

The class P captures decision problems solvable by deterministic polynomial-time algorithms. A deterministic algorithm has its next step determined by its current configuration. A *randomized algorithm*, in contrast, may consult random bits, so the same input can lead to different computation paths and possibly different outputs. To model this additional resource, we use a *probabilistic Turing machine*, which extends the deterministic model with random choices. We will formally define probabilistic Turing machines in a later chapter, but for now we can think of them as deterministic Turing machines that have access to a source of random bits.

Running time and correctness

Randomness requires us to distinguish two kinds of guarantees. First, does the algorithm run in polynomial time for *every* sequence of random choices, or is its running time polynomial only *in expectation*? Second, is its answer always correct, or can some random choices produce an

incorrect answer? An expected-time bound is not a worst-case bound, and a small probability of error is not the same as correctness on every computation.

These probabilities concern the algorithm's internal randomness, not a distribution over inputs. For example, the guarantee

$$\Pr[\text{the algorithm answers correctly on } x] \geq \frac{2}{3}$$

means that, for each fixed input x, at least two thirds of the probability lies on random choices that produce the correct answer. It is not enough for the algorithm to work on most inputs.

Why we need another complexity class

A randomized polynomial-time algorithm alone does not establish membership in P. Membership in P requires a deterministic polynomial-time decider that is correct on every input.

This does not mean that problems in P cannot have useful randomized algorithms. Primality testing and minimum cut are examples where deterministic and randomized algorithms coexist. Another good example is the famous quick-sort algorithm, which is a randomized algorithm that sorts an array in *expected* $O(n \log n)$ time—although there are deterministic sorting algorithms achieving the same time complexity even in the worst case, quick-sort is often preferred in practice due to its memory efficiency and simplicity. However, the existence of a randomized algorithm does not automatically imply that the problem is in P. Rather, P does not directly express guarantees involving random choices and a bounded probability of error.

The class BPP provides such a framework. Informally, it consists of languages decidable by randomized algorithms that run in polynomial time for every input and every sequence of random choices, and answer correctly with probability at least $2/3$ on each input. A deterministic algorithm can simply ignore the random bits, so

$$\mathsf{P} \subseteq \mathsf{BPP}.$$

Introducing a different computational model does not, by itself, show that randomness enlarges the class of efficiently solvable problems.

What we defer

In a later chapter, we will formally define probabilistic Turing machines and BPP, explain how repetition reduces error, distinguish one-sided from two-sided error, and study the relationship between deterministic and randomized polynomial time.

We will also address modeling details such as uniform sampling from finite sets using fair random bits. A procedure that retries until sampling succeeds may give an expected-time guarantee rather than a worst-case bound; limiting the retries requires accounting for the probability of failure.

For now, the essential point is that analyzing randomized algorithms requires both a model of random choices and explicit guarantees about running time and correctness, beyond those expressed by P.

Chapter 3
Non-Deterministic Polynomial Time: NP

3.1 From Efficient Solution to Efficient Verification

In earlier lectures, we defined computational problems as decision problems and represented them by languages. Recall that an input string x is a yes-instance of a problem represented by a language L precisely when $x \in L$. We also introduced the class P, consisting of the languages decidable by deterministic Turing machines in polynomial time.

The class P formalizes the idea that a decision problem can be solved efficiently. In this chapter, we study a related but potentially different notion: whether a proposed solution can be verified efficiently.

Finding a solution versus checking a solution

Consider the following informal problem. Given a directed graph G with designated vertices s and t, we want to determine whether there is a path from s to t that visits every vertex exactly once. Such a path is called a *Hamiltonian path* from s to t.

Finding such a path may require considering many possible orders of the vertices. However, suppose someone supplies the sequence

$$s, v_2, v_3, \ldots, v_{m-1}, t.$$

We can check efficiently whether this sequence is a valid Hamiltonian path:

1. Check that every vertex of G appears exactly once.
2. Check that the sequence starts at s and ends at t.
3. Check that each consecutive pair in the sequence is connected by a directed edge.

Thus, even if we do not know how to find a Hamiltonian path efficiently, we can efficiently check whether a proposed path is valid.

Remark 3.1.1 (Solving and verifying are different tasks)**.** *A verification algorithm receives both the original input and a proposed solution. A decision algorithm receives only the original input and must determine the answer without being given a proposed solution.*

This distinction motivates the class NP. Informally, NP contains decision problems whose yes-instances have short certificates that can be checked efficiently.

The central idea of NP is efficient verification: a yes-instance has a polynomial-length witness whose validity can be checked in polynomial time.

Roadmap

Our development proceeds in four stages:

- We formalize witnesses and polynomial-time verifiers.

- We prove that several representative problems belong to NP.
- We prove that P $\subseteq$ NP and discuss the P versus NP question.
- We define nondeterministic Turing machines and prove that they give an equivalent definition of NP.

The verifier definition will be our first formal definition of NP. Nondeterminism will then provide a machine-based interpretation of the same class.

3.2 Witnesses, Polynomial-Time Verifiers, and the Definition of NP

Witnesses

Let $L \subseteq \Sigma^*$ be a language, where Σ is the alphabet; throughout this course, we will use the binary alphabet $\Sigma = \{0, 1\}$.

A *witness*, also called a *certificate*, is a string intended to provide evidence that an input x belongs to L.

For example:

- For a graph containing a Hamiltonian path, a witness may be an ordered list of vertices.
- For a composite integer, a witness may be a nontrivial factor.

A witness does not need to explain how it was found. Its purpose is only to provide enough information for a deterministic algorithm to verify the yes-answer.

Definition 3.2.1 (Verifier)**.** *A* verifier *for a language $L \subseteq \Sigma^*$ is a deterministic Turing machine V such that*

$$x \in L \iff \exists w \in \Sigma^* \text{ such that } V(x, w) = 1. \tag{3.1}$$

The string x is the problem instance, the string w is a witness, and $V(x, w) = 1$ means that the verifier accepts the pair (x, w). ◊

The two directions of Equation (3.1) express two different correctness requirements:

- **Completeness:** If $x \in L$, then at least one witness w causes V to accept.
- **Soundness:** If $x \notin L$, then no witness w causes V to accept.

A yes-instance may have many valid witnesses. The definition requires only that at least one valid witness exist. By contrast, a no-instance must have no valid witness at all.

Example 3.2.1 (A witness for membership in PATH)**.** *Recall that*

$$\mathsf{PATH} = \{\langle G, s, t\rangle : \textit{there is a directed path from } s \textit{ to } t \textit{ in } G\}.$$

Consider the instance

$$x = \langle G, s, t\rangle,$$

where

$$V(G) = \{s, a, b, t\}$$

and

$$E(G) = \{(s, a), (s, b), (a, b), (b, t), (a, t)\}.$$

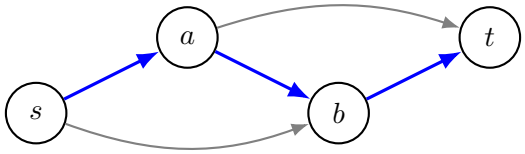
The statement being verified is

$$x \in \mathsf{PATH};$$

that is, G contains a directed path from s to t. A witness for this statement is a sequence of vertices describing such a path. For example, consider

$$w = (s, a, b, t).$$

The graph and the path described by w are illustrated below. The edges of the proposed path are highlighted in blue.



The witness $w = (s, a, b, t)$ is shown in blue.

To verify w, the verifier checks that w begins at s, ends at t, and that every consecutive pair in w is an edge of G:

$$(s, a) \in E(G), \qquad (a, b) \in E(G), \qquad (b, t) \in E(G).$$

All these checks succeed, so w certifies that

$$\langle G, s, t \rangle \in \mathsf{PATH}.$$

In contrast, consider the proposed witness

$$w' = (s, b, a, t).$$

The verifier rejects w' because

$$(b, a) \notin E(G).$$

Rejecting w' does not show that $\langle G, s, t \rangle \notin \mathsf{PATH}$. It shows only that w' is not a valid witness. Indeed, the valid witness $w = (s, a, b, t)$ establishes that this instance belongs to PATH.

Polynomially Bounded Witnesses

To capture efficient verification, we must impose two efficiency requirements:

1. The witness must have polynomial length.
2. The verifier must run in polynomial time.

The witness-length requirement is necessary because an algorithm cannot efficiently process a witness that is exponentially longer than the original input.

Definition 3.2.2 (Polynomially bounded witness)**.** *A verifier V uses* polynomially bounded witnesses *if there is a polynomial p such that every relevant witness w satisfies*

$$|w| \leq p(|x|),$$

where $|x|$ and $|w|$ denote the lengths of the input and witness, respectively. ◊

For instance, if $|x| = n$, a witness of length $n^3 + 2n$ is polynomially bounded. A witness of length 2^n is not polynomially bounded.

The running time of a verifier is naturally measured in terms of the total input length $|x| + |w|$. If

$$|w| \leq p(|x|)$$

and the verifier runs in time polynomial in $|x| + |w|$, then its running time is also polynomial in $|x|$. Indeed, if the running time is bounded by a polynomial q, then

$$q(|x| + |w|) \leq q\big(|x| + p(|x|)\big),$$

which is polynomial in $|x|$.

The Verifier Definition of NP

We can now give our first formal definition of NP.

Definition 3.2.3 (The class NP: verifier definition)**.** *The class* NP *consists of all languages $L \subseteq \Sigma^*$ for which there exist a deterministic polynomial-time Turing machine (dubbed* a verifier*) V and a polynomial p such that, for every $x \in \Sigma^*$,*

$$x \in L \iff \exists w \in \Sigma^* \text{ such that } |w| \leq p(|x|) \text{ and } V(x, w) = 1. \tag{3.2}$$

$\Diamond$

The verifier V and polynomial p are fixed for the language L. They cannot be chosen separately for each input.

Let us unpack Equation (3.2):

- x is the original instance of the decision problem.
- w is a proposed witness for the claim $x \in L$.
- The existential quantifier means that at least one accepted witness must exist for each yes-instance.
- The condition $|w| \leq p(|x|)$ ensures that the witness is not excessively long.
- The verifier V checks the witness deterministically in polynomial time.
- If $x \notin L$, every possible witness must be rejected.

What the definition does not provide. The definition of NP does not give an efficient method for finding a witness. It says only that if a suitable witness is supplied, then its validity can be checked efficiently.

3.3 Examples of Problems in NP

The examples in this section illustrate several kinds of witnesses. A witness may be a truth assignment, an ordering of vertices, a subset of vertices or numbers, or an arithmetic factor. In each case, the central task is to verify a proposed solution rather than to find one.

3.3.1 Boolean Satisfiability

Motivation

Many problems in computer science ask whether a collection of constraints can be satisfied simultaneously. For example:

- A software configuration tool may need to select features subject to requirements such as "if feature A is enabled, then feature B must also be enabled" and "features C and D cannot both be enabled." See Supplementary Box 3.3.1 for a concrete example of this type of problem.
- A package manager (e.g., *pip*, *conda*, *brew*, *npm*, *yarn*, *apm*) may need to determine whether packages can be installed without violating dependency and incompatibility constraints.
- The model checker *CBMC* can analyze a C program to determine whether some input causes an assertion to fail, an array access to go out of bounds, or an arithmetic overflow to occur.
- The AI planning system *Fast Downward* can plan actions for a warehouse robot, such as moving between locations, picking up a package, and delivering it to a packing station while respecting the preconditions of each action. See Supplementary Box 3.3.2 for a concrete example of this type of problem.

In each of these applications, a binary choice can be represented by a Boolean variable. Logical formulas can then express the constraints that valid choices must satisfy. The resulting computational question is:

Is there an assignment of values to the variables that makes all the constraints true?

This is the Boolean satisfiability problem. It is one of the most fundamental problems in theoretical computer science and also forms the basis of practical tools called *SAT solvers*.

Supplementary Box 3.3.1: Configuring a Linux Kernel as a Satisfiability Problem

Consider configuring a custom Linux kernel for an embedded device. Linux provides a real configuration system, called *Kconfig*, through which a developer chooses which kernel capabilities and device drivers will be included in the compiled system.
For example, introduce Boolean variables with the following meanings:

$$N = \text{"general networking support is enabled,"}$$
$$W = \text{"wireless-networking support is enabled,"}$$
$$U = \text{"USB support is enabled,"}$$
$$K = \text{"the USB-keyboard driver is enabled."}$$

These choices cannot be made independently. Wireless networking requires general networking support, so the configuration must satisfy

$$W \to N.$$

Similarly, the USB-keyboard driver is useful only when USB support is enabled, giving the constraint

$$K \to U.$$

Using the logical equivalence

$$A \to B \equiv \neg A \lor B,$$

these requirements can be written as the clauses

$$(\neg W \lor N) \qquad \text{and} \qquad (\neg K \lor U).$$

Suppose that the embedded device must support both Wi-Fi and a USB keyboard. We then add the requirements

$$W \qquad \text{and} \qquad K.$$

The complete configuration formula becomes

$$(\neg W \lor N) \land (\neg K \lor U) \land W \land K.$$

A satisfying assignment is

$$N = 1, \qquad W = 1, \qquad U = 1, \qquad K = 1.$$

This assignment describes a valid kernel configuration containing all the infrastructure required by the selected functionality.
In a realistic configuration system, there may be thousands of such options and dependencies. Determining whether all requested capabilities can be enabled simultaneously is therefore a Boolean satisfiability problem.

Supplementary Box 3.3.2: Controlling a Warehouse Robot

Consider using an AI planning system such as *Fast Downward* to control a warehouse robot. Fast Downward accepts a description of the available actions, their preconditions and effects, the initial state, and the desired goal. It then searches for a sequence of actions that reaches the goal.
Suppose a package is initially in a storage room and must be delivered to a packing station. The robot can move between the two locations, pick up the package, and put it down. For each time step t, introduce Boolean variables with the following meanings:

$$\begin{aligned} R_{S,t} &= \text{“the robot is in the storage room at time } t\text{,”} \\ R_{P,t} &= \text{“the robot is at the packing station at time } t\text{,”} \\ B_t &= \text{“the robot is carrying the package at time } t\text{,”} \\ L_{S,t} &= \text{“the package is in the storage room at time } t\text{,”} \\ L_{P,t} &= \text{“the package is at the packing station at time } t\text{.”} \end{aligned}$$

We also introduce variables representing the actions that the robot may perform:

$$\begin{aligned} \mathit{Pick}_t &= \text{“the robot picks up the package at time } t\text{,”} \\ \mathit{Move}_{S,P,t} &= \text{“the robot moves from storage to packing at time } t\text{,”} \\ \mathit{Drop}_t &= \text{“the robot puts down the package at time } t\text{.”} \end{aligned}$$

The initial state is described by

$$R_{S,0} \land L_{S,0} \land \neg B_0.$$

Thus, the robot and package are both in the storage room, and the robot is not yet carrying the package.
Each action has preconditions. For example, the robot can pick up the package only if both

the robot and the package are in the storage room:

$$Pick_t \rightarrow (R_{S,t} \wedge L_{S,t}).$$

Similarly, the robot can move from storage to the packing station only if it is currently in storage:

$$Move_{S,P,t} \rightarrow R_{S,t}.$$

The robot can put down the package only if it is carrying the package and is at the packing station:

$$Drop_t \rightarrow (B_t \wedge R_{P,t}).$$

Actions also have effects. Picking up the package causes the robot to carry it:

$$Pick_t \rightarrow B_{t+1}.$$

Moving from storage to packing changes the robot's location:

$$Move_{S,P,t} \rightarrow R_{P,t+1}.$$

Putting down the package at the packing station completes the delivery:

$$Drop_t \rightarrow L_{P,t+1}.$$

If the robot must complete the delivery within three actions, the goal condition is

$$L_{P,3}.$$

A satisfying assignment may select the actions

$$Pick_0 = 1, \qquad Move_{S,P,1} = 1, \qquad Drop_2 = 1.$$

These true variables describe the plan

$$Pick \ ; \ Move_{S,P} \ ; \ Drop.$$

In a realistic planning problem, the formula would also include constraints describing unchanged facts, mutually exclusive actions, battery capacity, blocked paths, multiple robots, and delivery deadlines. Finding a satisfying assignment then corresponds to finding a valid plan that achieves the goal without violating any of these constraints.

Boolean Formulas

A *Boolean variable* is a variable that takes a value in

$$\{0, 1\},$$

where 1 represents *true* and 0 represents *false*. We use variables such as

$$x_1, x_2, \ldots, x_m.$$

Boolean formulas are constructed from Boolean variables using logical connectives. The

three basic connectives used here are:

$$\begin{aligned} \neg\varphi &: \text{negation, meaning "not } \varphi\text{"},\\ \varphi \wedge \psi &: \text{conjunction, meaning "}\varphi \text{ and } \psi\text{"},\\ \varphi \vee \psi &: \text{disjunction, meaning "}\varphi \text{ or } \psi\text{"}. \end{aligned}$$

The disjunction $\varphi \vee \psi$ is inclusive: it is true when at least one of φ and ψ is true, including when both are true.

Definition 3.3.1 (Literal)**.** *A* literal *is either a Boolean variable x, called a* positive literal*, or the negation $\neg x$ of a Boolean variable, called a* negative literal*.* ◊

For example,

$$x_1, \qquad \neg x_2, \qquad x_3$$

are literals.

Definition 3.3.2 (Clause)**.** *A* clause *is a disjunction of one or more literals. Thus, a clause has the form*

$$\ell_1 \vee \ell_2 \vee \cdots \vee \ell_t,$$

where each ℓ_i is a literal. ◊

For example,

$$x_1 \vee \neg x_2 \vee x_4$$

is a clause. It is true whenever at least one of its literals is true.

Definition 3.3.3 (Conjunctive Normal Form)**.** *A Boolean formula is in* conjunctive normal form*, abbreviated* CNF*, if it is a conjunction of clauses. A CNF formula therefore has the form*

$$C_1 \wedge C_2 \wedge \cdots \wedge C_s,$$

where each C_i is a clause. ◊

For example,

$$(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3) \wedge (\neg x_1 \vee x_3)$$

is a CNF formula consisting of three clauses. A CNF formula is true exactly when every one of its clauses is true.

A general Boolean formula need not be in CNF. For example,

$$\neg(x_1 \wedge x_2) \vee (x_3 \wedge x_4)$$

is a Boolean formula, but it is not written in CNF.

Definition 3.3.4 (Truth assignment)**.** *Let φ be a Boolean formula whose variables belong to*

$$\{x_1, x_2, \ldots, x_m\}.$$

A truth assignment *for φ is a function*

$$\alpha : \{x_1, x_2, \ldots, x_m\} \longrightarrow \{0, 1\}.$$

Thus, $\alpha(x_i)$ specifies whether x_i is assigned false or true. ◊

The value of a formula under an assignment is determined recursively. For example,

$$\begin{aligned} \neg x_i \text{ is true} &\iff \alpha(x_i) = 0, \\ \varphi \wedge \psi \text{ is true} &\iff \varphi \text{ and } \psi \text{ are both true}, \\ \varphi \vee \psi \text{ is true} &\iff \text{at least one of } \varphi \text{ and } \psi \text{ is true}. \end{aligned}$$

If φ evaluates to true under α, we say that α *satisfies* φ, and write

$$\alpha \models \varphi.$$

Such an assignment is called a *satisfying assignment* for φ. A formula is *satisfiable* if it has at least one satisfying assignment; otherwise, it is *unsatisfiable*.

Proposition 3.3.1 (Polynomial-time conversion to CNF)**.** *Let $F(x_1, \ldots, x_n)$ be a Boolean formula. There is an algorithm that, given F, constructs a CNF formula*

$$C(x_1, \ldots, x_n, z_1, \ldots, z_m)$$

such that, for every assignment α to the original variables,

$F(\alpha) = 1 \iff$ *there exists an assignment β to the auxiliary variables such that $C(\alpha, \beta) = 1$.*

Equivalently,

$$F(\boldsymbol{x}) \equiv \exists \boldsymbol{z}\, C(\boldsymbol{x}, \boldsymbol{z}).$$

Moreover, C has size $O(|F|)$, and it can be constructed in time $O(|F|)$.

Proof. We first describe the construction and then prove its correctness. The main idea is to introduce one auxiliary variable for each nontrivial subformula of F. This variable records the truth value of the corresponding subformula. We use a constant number of CNF clauses to enforce the intended relationship between each subformula and its immediate children. Finally, we require the variable representing the entire formula F to be true.

The construction: For every nontrivial subformula φ of F, introduce a fresh variable z_φ. This variable is intended to satisfy

$$z_\varphi = 1 \iff \varphi = 1.$$

We now add clauses that enforce this condition. The clauses depend on the outermost connective of φ.

If $\varphi = \neg\psi$, we enforce

$$z_\varphi \leftrightarrow \neg z_\psi$$

using the CNF formula

$$(\neg z_\varphi \vee \neg z_\psi) \wedge (z_\varphi \vee z_\psi).$$

If $\varphi = \psi \wedge \theta$, we enforce

$$z_\varphi \leftrightarrow (z_\psi \wedge z_\theta)$$

using

$$(\neg z_\varphi \vee z_\psi) \wedge (\neg z_\varphi \vee z_\theta) \wedge (z_\varphi \vee \neg z_\psi \vee \neg z_\theta).$$

Finally, if $\varphi = \psi \vee \theta$, we enforce

$$z_\varphi \leftrightarrow (z_\psi \vee z_\theta)$$

using

$$(z_\varphi \vee \neg z_\psi) \wedge (z_\varphi \vee \neg z_\theta) \wedge (\neg z_\varphi \vee z_\psi \vee z_\theta).$$

Here, if ψ or θ is an original variable, we use that variable directly instead of introducing an additional auxiliary variable. Let z_F be the variable associated with the entire formula. After adding the clauses for all subformulas, we add the unit clause

$$(z_F),$$

which requires the original formula to be true. Let C be the conjunction of all these clauses.

We next verify that this construction has the required correctness property.

For the forward direction: Fix an assignment α to the original variables and suppose that

$$F(\alpha) = 1.$$

We must show that α can be extended to the auxiliary variables so that C is satisfied.

For each subformula φ, assign z_φ the truth value of φ under α:

$$z_\varphi \coloneqq \varphi(\alpha).$$

By construction, each auxiliary variable now has exactly the same truth value as the subformula it represents. Consequently, every equivalence encoded above is satisfied, and hence all the corresponding CNF clauses are satisfied.

In addition, the assumption $F(\alpha) = 1$ gives

$$z_F = 1.$$

Therefore, the final unit clause (z_F) is also satisfied. Thus, the assignment α can be extended to an assignment satisfying C.

For the reverse direction: Suppose that an assignment α to the original variables can be extended to an assignment satisfying C. We must show that

$$F(\alpha) = 1.$$

We first show, proceeding from the smallest subformulas to the entire formula, that every auxiliary variable has the same truth value as the subformula it represents. Indeed, the clauses associated with a subformula $\varphi = \neg\psi$ enforce

$$z_\varphi = \neg z_\psi.$$

Similarly, the clauses associated with $\varphi = \psi \wedge \theta$ enforce

$$z_\varphi = z_\psi \wedge z_\theta,$$

and the clauses associated with $\varphi = \psi \vee \theta$ enforce

$$z_\varphi = z_\psi \vee z_\theta.$$

Applying these observations recursively from the original variables upward through the syntax tree of F, we obtain

$$z_\varphi = \varphi(\alpha)$$

for every subformula φ. In particular,

$$z_F = F(\alpha).$$

The unit clause (z_F) belongs to C, so any satisfying assignment of C must have $z_F = 1$. It

follows that

$$F(\alpha) = 1,$$

as required.

Combining the two directions, for every assignment α to the original variables,

$$F(\alpha) = 1 \quad \Longleftrightarrow \quad \exists \beta \; C(\alpha, \beta) = 1.$$

Size and running time: There are at most $O(|F|)$ subformulas. For each nontrivial subformula, the construction introduces one auxiliary variable and at most three clauses, each of constant size. Therefore, the total number of variables, clauses, and literal occurrences in C is $O(|F|)$.

The algorithm constructs these clauses by traversing the syntax tree of F once and performing a constant amount of work at each node. Hence, the construction takes $O(|F|)$ time.

This finishes the proof of Proposition 3.3.1.

■

Defining the Boolean Satisfiability Problem

Definition 3.3.5 (Boolean satisfiability)**.** *Define*

$$\text{SAT} \coloneqq \left\{ \langle \varphi \rangle \;\middle|\; \begin{array}{l} \varphi \text{ is a CNF formula and there exists} \\ \text{a truth assignment } \alpha \text{ such that } \alpha \models \varphi \end{array} \right\}.$$

Equivalently, SAT *is the problem of deciding whether a given CNF Boolean formula has a satisfying assignment.* ◊

Proposition 3.3.2. $\text{SAT} \in \mathsf{NP}$.

Proof. The proof of Proposition 3.3.2 is straightforward: Given a candidate witness, the verifier evaluates the formula under the specified assignment. If the formula evaluates to true, the verifier accepts; otherwise, it rejects. The evaluation can be performed in polynomial time, and the witness is polynomially bounded in length. ■

Example 3.3.1 (Verifying a satisfying assignment)**.** *Consider the CNF formula*

$$\varphi = \underbrace{(x_1 \vee \neg x_2)}_{C_1} \wedge \underbrace{(x_2 \vee x_3)}_{C_2} \wedge \underbrace{(\neg x_1 \vee x_3)}_{C_3}.$$

The formula consists of the three clauses

$$\begin{aligned} C_1 &= x_1 \vee \neg x_2, \\ C_2 &= x_2 \vee x_3, \\ C_3 &= \neg x_1 \vee x_3. \end{aligned}$$

The expressions

$$x_1, \quad \neg x_2, \quad x_2, \quad x_3, \quad \neg x_1$$

appearing inside these clauses are literals.

Suppose that the witness specifies the assignment

$$\alpha(x_1) = 1, \qquad \alpha(x_2) = 0, \qquad \alpha(x_3) = 1.$$

Equivalently, the witness is the bit string

$$w = 101.$$

Evaluating the three clauses under this assignment gives

$$\begin{aligned} C_1 &= x_1 \vee \neg x_2 = 1 \vee \neg 0 = 1 \vee 1 = 1, \\ C_2 &= x_2 \vee x_3 = 0 \vee 1 = 1, \\ C_3 &= \neg x_1 \vee x_3 = \neg 1 \vee 1 = 0 \vee 1 = 1. \end{aligned}$$

Therefore,

$$\varphi = C_1 \wedge C_2 \wedge C_3 = 1 \wedge 1 \wedge 1 = 1.$$

Thus,

$$\alpha \models \varphi,$$

so $w = 101$ *is a certificate that*

$$\langle \varphi \rangle \in \text{SAT}.$$

3.3.2 Hamiltonian Path

A directed path in a directed graph is a sequence of vertices in which every consecutive pair is connected by a directed edge. A Hamiltonian path is a path that visits every vertex exactly once.

Remark 3.3.1. *We want to distinguish between a Hamiltonian path and a Hamiltonian* cycle*. A Hamiltonian cycle is a Hamiltonian path with an additional directed edge from its last vertex back to its first vertex. Thus, both a Hamiltonian path and a Hamiltonian cycle visit every vertex exactly once, but a Hamiltonian cycle returns to its starting vertex, whereas a Hamiltonian path need not do so. Consequently, a graph may contain a Hamiltonian path without containing a Hamiltonian cycle.*

Definition 3.3.6 (Hamiltonian path)**.** *Define*

$$\text{HAMPATH} \coloneqq \left\{ \langle G, s, t \rangle \;\middle|\; \begin{array}{l} G \text{ is a directed graph containing a Hamiltonian} \\ \text{path that begins at } s \text{ and ends at } t \end{array} \right\}.$$

◊

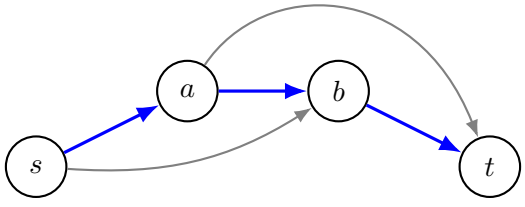


Figure 3.1: A directed graph containing the Hamiltonian path (s, a, b, t), highlighted in blue.

Proposition 3.3.3. HAMPATH $\in$ NP.

Proof. Let G have m vertices. The witness is an ordered sequence

$$w = (v_1, v_2, \ldots, v_m)$$

that claims to be a Hamiltonian path from s to t.

The verifier performs the following checks:

1. Check that the sequence contains exactly m vertices.
2. Check that every vertex of G appears exactly once.
3. Check that $v_1 = s$ and $v_m = t$.
4. For each $i \in \{1, \ldots, m-1\}$, check that

$$(v_i, v_{i+1}) \in E(G).$$

The witness contains m vertex names, each of which has a representation of length polynomial in the size of the graph encoding. Hence the total witness length is polynomially bounded.

If G has a Hamiltonian path from s to t, listing its vertices in order produces an accepted witness. Conversely, if the verifier accepts, the listed vertices form a directed path from s to t and include every vertex exactly once. Therefore, an accepted witness describes a Hamiltonian path.

There are $m-1$ consecutive pairs to check. Duplicate detection and edge lookup can also be implemented in polynomial time under any reasonable graph encoding. Thus the verifier runs in polynomial time, and HAMPATH $\in$ NP. ■

For the graph in Figure 3.1, the sequence

$$(s, a, b, t)$$

is accepted because all four vertices appear exactly once and

$$(s, a), (a, b), (b, t) \in E(G).$$

3.3.3 Clique

Let $G = (V, E)$ be an undirected graph. A set $C \subseteq V$ is a *clique* if every pair of distinct vertices in C is connected by an edge.

Remark 3.3.2 (Why is it called a clique?)**.** *A clique is nothing but a "fully connected" set of vertices. The term* clique *comes from social groups. In everyday language, a clique a small, exclusive group of people who spend time together and do not welcome outsidersThus, informally, a clique is a group of vertices that are all "friends" with one another.*

Definition 3.3.7 (Clique)**.** *Define*

$$\text{CLIQUE} := \{\langle G, k\rangle \mid G \text{ contains a clique of size at least } k\}.$$

♢

Supplementary Box 3.3.3: Maximum Course Load

Suppose a student wants to know the maximum number of courses she can take this semester without any timetable conflicts. Each course may have several sessions, and sessions from different courses may meet at overlapping times.
Construct a graph in which:

- each vertex represents a course; and

- two vertices are connected by an edge if the corresponding courses' meeting times do not conflict.

A clique in this graph is a collection of courses that are pairwise compatible. Therefore, every clique represents a conflict-free schedule. The size of a maximum clique gives the largest number of courses the student can take without any timetable conflicts.

Proposition 3.3.4. CLIQUE $\in$ NP.

Proof. For an input $\langle G, k\rangle$, the witness is a list

$$w = (v_1, v_2, \ldots, v_k)$$

of vertices claimed to form a clique.

The verifier operates as follows:

1. Check that the witness lists k distinct vertices of G.
2. For every pair i, j with $1 \le i < j \le k$, check that
$$\{v_i, v_j\} \in E(G).$$
3. Accept if and only if all checks succeed.

If G contains a clique of size at least k, any k vertices from that clique form an accepted witness. Conversely, if the verifier accepts, every pair of listed vertices is connected, so the listed vertices form a clique of size k.

The verifier checks

$$\binom{k}{2} = \frac{k(k-1)}{2}$$

pairs. Because a graph with m vertices cannot contain a clique larger than m, we need only consider $k \le m$. Thus the number of checks is $O(m^2)$, which is polynomial in the input size. The witness length is also polynomially bounded. Therefore, CLIQUE $\in$ NP. ■

Example 3.3.2 (Checking a proposed clique)**.** *Suppose an undirected graph contains vertices a, b, c, d, and its edges include*

$$\{a, b\}, \quad \{a, c\}, \quad \{b, c\}.$$

Then the witness

$$w = (a, b, c)$$

certifies the existence of a clique of size 3*, because all three required pairs are edges.*

If $\{b, d\} \notin E(G)$, then the proposed witness

$$w' = (a, b, d)$$

is rejected. As in Example 3.2.1, rejecting one witness does not establish that the instance is a no-instance; another witness may still succeed.

3.3.4 Independent Set

Let $G = (V, E)$ be an undirected graph. A set $I \subseteq V$ is an *independent set* if no pair of distinct vertices in I is connected by an edge.

Definition 3.3.8 (Independent Set)**.** *Define*

$$\text{INDEPENDENT-SET} \coloneqq \{\langle G, k\rangle \mid G \textit{ contains an independent set of size at least } k\}\,.$$

$\Diamond$

Supplementary Box 3.3.4: Seating Guests at a Wedding

Suppose a couple is planning the seating arrangement for their wedding. Unfortunately, some guests do not get along and should not be seated at the same table. For example, the guest list may include former couples who had unpleasant breakups, such as Alice and her ex-boyfriend Bob, or Carol and her ex-girlfriend Diana.
Construct a graph in which:

- each vertex represents a wedding guest; and
- two vertices are connected by an edge if the corresponding guests should not be seated together.

An independent set in this graph is a group of guests with no conflicts between them. Therefore, all guests in an independent set can be seated at the same table without placing any incompatible pair together. The size of a maximum independent set gives the largest number of mutually compatible guests who can safely be seated at one table.

Proposition 3.3.5. INDEPENDENT-SET $\in \mathsf{NP}$.

Proof. Omitted. ■

3.3.5 Graph Three-Coloring

Let $G = (V, E)$ be an undirected graph. A *three-coloring* of G is an assignment of one of three colors to each vertex such that no two adjacent vertices receive the same color. A graph is *three-colorable* if it has such a coloring.

Definition 3.3.9 (Graph Three-Coloring)**.** *Define*

$$\text{G3C} \coloneqq \{\langle G\rangle \mid G \textit{ has a three-coloring}\}\,.$$

$\Diamond$

Supplementary Box 3.3.5: Applications of Graph Coloring

Graph coloring naturally models situations in which conflicting objects must be assigned different labels, times, or resources.

Coloring a map. Suppose we want to color a map using at most three colors so that neighboring regions receive different colors. Construct a graph in which:

- each vertex represents a region of the map; and
- two vertices are connected by an edge if the corresponding regions share a boundary.

A three-coloring of this graph assigns one of three colors to each region while ensuring that neighboring regions have different colors. Thus, the map can be properly colored with at most three colors if and only if its corresponding graph is three-colorable.

Scheduling final examinations. Suppose a university wants to arrange its centralized final

examinations using three time slots. Two courses may have examinations in the same time slot only if they have no students in common.
Construct a graph in which:

- each vertex represents a course; and
- two vertices are connected by an edge if the corresponding courses have at least one student in common.

Assigning a color to a vertex represents scheduling the corresponding examination in one of the three time slots. Because adjacent vertices must receive different colors, courses that have a student in common are assigned different examination times. Therefore, all examinations can be scheduled using three time slots without conflicts if and only if the resulting graph is three-colorable.

Proposition 3.3.6. G3C $\in$ NP.

Proof. Omitted. ■

3.3.6 Knapsack

Suppose there are n items. Each item i has a positive integer weight w_i and a positive integer value v_i. Given a weight capacity W and a target value V, the knapsack problem asks whether some subset of the items has total weight at most W and total value at least V.

Definition 3.3.10 (Knapsack)**.** *Define*

$$\text{KNAPSACK} \coloneqq \left\{ \langle (w_1, v_1), \ldots, (w_n, v_n), W, V \rangle \;\middle|\; \begin{array}{l} \textit{there exists } S \subseteq \{1, \ldots, n\} \textit{ such that} \\ \sum_{i \in S} w_i \leq W \quad \textit{and} \quad \sum_{i \in S} v_i \geq V \end{array} \right\}.$$

◊

Supplementary Box 3.3.6: A Burglar in a Jewelry Store

Imagine that a burglar breaks into a jewelry store carrying a bag that can hold at most W kilograms. The store contains n pieces of jewelry (e.g., necklaces, bracelets, rings, diamonds, gold bars). Each piece i has:

- a weight w_i; and
- a monetary value v_i.

The burglar cannot carry every piece because the bag has a limited capacity. Suppose the burglar wants to know whether it is possible to steal jewelry worth at least V dollars without exceeding the bag's weight capacity.
A selection S of jewelry is feasible precisely when

$$\sum_{i \in S} w_i \leq W,$$

and it achieves the burglar's target precisely when

$$\sum_{i \in S} v_i \geq V.$$

Thus, the question

> Can the burglar select jewelry with total value at least V while carrying total weight at most W?

is an instance of the decision version of the knapsack problem.

Proposition 3.3.7. KNAPSACK $\in$ NP.

Proof. For an input

$$x = \langle (w_1, v_1), \ldots, (w_n, v_n), W, V \rangle ,$$

the witness is a subset $S \subseteq \{1, \ldots, n\}$ identifying the items to be placed in the knapsack.

The verifier operates as follows:

1. Compute the total weight
$$w(S) = \sum_{i \in S} w_i.$$
2. Compute the total value
$$v(S) = \sum_{i \in S} v_i.$$
3. Accept if and only if
$$w(S) \leq W \qquad \text{and} \qquad v(S) \geq V.$$

If the input is a yes-instance, there exists a subset S satisfying both inequalities, and that subset is an accepted witness. Conversely, if the verifier accepts a witness S, then the selected items have total weight at most W and total value at least V. Therefore, the input is a yes-instance.

The witness can be represented by an n-bit string indicating which items are selected. The required additions and comparisons can be performed in time polynomial in the length of the input. Therefore,

$$\text{KNAPSACK} \in \mathsf{NP}.$$

■

3.3.7 Subset Sum

The subset-sum problem asks whether some of the integers in a given list add up to a specified target.

Definition 3.3.11 (Subset sum)**.** *Define*

$$\text{SUBSET-SUM} \coloneqq \left\{ \langle a_1, \ldots, a_m, t \rangle \;\middle|\; \exists I \subseteq \{1, \ldots, m\} \text{ such that } \sum_{i \in I} a_i = t \right\}.$$

◊

Proposition 3.3.8. SUBSET-SUM $\in$ NP.

Proof. The witness is an m-bit string

$$w = w_1 w_2 \cdots w_m,$$

where

$$w_i = \begin{cases} 1, & \text{if } a_i \text{ is selected,} \\ 0, & \text{if } a_i \text{ is not selected.} \end{cases}$$

The witness has length m, which is at most the length of the input encoding.

The verifier computes

$$S = \sum_{i=1}^{m} w_i a_i \tag{3.3}$$

and accepts if and only if $S = t$.

If the input is a yes-instance, the indicator string of a subset whose sum is t is accepted. Conversely, an accepted indicator string identifies a subset whose sum is exactly t.

The integers are represented in binary. Adding polynomially many binary integers, each having at most the input length in bits, can be done in polynomial time. Therefore, the verifier runs in polynomial time, and SUBSET-SUM $\in$ NP. ■

Example 3.3.3 (A subset-sum witness)**.** *Consider the instance*

$$\langle 3, 7, 10, 14, 17 \rangle,$$

where the final number 17 *is the target. The witness*

$$w = 1001$$

selects 3 *and* 14*. Using Equation (3.3), the verifier computes*

$$1 \cdot 3 + 0 \cdot 7 + 0 \cdot 10 + 1 \cdot 14 = 17.$$

It therefore accepts.

By contrast, the witness 0110 *selects* 7 *and* 10*, which also sums to* 17*, so a yes-instance may have more than one accepted witness.*

3.3.8 Composite Integers

An integer $n > 1$ is composite if it can be expressed as a product of two smaller positive integers, neither of which is 1.

Definition 3.3.12 (Composite integers)**.** *Define*

$$\text{COMPOSITES} := \{\langle n \rangle \mid n = ab \text{ for some integers } a, b \text{ with } 1 < a < n \text{ and } 1 < b < n\}.$$

◊

Proposition 3.3.9. COMPOSITES $\in$ NP.

Proof. The witness is a pair of binary integers $\langle a, b \rangle$. The verifier checks that

$$1 < a < n, \qquad 1 < b < n,$$

computes ab, and accepts if and only if

$$ab = n.$$

If n is composite, its nontrivial factors provide an accepted witness. Conversely, if the verifier accepts, then it has found a nontrivial factorization of n, so n is composite.

Because $a < n$ and $b < n$, each factor uses at most as many binary digits as n. Binary comparison and multiplication can be performed in polynomial time. Hence the witness has polynomial length, the verifier runs in polynomial time, and COMPOSITES $\in$ NP.

■

Remark 3.3.3 (More than one verifier for the same language)**.** *The language* COMPOSITES *is a special example because it illustrates that the same language may have more than one polynomial-time verification algorithm.*

The verifier in the proof above uses a nontrivial factorization $\langle a, b\rangle$ as its witness. However, a different verifier can use the empty string

$$\varepsilon$$

as its witness. On input $(\langle n\rangle, \varepsilon)$, this verifier operates as follows:

1. *If $n \leq 1$, reject.*
2. *Run the [AKS04] primality-testing algorithm on n.*
3. *Accept if and only if the primality-testing algorithm reports that n is not prime.*

Since every integer $n > 1$ is either prime or composite, this verifier accepts exactly when n is composite. It runs in polynomial time because primality testing can be performed in polynomial time.

Thus, COMPOSITES *has at least two natural verification algorithms:*

- *one that verifies a nontrivial factorization supplied as a witness; and*
- *one that ignores the witness and directly decides whether the input is composite.*

This illustrates an important point: witnesses and verifiers are not unique. Membership in NP *requires only that* some *polynomial-time verifier with polynomially bounded witnesses exists.*

3.4 The Relationship Between P and NP

We have described P as the class of efficiently solvable decision problems and NP as the class of decision problems with efficiently verifiable yes-instances. Every efficiently solvable problem is also efficiently verifiable.

Theorem 3.4.1.

$$\mathsf{P} \subseteq \mathsf{NP}.$$

◊

Proof. Let $L \in$ P. By the definition of P, there is a deterministic polynomial-time Turing machine M that decides L.

Construct a verifier V that operates as follows on input (x, w):

1. Ignore the witness w.
2. Run M on x.
3. Accept if and only if M accepts.

We may use the empty string ε as the witness. Its length is

$$|\varepsilon| = 0,$$

so it is polynomially bounded.

If $x \in L$, then M accepts x, and hence $V(x, \varepsilon)$ accepts. If $x \notin L$, then M rejects x, so $V(x, w)$ rejects for every possible witness w.

The verifier has the same polynomial running time as M. Thus $L \in \mathsf{NP}$. Because L was an arbitrary language in P, we conclude that

$$\mathsf{P} \subseteq \mathsf{NP}.$$

■

The proof of Theorem 3.4.1 has a simple interpretation: if we can solve a problem efficiently, then we can verify its answer efficiently by solving the problem again. A witness is unnecessary in this case.

3.4.1 The P Versus NP Question

We know from Theorem 3.4.1 that every language in P is in NP. The major unresolved question is whether the reverse inclusion also holds:

$$\mathsf{P} \stackrel{?}{=} \mathsf{NP}. \tag{3.4}$$

There are two possibilities:

- If $\mathsf{P} = \mathsf{NP}$, every decision problem whose yes-instances can be verified efficiently can also be solved efficiently.
- If $\mathsf{P} \neq \mathsf{NP}$, some decision problems have efficiently verifiable yes-instances but cannot be solved by any deterministic polynomial-time algorithm.

At present, no proof of either equality or inequality is known. It is widely conjectured that

$$\mathsf{P} \neq \mathsf{NP},$$

but a conjecture is not a theorem.

3.5 Nondeterministic Turing Machines

Nothing from Section 3.5 will be tested on quizzes or exams!

The verifier definition describes NP in terms of witnesses. We now introduce a machine model that captures the same existential behavior.

3.5.1 Nondeterministic Computation

A deterministic Turing machine has at most one legal next move from any configuration. A nondeterministic Turing machine may have several possible next moves.

Definition 3.5.1 (Nondeterministic Turing machine)**.** *A (single-tape)* nondeterministic Turing machine*, abbreviated NTM, is a Turing machine whose transition relation may specify multiple possible next moves for a given state and tape symbol.*

In a standard single-tape model, its transition relation may be written as

$$\delta : Q \times \Gamma \longrightarrow \mathcal{P}\big(Q \times \Gamma \times \{L, S, R\}\big),$$

where:

- Q *is the finite set of states;*
- Γ *is the tape alphabet;*
- L, S, *and* R *denote left, stay, and right head movements;*
- $\mathcal{P}(\mathcal{S})$ *denotes the power set of a set* $\mathcal{S}$.

$\Diamond$

Thus, $\delta(q, a)$ is a set of legal transitions rather than a single transition. If this set contains several transitions, the computation branches into several possible computations.

An inaccurate-yet-helpful analogy

It is sometimes helpful to imagine that a nondeterministic machine explores all of its choices simultaneously. Under this analogy, one branch tries one candidate witness, another branch tries a different candidate witness, and so on.

This analogy helps us visualize the computation tree, but it is not the formal definition. A nondeterministic Turing machine is a mathematical model. It does not assert that an ordinary physical computer can execute exponentially many computations for free.

Computation Trees

A computation of an NTM can be represented as a rooted tree:

- The root is the initial *configuration*. Here, "configuration" means the current state, tape contents, and head position; it is a snapshot of the machine's status at the current moment.
- Each child of a configuration represents one legal next move.
- Each root-to-leaf path is a computation branch.
- A leaf may be an accepting configuration or a rejecting configuration.

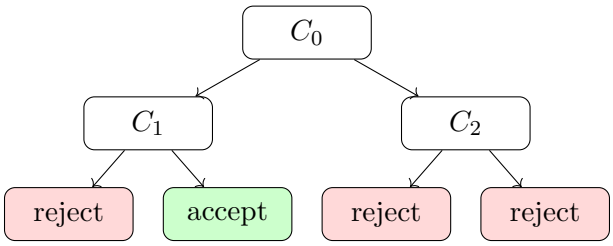


Figure 3.2: A nondeterministic computation accepts because at least one branch reaches an accepting configuration.

Definition 3.5.2 (Acceptance by an NTM)**.** *A nondeterministic Turing machine* N *accepts an input* x *if at least one computation branch of* N *on* x *reaches an accepting state.*

The machine rejects x *if every computation branch reaches a rejecting state.* $\Diamond$

Equivalently,

$$x \in L(N) \iff \exists \text{ an accepting computation branch of } N \text{ on } x. \tag{3.5}$$

The existential quantifier in Equation (3.5) is essential. In Figure 3.2, the machine accepts even though three of its four branches reject.

Remark 3.5.1 (Nondeterminism is not randomness)**.** *A nondeterministic machine does not choose a branch according to a probability distribution. It accepts whenever at least one branch accepts, regardless of how many other branches reject. A randomized machine, by contrast, is analyzed using the probability that it accepts.*

Running Time of an NTM

A nondeterministic computation may have many branches of different lengths. Its running time must account for all of them.

Definition 3.5.3 (Nondeterministic running time)**.** *The running time of a nondeterministic Turing machine N on input x is the maximum number of steps used by any computation branch of N on x.*

The machine N runs in time $T(n)$ if, for every input x of length n, every computation branch halts within at most $T(n)$ steps. ◊

Using the maximum branch length ensures that a polynomial-time NTM cannot hide an extremely long or nonhalting branch. For a nondeterministic polynomial-time decider, every branch must halt in polynomial time.

3.5.2 The Guess-and-Check View

High-level descriptions of nondeterministic algorithms often use the word *guess*. A typical nondeterministic algorithm has the following form:

1. Nondeterministically guess a candidate witness w.
2. Deterministically check whether w is valid.
3. Accept if the check succeeds; otherwise, reject this branch.

Here, guessing does not mean making a random choice. It means branching over the possible choices. The machine accepts if one branch guesses a valid witness.

> You can think of "guess" as a magical command with the following behavior: if a valid witness exists, the command automatically finds one. If no valid witness exists, however, the command cannot magically create one and will only return an invalid candidate. In either case, the command does not tell you whether the returned candidate is valid. You must therefore design a mechanism that verifies its validity.

3.5.3 The Nondeterministic Definition of NP

We can now state the machine-based definition of NP.

Definition 3.5.4 (The class NP: nondeterministic definition)**.** *The class* NP *consists of all languages decidable by a nondeterministic Turing machine in polynomial time.* ◊

If $\mathrm{NTIME}(T(n))$ denotes the class of languages decidable by nondeterministic Turing machines in time $O(T(n))$, then

$$\mathsf{NP} = \bigcup_{k \geq 1} \mathrm{NTIME}(n^k). \tag{3.6}$$

At first, Definition 3.5.4 appears different from the verifier definition in Definition 3.2.3. One definition refers to witnesses, while the other refers to branching computations. We next prove that the two definitions describe exactly the same class.

3.6 Equivalence of the Two Definitions of NP

Nothing from Section 3.6 will be tested on quizzes or exams!

We start with an (informal) example of a nondeterministic algorithm for CLIQUE.

Example 3.6.1 (A nondeterministic algorithm for CLIQUE)**.** *On input $\langle G, k \rangle$, a nondeterministic machine N operates as follows:*

1. *Nondeterministically choose (or "guess") a witness candidate:*

$$v_1, v_2, \ldots, v_k.$$

2. *Reject if the chosen vertices are not distinct.*

3. *For every $i < j$, check whether*

$$\{v_i, v_j\} \in E(G).$$

4. *Accept if every required edge exists.*

If G contains a clique of size k, at least one branch chooses its vertices and accepts. If G contains no clique of size k, every branch chooses a set that fails at least one check, so every branch rejects.

Each individual branch chooses only k vertices and checks at most

$$\binom{k}{2}$$

edges. Therefore, each branch runs in polynomial time.

The above example illustrates the guess-and-check view of nondeterministic computation. The machine guesses a candidate clique and checks whether it is valid. If a valid clique exists, at least one branch guesses it and accepts. This gives us a impression that it seems we can always "guess" the correct witness (if one indeed exist) and verify it in polynomial time. Conversely, if we have a verifier that can check a witness in polynomial time, we can construct a nondeterministic machine that guesses the witness and verifies it.

Actually, this impression is correct, and we can formalize it as follows: the connection between Definition 3.5.4 and Definition 3.2.3 is that a witness can describe an accepting branch, and a nondeterministic branch can guess a witness.

Theorem 3.6.1 (Verifier–nondeterminism equivalence)**.** *For every language L, the following statements are equivalent:*

1. *L has a deterministic polynomial-time verifier using polynomially bounded witnesses.*

2. L is decidable by a nondeterministic polynomial-time Turing machine.

$\Diamond$

3.6.1 From Verifiers to Nondeterministic Machines

We first show that efficient verification implies efficient nondeterministic computation.

Lemma 3.6.1. *If a language L has a deterministic polynomial-time verifier using polynomially bounded witnesses, then L is decidable by a nondeterministic polynomial-time Turing machine.* $\Diamond$

Proof. Suppose that V is a deterministic polynomial-time verifier for L. Let p be a polynomial such that

$$x \in L \iff \exists w,\ |w| \le p(|x|) \text{ and } V(x, w) = 1.$$

Construct a nondeterministic Turing machine N that operates as follows on input x:

1. Nondeterministically generate a string w of length at most $p(|x|)$.
2. Run the deterministic verifier V on (x, w).
3. Accept if and only if $V(x, w)$ accepts.

We prove correctness in both cases.

Case 1: $x \in L$

By the verifier property, there is a witness w satisfying

$$|w| \le p(|x|) \qquad \text{and} \qquad V(x, w) = 1.$$

One branch of N generates this particular witness. On that branch, V accepts. Therefore, N accepts x.

Case 2: $x \notin L$

By soundness of the verifier, no string w of the allowed length satisfies $V(x, w) = 1$. Hence every branch of N generates a witness that V rejects. Therefore, every branch rejects, and N rejects x.

It remains to analyze the running time. Generating at most $p(|x|)$ symbols takes polynomial time. Because V runs in polynomial time and $|w|$ is polynomially bounded in $|x|$, running $V(x, w)$ also takes polynomial time in $|x|$. Thus every branch of N halts in polynomial time.

Therefore, N is a nondeterministic polynomial-time decider for L.

■

The main idea of Lemma 3.6.1 is that the branches of N correspond to the possible witnesses. The NTM accepts exactly when one of those witnesses passes verification.

3.6.2 From Nondeterministic Machines to Verifiers

We now prove the reverse direction. An accepting branch of a nondeterministic computation will become the witness.

Lemma 3.6.2. *If a language L is decidable by a nondeterministic polynomial-time Turing machine, then L has a deterministic polynomial-time verifier using polynomially bounded witnesses.* $\Diamond$

Proof. Suppose that a nondeterministic Turing machine N decides L in time $p(n)$, where p is a polynomial. Thus, on every input x, every computation branch halts within

$$p(|x|)$$

steps.

Because N has a finite transition relation, there is a constant $b \leq |\mathcal{P}(Q \times \Gamma \times \{L, R, S\})|$ such that each configuration has at most b legal next moves. Note that this b depends only on the TM N and not on the input x, and thus it is a constant for the purposes of this proof.

Number these choices

$$1, 2, \ldots, b.$$

A computation branch can be described by a sequence

$$w = c_1 c_2 \cdots c_t, \qquad t \leq p(|x|),$$

where c_i specifies which transition to follow at the i-th nondeterministic step.

Each choice can be encoded using at most

$$\lceil \log_2 b \rceil$$

bits. Because b is a constant depending only on N, the witness length is at most

$$p(|x|) \lceil \log_2 b \rceil , \tag{3.7}$$

which is polynomial in $|x|$.

Construct a deterministic verifier V as follows:

1. Interpret w as a sequence of nondeterministic choices.
2. Begin simulating N on input x.
3. Whenever N has several legal transitions, use the next choice encoded in w.
4. Reject if w specifies an illegal transition or fails to specify a required choice.
5. Accept if and only if the simulated branch reaches an accepting state.

We again prove correctness in both cases.

Case 1: $x \in L$

Because N accepts x, at least one computation branch is accepting. Let w encode the sequence of choices made along that branch. The verifier follows exactly those choices and reaches the accepting state. Therefore, $V(x, w) = 1$.

Case 2: $x \notin L$

Because N decides L, every computation branch rejects x. Any well-formed witness describes one of these rejecting branches, and a malformed witness is rejected by the verifier. Hence no witness causes V to accept.

Finally, the verifier simulates at most $p(|x|)$ steps of N. A deterministic Turing machine can simulate each selected transition with at most polynomial overhead. Therefore, V runs in polynomial time. By Equation (3.7), its witnesses also have polynomial length.

Thus V is a deterministic polynomial-time verifier for L. ∎

Chapter 4
Reducibility and NP-Completeness

In the previous chapter, we introduced the complexity classes P and NP, together with several important decision problems such as SAT and CLIQUE. We now develop a systematic way to compare the computational difficulty of such problems.

The central tool is a *polynomial-time reduction*. A reduction translates instances of one problem into instances of another while preserving the correct answer. The requirement that the translation run in polynomial time is essential: without it, the translation itself might perform all of the difficult computation.

After defining reductions, we introduce NP-hardness and NP-completeness. We then study a concrete reduction from 3SAT to CLIQUE. The main result of the chapter is the Cook–Levin theorem, which states that SAT is NP-complete. Its proof shows how to encode an entire polynomial-time nondeterministic computation as a Boolean formula.

4.1 Polynomial-Time Reducibility

Let us begin with the basic idea. Suppose that we want to decide membership in a language L_A, but we already have an algorithm for another language L_B. If we can efficiently transform every input x for L_A into an input $f(x)$ for L_B, in such a way that

$$x \in L_A \quad \Longleftrightarrow \quad f(x) \in L_B,$$

then an algorithm for L_B can be used to decide L_A.

There are two separate requirements here:

- The transformation must preserve the correct yes-or-no answer.
- The transformation must itself be computationally efficient.

The second requirement motivates the following definition.

4.1.1 Polynomial-Time Computable Functions

Definition 4.1.1 (Polynomial-time computable function)**.** *A function*

$$f : \Sigma^* \longrightarrow \Sigma^*$$

is polynomial-time computable *if there is a deterministic Turing machine M_f and a polynomial p such that M_f computes f in time $p(n)$ as per Definition 1.2.3.* $\Diamond$

A polynomial-time machine can write at most one tape symbol per step. Consequently, if f is polynomial-time computable, then the output length $|f(x)|$ is bounded by a polynomial in $|x|$. Thus, the definition controls both the time needed to construct $f(x)$ and the length of the constructed instance.

4.1.2 Polynomial-Time Reductions

Definition 4.1.2 (Polynomial-time reduction)**.** *Let $L_A, L_B \subseteq \Sigma^*$ be languages. We say that L_A is* polynomial-time mapping reducible *to L_B, written*

$$L_A \leq_{\mathrm{P}} L_B,$$

if there is a polynomial-time computable function

$$f : \Sigma^* \longrightarrow \Sigma^*$$

such that, for every $x \in \Sigma^*$,

$$x \in L_A \iff f(x) \in L_B. \tag{4.1}$$

The function f is called a polynomial-time reduction *from L_A to L_B.* ◊

Some texts call this notion *polynomial-time many-one reducibility.*

Example 4.1.1 (A simple polynomial-time reduction)**.** *Consider the languages*

$$L_A \coloneqq \{x\#y : x, y \in \{0,1\}^*,\ |x| = |y|,\ \text{and } x = y\}$$

where $\#$ is a special separator symbol not in $\{0,1\}$, and

$$L_B \coloneqq \{zz : z \in \{0,1\}^*\}.$$

Thus, L_A contains two equal strings separated by $\#$, while L_B contains strings consisting of two identical halves.

Define f as follows:

$$f(u) \coloneqq \begin{cases} xy, & \text{if } u = x\#y \text{ with } |x| = |y|, \\ 01, & \text{otherwise.} \end{cases}$$

Parsing the input and comparing the two lengths can be done in polynomial time. Notice that the reduction does not need to test whether $x = y$.

If $u = x\#y$ and $|x| = |y|$, then

$$u \in L_A \iff x = y \iff xy = xx \iff f(u) \in L_B.$$

If u is not of the required form, then $f(u) = 01$, which is not in L_B. Therefore,

$$L_A \leq_{\mathrm{P}} L_B.$$

It is important to notice that Example 4.1.1 demonstrates the structure of a reduction: transform the input efficiently and then prove a two-directional correctness statement. It also demonstrates that the reduction does not need to solve the original problem; it only needs to preserve the correct answer.

4.1.3 Using a Reduction to Obtain an Algorithm

The main algorithmic consequence of a reduction is the following theorem.

Theorem 4.1.1 (Reduction to a polynomial-time problem)**.** *If*

$$L_A \leq_{\mathrm{P}} L_B \qquad \text{and} \qquad L_B \in \mathsf{P},$$

then $L_A \in \mathsf{P}$. $\Diamond$

Proof. Let f be a polynomial-time reduction from L_A to L_B, and let M_B be a deterministic polynomial-time decider for L_B. Construct a machine M_A that operates as follows.

Algorithm 4.1.1: Deciding L_A using a reduction to L_B

On input x, $M_A(x)$ works as follows:

1. Compute $y = f(x)$.
2. Run M_B on input y.
3. Accept if M_B accepts, and reject if M_B rejects.

The correctness of M_A follows from the defining property of the reduction:

$$x \in L_A \iff f(x) \in L_B \iff M_B \text{ accepts } f(x).$$

It remains to analyze the running time. Suppose that f is computed in time $p(n)$, and M_B runs in time $q(m)$ on inputs of length m, where p and q are polynomials. Since a machine running for $p(n)$ steps can write at most $p(n)$ output symbols,

$$|f(x)| \le p(|x|).$$

Therefore, the total running time is at most

$$p(|x|) + q\big(|f(x)|\big) \le p(|x|) + q\big(p(|x|)\big).$$

A composition of polynomials is a polynomial, so M_A runs in polynomial time. Hence $L_A \in \mathsf{P}$. ■

The direction of a reduction matters. The statement $L_A \le_{\mathrm{P}} L_B$ means that an algorithm for L_B can be used to solve L_A. Thus, the arrow points from the problem we want to solve toward the problem whose algorithm we want to use.

4.1.4 Composition of Reductions

Polynomial-time reductions may be chained together.

Proposition 4.1.1 (Transitivity of polynomial-time reductions)**.** *If*

$$L_A \le_{\mathrm{P}} L_B \qquad \text{and} \qquad L_B \le_{\mathrm{P}} L_C,$$

then

$$L_A \le_{\mathrm{P}} L_C.$$

Proof. Let f reduce L_A to L_B, and let g reduce L_B to L_C. Define

$$h(x) := g(f(x)).$$

For every $x \in \Sigma^*$,

$$\begin{aligned} x \in L_A &\iff f(x) \in L_B \\ &\iff g(f(x)) \in L_C \\ &\iff h(x) \in L_C. \end{aligned}$$

Thus, h preserves membership.

Because f and g are polynomial-time computable, the output length $|f(x)|$ is polynomially bounded in $|x|$, and computing $g(f(x))$ takes polynomial time in $|x|$. Therefore, h is polynomial-time computable, and $L_A \leq_{\mathrm{P}} L_C$.

■

To prove that a target problem is computationally hard, reduce a problem already known to be hard *to* the target problem.

4.2 NP-Hardness and NP-Completeness

Polynomial-time reductions allow us to compare individual problems. We now use them to identify languages that capture the difficulty of the entire class NP.

Definition 4.2.1 (NP-hardness)**.** *A language L_B is* NP-hard *if, for every language $L_A \in \mathsf{NP}$,*

$$L_A \leq_{\mathrm{P}} L_B.$$

◊

An NP-hard language is at least as hard, under polynomial-time reductions, as every language in NP. The definition does not require $L_B \in \mathsf{NP}$. Indeed, an NP-hard problem need not even be decidable.

Definition 4.2.2 (NP-completeness)**.** *A language L_B is* NP-complete *if it satisfies both of the following conditions:*

1. *$L_B \in \mathsf{NP}$;*
2. *L_B is NP-hard.*

◊

Thus, an NP-complete language is both efficiently verifiable and at least as hard as every other language in NP.

The two obligations in an NP-completeness proof

To prove that a language L_B is NP-complete, we must prove two separate claims:

- *Membership in* NP*:* describe a polynomial-size certificate and a polynomial-time verifier for L_B.
- *NP-hardness:* show that every language in NP reduces to L_B, usually by reducing a previously known NP-complete language to L_B.

Theorem 4.2.1 (Consequence of a polynomial-time NP-complete problem)**.** *If an NP-complete language L_B belongs to* P*, then*

$$\mathsf{P} = \mathsf{NP}.$$

◊

Proof. Because L_B is NP-hard, every $L_A \in \mathsf{NP}$ satisfies $L_A \leq_{\mathrm{P}} L_B$. If $L_B \in \mathsf{P}$, then Theorem 4.1.1 implies that every $L_A \in \mathsf{NP}$ belongs to P. Therefore,

$$\mathsf{NP} \subseteq \mathsf{P}.$$

We always have $\mathsf{P} \subseteq \mathsf{NP}$, so $\mathsf{P} = \mathsf{NP}$. ■

Once we know one NP-complete language, we can prove that other languages are NP-complete by reducing from it.

Theorem 4.2.2 (NP-completeness transfer)**.** *Suppose that L_B is NP-complete, $L_B \leq_{\mathrm{P}} L_C$, and $L_C \in \mathsf{NP}$. Then L_C is NP-complete.* ◊

Proof. The assumption $L_C \in \mathsf{NP}$ establishes the membership requirement. It remains to prove that L_C is NP-hard.

Let L_A be any language in NP. Since L_B is NP-complete,

$$L_A \leq_{\mathrm{P}} L_B.$$

By assumption,

$$L_B \leq_{\mathrm{P}} L_C.$$

The transitivity result in Proposition 4.1.1 gives

$$L_A \leq_{\mathrm{P}} L_C.$$

Since this holds for every $L_A \in \mathsf{NP}$, the language L_C is NP-hard. Together with $L_C \in \mathsf{NP}$, this proves that L_C is NP-complete.

■

4.3 Example: Reduce 3SAT to CLIQUE

We show as an example the reduction connecting a logical problem to a graph problem. Although Boolean formulas and graphs appear quite different, the reduction will encode a consistent choice of true literals as a clique.

4.3.1 The Language 3SAT

Let us first recall some notion from Section 3.3.1.

Definition 4.3.1 (Literals, clauses, CNF, and 3-CNF)**.** *Let x be a Boolean variable.*

- *A* literal *is either a variable x or its negation $\neg x$.*
- *A* clause *is a disjunction of literals.*
- *A Boolean formula is in* conjunctive normal form*, or* CNF*, if it is a conjunction of clauses.*

A formula is in 3-CNF *if every clause contains exactly three literals.* ◊

For example,

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge (x_2 \vee \neg x_3 \vee \neg x_4)$$

is a 3-CNF formula.

A CNF formula is satisfied exactly when every clause contains at least one true literal.

Definition 4.3.2 (3SAT)**.** *The language* 3SAT *is*

$$\text{3SAT} \coloneqq \{\langle \varphi \rangle : \varphi \textit{ is a satisfiable 3-CNF formula}\}.$$

$\Diamond$

We permit repeated literals in a clause. For example,

$$(x \vee y \vee y)$$

is treated as a three-literal clause. This convention will be useful when converting shorter clauses to 3-CNF.

Example 4.3.1 (Satisfying a 3-CNF formula)**.** *Consider*

$$\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge (\neg x_3 \vee \neg x_4 \vee x_2).$$

The assignment

$$x_1 = 1, \qquad x_2 = 1, \qquad x_3 = 0, \qquad x_4 = 0$$

satisfies φ. For example, we may identify one true literal in each clause:

$$\underbrace{x_1}_{\textit{true}}, \qquad \underbrace{x_2}_{\textit{true}}, \qquad \underbrace{\neg x_3}_{\textit{true}}.$$

The three selected literals are mutually consistent: no selected literal is the negation of another selected literal.

4.3.2 The Target Problem

Recall that a *clique* in an undirected graph is a set of vertices in which every two distinct vertices are connected by an edge. The decision language is

$$\text{CLIQUE} \coloneqq \{\langle G, k \rangle : G \text{ is an undirected graph containing a clique of size at least } k\}.$$

Our reduction will create one group of vertices for each clause. A clique will select one vertex from every group, and the graph's edges will ensure that the selected literals do not contradict one another.

Theorem 4.3.1.

$$\text{3SAT} \leq_{\text{P}} \text{CLIQUE}.$$

$\Diamond$

Proof. Let the input be a 3-CNF formula

$$\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_m, \tag{4.2}$$

where each clause C_i contains exactly three literal occurrences.

We construct an undirected graph G_φ.

Vertices. For each literal occurrence in each clause, create one vertex. If

$$C_i = (\ell_{i,1} \vee \ell_{i,2} \vee \ell_{i,3}),$$

then clause C_i contributes the three vertices

$$v_{i,1}, \quad v_{i,2}, \quad v_{i,3}.$$

Label $v_{i,r}$ by the corresponding literal $\ell_{i,r}$. Notice that two occurrences of the same literal in different clauses produce two different vertices.

Edges. Add an edge between $v_{i,r}$ and $v_{j,s}$ exactly when both conditions hold simultaneously:

1. $i \neq j$, so the vertices come from different clauses; and
2. $\ell_{i,r}$ and $\ell_{j,s}$ are not *contradictory*, where "contradictory" is defined as follows: Two literals are contradictory if one is x and the other is $\neg x$ for the same variable x.

Note that there are no edges between vertices arising from the same clause.

The reduction outputs

$$f(\langle\varphi\rangle) \coloneqq \langle G_\varphi, m\rangle.$$

We prove that

$$\varphi \text{ is satisfiable} \quad \Longleftrightarrow \quad G_\varphi \text{ has a clique of size } m. \tag{4.3}$$

Forward direction. Suppose that φ has a satisfying assignment. Every clause contains at least one true literal. From each clause C_i, select one vertex whose literal is true.

Exactly m vertices are selected, one from each clause. Consider any two selected vertices. They come from different clauses, and their literals cannot be contradictory because both literals are true under the same assignment. Therefore, the construction places an edge between every pair of selected vertices. The selected vertices form an m-clique.

Reverse direction. Suppose that G_φ contains a clique K of size m. Because no two vertices from the same clause are adjacent, K contains at most one vertex from each clause group. There are m clause groups and K has m vertices, so K contains exactly one vertex from every clause.

No two literals labeling vertices of K are contradictory. We may therefore assign truth values so that every selected literal is true:

- if K contains a vertex labeled x, assign $x = 1$;
- if K contains a vertex labeled $\neg x$, assign $x = 0$;
- assign arbitrary values to variables not represented in K.

This assignment is well-defined because K cannot contain both x and $\neg x$. Each clause has one selected vertex, and the literal labeling that vertex is true. Hence every clause is satisfied, so φ is satisfiable.

Running time. The graph contains $3m$ vertices. We may examine every pair of vertices, test whether they come from different clauses, and test whether their labels contradict each other. There are at most

$$\binom{3m}{2} = O(m^2)$$

pairs. Thus, G_φ and its edge set can be produced in polynomial time.

The construction is therefore a polynomial-time computable function, and Equation (4.3) proves that it is a reduction. ■

Example 4.3.2 (The clique selected by a satisfying assignment)**.** *Return to the formula from Example 4.3.1:*

$$\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge (\neg x_3 \vee \neg x_4 \vee x_2).$$

The assignment

$$x_1 = 1, \qquad x_2 = 1, \qquad x_3 = 0, \qquad x_4 = 0$$

allows us to select the literals

$$x_1, \qquad x_2, \qquad \neg x_3.$$

They come from three different clauses, and no two are contradictory. Consequently, the corresponding three vertices are pairwise adjacent and form a 3-clique.

By contrast, a vertex labeled x_1 is not adjacent to a vertex labeled $\neg x_1$, even when those vertices come from different clauses. The missing edge prevents a clique from making logically inconsistent choices.

> In the reduction from 3SAT to CLIQUE, choosing one vertex from each clause represents choosing one literal that will satisfy that clause. Edges represent compatibility between the choices.

At this point, Theorem 4.3.1 tells us that a polynomial-time algorithm for CLIQUE would yield a polynomial-time algorithm for 3SAT. To conclude that CLIQUE is NP-complete, we will need to establish that 3SAT is NP-complete. The Cook–Levin theorem provides the starting point.

4.4 Warm-Up: Describing Computations with CNF Formulas

The Cook–Levin theorem encodes a Turing-machine computation as a Boolean formula. Before presenting the full proof, we develop the logical components of that encoding.

The main idea is that a Turing-machine transition is *local*. During one step, the machine changes only the tape cell under its head, its state, and the head position. A Boolean formula can describe these local restrictions, and the conjunction of all local restrictions can describe an entire computation.

4.4.1 Clauses as Constraints

A clause is false only when all of its literals are false. We may therefore use clauses to exclude unwanted assignments.

Example 4.4.1 (Implication and equivalence)**.** *The implication $x \to y$ is false only when $x = 1$ and $y = 0$. It can be written as*

$$x \to y \equiv \neg x \vee y.$$

Similarly, $x \leftrightarrow y$ requires both implications:

$$\begin{aligned} x \leftrightarrow y &\equiv (x \to y) \wedge (y \to x) \\ &\equiv (\neg x \vee y) \wedge (x \vee \neg y). \end{aligned}$$

The first clause rules out $x = 1, y = 0$, and the second rules out $x = 0, y = 1$.

Exactly one variable is true

Suppose that exactly one of

$$y_1, y_2, \ldots, y_r$$

should be true. We divide the requirement into two parts.

At least one variable must be true:

$$y_1 \vee y_2 \vee \cdots \vee y_r. \tag{4.4}$$

No two different variables may both be true:

$$\bigwedge_{1 \le a < b \le r} (\neg y_a \vee \neg y_b). \tag{4.5}$$

The conjunction of Equations (4.4) and (4.5) says that exactly one of the r variables is true. We will use this construction to assert that every cell of a computation table contains exactly one symbol.

4.4.2 Forbidding a Local Assignment

Suppose that we want to forbid the assignment

$$(y_1, y_2, y_3) = (1, 0, 1).$$

The clause

$$\neg y_1 \vee y_2 \vee \neg y_3$$

is false exactly under this assignment. Thus, adding the clause excludes the forbidden pattern while permitting every other assignment.

More generally, suppose we want to forbid

$$(y_1, \ldots, y_r) = (b_1, \ldots, b_r), \qquad b_i \in \{0, 1\}.$$

For each position i, define

$$\ell_i := \begin{cases} \neg y_i, & \text{if } b_i = 1, \\ y_i, & \text{if } b_i = 0. \end{cases}$$

Then

$$\ell_1 \vee \ell_2 \vee \cdots \vee \ell_r \tag{4.6}$$

is false exactly on the assignment $(b_1, \ldots, b_r)$.

This observation gives a general method for encoding a relation on a constant number of Boolean variables:

1. List all assignments that violate the relation.
2. For each violating assignment, add one clause that forbids it.
3. Take the conjunction of all those clauses.

If the number of variables is constant, there are only constantly many possible assignments, so the resulting formula has constant size.

4.4.3 Encoding the Contents of One Cell

Suppose a table cell must contain exactly one symbol from a finite set

$$D = \{s_1, s_2, \ldots, s_d\}.$$

Introduce a Boolean variable x_s for each $s \in D$, with the intended meaning

$$x_s = 1 \quad \Longleftrightarrow \quad \text{the cell contains } s.$$

The exactly-one constraint is

$$\left(\bigvee_{s \in D} x_s \right) \wedge \left(\bigwedge_{\substack{s,t \in D \\ s \neq t}} (\neg x_s \vee \neg x_t) \right). \tag{4.7}$$

The first part says that the cell contains at least one symbol. The second part says that the cell cannot contain two different symbols.

In the Cook–Levin proof, D will depend on a fixed Turing machine, not on the input. Consequently, $|D|$ is a constant, and Equation (4.7) has constant size for each table cell.

4.4.4 Local Views of Turing-Machine Computations

We represent one configuration of a Turing machine as a row of symbols. To record both the state and the head position, we use a marked symbol

$$\langle q, a \rangle$$

to mean that the machine is in state q, its head is at this cell, and the cell contains tape symbol a. Every other cell contains an ordinary tape symbol.

Suppose, for example, that the transition function permits

$$(q_1, a) \longmapsto (q_2, b, R).$$

If the head is scanning a and the cell to its right contains c, then the local change is

d	$\langle q_1, a \rangle$	c
d	b	$\langle q_2, c \rangle$

The machine writes b, changes to state q_2, and moves right onto the cell containing c.

Cells far from the head remain unchanged. A window such as

a	b	c
a	b	c

is therefore legal when the head is outside this local region.

In contrast,

a	b	c
a	d	c

is illegal if no state symbol appears nearby. The middle tape symbol cannot change spontaneously.

Definition 4.4.1 (Legal computation window)**.** *Fix a Turing machine N. A 2×3 window is* legal for N *if its lower row can occur beneath its upper row when N performs one transition,*

including the unchanged behavior of cells away from the head and the prescribed behavior at tape boundaries. ◊

Because N has a finite state set and a finite tape alphabet, there are only finitely many possible 2×3 windows. Once N is fixed, the set of legal windows is also fixed.

Encoding legal windows

A window contains six cells. Suppose that

$$(s_1, s_2, s_3, s_4, s_5, s_6)$$

is an illegal choice of contents for those cells. Let the corresponding cell variables be

$$y_{1,s_1}, y_{2,s_2}, \ldots, y_{6,s_6}.$$

The clause

$$\neg y_{1,s_1} \vee \neg y_{2,s_2} \vee \cdots \vee \neg y_{6,s_6} \tag{4.8}$$

forbids exactly that window pattern, provided each cell is already constrained to contain exactly one symbol.

There are only constantly many illegal patterns because the machine and its alphabet are fixed. We may forbid all of them using a constant number of constant-width clauses. Finally, each clause can be converted to 3-CNF using Lemma 4.6.1.

A Turing-machine transition is a global statement about two configurations, but its correctness can be checked using overlapping constant-size windows. Constant-size local conditions can be encoded by constant-size Boolean formulas.

4.5 The Cook–Levin Theorem

We are now ready to prove the first NP-completeness result. The proof explains why Boolean satisfiability is expressive enough to represent every polynomial-time nondeterministic computation.

Theorem 4.5.1 (Cook–Levin theorem)**.** *The language* SAT *is NP-complete.* ◊

Supplementary Box 4.5.1: Some Historical Remarks

The Cook–Levin theorem arose from two independent discoveries made on opposite sides of the Cold War. In 1971, Stephen Cook, then at the University of Toronto, published "The Complexity of Theorem-Proving Procedures," in which he showed that Boolean satisfiability captures the difficulty of every problem solvable by a nondeterministic polynomial-time algorithm [Coo71]. Cook's work introduced the central ideas underlying polynomial-time reducibility and NP-completeness, thereby providing the foundation for the modern theory of computational complexity.

Independently, Leonid Levin developed a closely related theory in the Soviet Union. His results appeared in Russian in 1973 under the title "Universal Search Problems," where he identified several computational search problems that are universal in essentially the same sense [Lev73]. Because Cook and Levin worked in different scientific communities and Levin's work was not initially well known in the West, the connection between their discoveries was recognized only

later [ACM12]. The result is therefore now called the *Cook–Levin theorem*, acknowledging their independent establishment of the first NP-completeness theorem.

4.5.1 Proof of the Cook–Levin Theorem

We need to prove two things. First, we need to show that SAT $\in$ NP. Second, we need to prove that every language $L_A \in$ NP is polynomial-time reducible to SAT.

The first part is straightforward. We now focus on the second part.

At a high level, we encode an accepting computation of a nondeterministic Turing machine as a Boolean formula and prove the correctness of the encoding in both directions. Finally, we verify that the formula can be constructed in polynomial time.

4.5.1.1 Model, Normalization, and Computation Tableau

Let L_A be an arbitrary language in NP. By the definition of NP, there is a nondeterministic single-tape Turing machine N with a one-way infinite tape deciding L_A in time

$$T(n) = n^k$$

for some constant k, after increasing the polynomial bound if necessary. The tape has a leftmost cell and extends infinitely to the right. The head may move left and right, but it may not move to the left of the leftmost tape cell.

We make the following harmless normalizations:

- $T(n) \geq n$ for every relevant input length n;
- N has designated accept and reject states;
- the transition relation of N respects the left endpoint of the tape, so no transition moves the head to the left of the leftmost tape cell;
- once N enters the accept state, it remains accepting without changing the relevant tape contents;
- every branch can be padded so that it has exactly $T(n)$ transitions.

These changes increase the running time by at most a polynomial factor and therefore do not change the fact that N runs in polynomial time.

Goal of the construction

Fix an input x, and write

$$n \coloneqq |x|, \qquad T \coloneqq T(n).$$

We will construct a Boolean formula φ_x that describes a T-step accepting computation branch of N on x. The correctness of the construction will be established by proving the two directions of

$$x \in L_A \quad \Longleftrightarrow \quad \varphi_x \text{ is satisfiable.} \tag{4.9}$$

The forward direction will encode an accepting computation as a satisfying assignment. The reverse direction will decode a satisfying assignment into an accepting computation.

The computation tableau

A *tableau* is a rectangular table whose rows describe consecutive configurations of one computation branch of N on x.

The tableau has $T + 1$ rows, indexed by

$$i \in \{0, 1, \ldots, T\}.$$

Row 0 is the initial configuration, and row i describes the configuration after i transitions.

The tableau has $S \coloneqq T + 3$ columns, indexed by

$$j \in \{1, 2, \ldots, S\}.$$

Within each row, Column 1 contains a fixed left boundary marker, and column S contains a fixed right boundary marker. The remaining $T+1$ columns represent the first $T+1$ cells of the one-way infinite tape. More precisely, tape cell r, for $1 \le r \le T + 1$, is represented by tableau column $r + 1$.

The $T+1$ working columns are sufficient because the head starts on the leftmost tape cell and can move to the right by at most T cells during T transitions.[1] Thus, during the computation, the head can scan only tape cells

$$1, 2, \ldots, T + 1.$$

The marker in column 1 represents the fixed left endpoint of the tape. The marker in column $S = T+3$ is an artificial right boundary placed one column beyond the portion of the tape that can be reached within T steps.

Let Q be the state set of N, let Γ be its tape alphabet, and let $\# \notin \Gamma$ be a boundary marker. Define the tableau alphabet

$$D \coloneqq \Gamma \cup (Q \times \Gamma) \cup \{\#\}. \tag{4.10}$$

An unmarked symbol $a \in \Gamma$ represents a tape cell containing a that is not currently scanned by the head. A marked symbol $\langle q, a \rangle \in Q \times \Gamma$ records that the head is scanning a cell containing a while the machine is in state q.

Thus, a tableau has the following conceptual form:

$$\begin{array}{cccccccc}
\# & \langle q_0, x_1 \rangle & x_2 & \cdots & x_n & \sqcup & \cdots & \# \\
\# & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \# \\
\vdots & & & & & & & \vdots \\
\# & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \#
\end{array}.$$

An *accepting tableau* satisfies all of the following conditions:

1. Every cell contains exactly one symbol from D.
2. The first row is the start configuration of N on x.
3. Every pair of consecutive rows describes a legal transition of N.
4. The final row contains the accepting state.

If N accepts x, the configurations along an accepting branch form an accepting tableau, after padding the branch to length T if necessary. Conversely, the rows of an accepting tableau describe an accepting computation branch of N on x. Therefore,

$$N \text{ accepts } x \quad \Longleftrightarrow \quad \text{an accepting tableau for } N \text{ on } x \text{ exists.} \tag{4.11}$$

We now construct a formula whose satisfying assignments describe exactly these accepting tableaux.

[1] Note that within T moves, the Turing machine N can visit $T + 1$ tape cells because it begins on one cell before making any move. If it moves in the same direction at every step, each move reaches one new cell; thus, the initial cell plus T newly reached cells requires $T + 1$ cells in total.

4.5.1.2 Constraints as Boolean Formulas

For every row i, column j, and symbol $s \in D$, introduce a Boolean variable

$$X_{i,j,s}.$$

Its intended meaning is

$$X_{i,j,s} = 1 \quad \Longleftrightarrow \quad \text{cell } [i,j] \text{ contains symbol } s. \tag{4.12}$$

The complete formula is

$$\varphi_x \coloneqq \varphi_{\text{cell}} \wedge \varphi_{\text{boundary}} \wedge \varphi_{\text{start}} \wedge \varphi_{\text{move}} \wedge \varphi_{\text{accept}}. \tag{4.13}$$

The five components enforce, respectively, that every cell has exactly one symbol, that the first row is the initial configuration, that the boundary markers are correct, that consecutive rows are related by legal transitions, and that the final row is accepting.

(1/4) Cell constraints

For each cell $[i,j]$, we require exactly one variable $X_{i,j,s}$ to be true. This requirement consists of two parts.

First, the clause

$$\bigvee_{s \in D} X_{i,j,s}$$

requires at least one symbol to occur in the cell: if this disjunction is satisfied, then at least one of its literals $X_{i,j,s}$ is true.

Second, for every pair of distinct symbols $s, t \in D$, the clause

$$\neg X_{i,j,s} \vee \neg X_{i,j,t}$$

prevents both $X_{i,j,s}$ and $X_{i,j,t}$ from being true simultaneously. Together, these clauses require exactly one symbol in the cell.

Taking their conjunction over every tableau cell gives

$$\varphi_{\text{cell}} \coloneqq \bigwedge_{\substack{0 \le i \le T \\ 1 \le j \le S}} \left[\left(\bigvee_{s \in D} X_{i,j,s} \right) \wedge \left(\bigwedge_{\substack{s,t \in D \\ s \neq t}} (\neg X_{i,j,s} \vee \neg X_{i,j,t}) \right) \right]. \tag{4.14}$$

Consequently, any assignment satisfying φ_{cell} assigns exactly one symbol from D to every tableau cell. Since D depends only on the fixed machine N, the number of clauses associated with each cell is constant.

Boundary constraints. In particular, the first and last columns must contain boundary markers at every time:

$$\varphi_{\text{boundary}} \coloneqq \bigwedge_{i=0}^{T} (X_{i,1,\#} \wedge X_{i,S,\#}). \tag{4.15}$$

Thus, if $\varphi_{\text{boundary}}$ is satisfied, both endpoint cells in every row contain #. The marker in column 1 represents the fixed left endpoint of the one-way infinite tape, while the marker in column S is an artificial right boundary lying beyond every tape cell that the head can reach within T steps. These fixed boundaries ensure that local windows near the endpoints are interpreted consistently.

(2/4) Start constraints

Let the initial head position be

$$c \coloneqq 2,$$

the first working column. Write the input as

$$x = x_1 x_2 \cdots x_n.$$

The first row must contain:

- # in columns 1 and S;
- $\langle q_0, x_1 \rangle$ in column $c = 2$, assuming $n > 0$;
- x_r in column $c + r - 1 = r + 1$ for $2 \le r \le n$;
- the blank symbol $\sqcup$ in every other working cell.

If $n = 0$, column $c = 2$ instead contains $\langle q_0, \sqcup \rangle$.

Because $T \ge n$, the $T + 1$ working columns are sufficient to contain the entire input and all tape cells that can be reached during the computation.

The above discussion shows that the initial configuration is *uniquely* determined by the input x. Thus, for each column j, we can use $s_j^{\text{start}} \in D$ to denote the uniquely required symbol in the initial configuration. Define

$$\varphi_{\text{start}} \coloneqq \bigwedge_{j=1}^{S} X_{0,j,s_j^{\text{start}}}. \tag{4.16}$$

Because every conjunct is a unit clause, an assignment satisfies φ_{start} only if each cell in row 0 contains its prescribed initial symbol.

(3/4) Move constraints

We next enforce that each row is a legal successor of the preceding row. The main idea is that one transition of N affects only the current head cell and its immediate neighborhood. It is therefore enough to check every 2×3 local window between two consecutive rows.

For $0 \le i < T$ and $2 \le j < S$, the (i, j)-window is the 2×3 block

$$\begin{array}{ccc} \text{cell } [i, j-1] & \text{cell } [i, j] & \text{cell } [i, j+1] \\ \text{cell } [i+1, j-1] & \text{cell } [i+1, j] & \text{cell } [i+1, j+1] \end{array}$$

consisting of three adjacent cells in each of two consecutive rows.

Let

$$W_N \subseteq D^6$$

be the set of legal 2×3 windows for the one-way infinite-tape machine N, as in Definition 4.4.1. In addition to encoding the ordinary transitions of N, these windows enforce the behavior of the left endpoint: a marked symbol may never occur in column 1, and a transition may never place the head to the left of column 2. The marker in column S is likewise never scanned by the head. Since N is fixed, both D and W_N are fixed finite sets, and their sizes do not depend on the input x. Thus, they are of *constant* size.

For each illegal pattern

$$\mathbf{a} = (a_1, a_2, a_3, a_4, a_5, a_6) \in D^6 \setminus W_N,$$

include the clause

$$\psi_{i,j,\mathbf{a}} \coloneqq \neg X_{i,j-1,a_1} \vee \neg X_{i,j,a_2} \vee \neg X_{i,j+1,a_3} \vee \\ \neg X_{i+1,j-1,a_4} \vee \neg X_{i+1,j,a_5} \vee \neg X_{i+1,j+1,a_6}. \tag{4.17}$$

To see how this clause works, first suppose that the six cells contain exactly the illegal pattern $\mathbf{a}$. Then all six corresponding variables in Equation (4.17) are true, so all six negated literals are false and the clause is false. Conversely, if the six cells do not contain $\mathbf{a}$, then at least one of the six cells contains a different symbol. The corresponding variable in Equation (4.17) is therefore false, so its negation is true and the clause is satisfied. This reasoning uses the exactly-one conditions in φ_{cell}.

We forbid every illegal pattern at every window position:

$$\varphi_{\text{move}} \coloneqq \bigwedge_{\substack{0 \le i < T \\ 2 \le j < S}} \bigwedge_{\mathbf{a} \in D^6 \setminus W_N} \psi_{i,j,\mathbf{a}}. \tag{4.18}$$

It follows that, under the cell constraints, φ_{move} is satisfied exactly when every 2×3 window between consecutive rows belongs to W_N.

The key fact connecting these local constraints to a complete transition is the following fact.

Fact 4.5.1 (Local consistency implies a legal transition)**.** *Suppose that row i is a valid configuration of the one-way infinite-tape machine N, both boundary cells are correct, and every 2×3 window between rows i and $i+1$ is legal. Then row $i+1$ is the configuration obtained from row i by one transition permitted by N.*

Fact 4.5.1 can now be applied successively to the tableau rows. The first application uses row 0: because φ_{start} makes row 0 a valid start configuration and all windows between rows 0 and 1 are legal, Fact 4.5.1 shows that row 1 is a valid legal successor. This observation can be applied repeatedly for

$$i = 0, 1, \ldots, T-1.$$

It follows inductively that every row is a valid configuration and that all rows describe a single legal computation branch of N.

(4/4) Accepting constraint

Let q_{acc} be the accepting state. The final row is accepting if some cell contains $\langle q_{\text{acc}}, a \rangle$ for some $a \in \Gamma$. Define

$$\varphi_{\text{accept}} \coloneqq \bigvee_{\substack{1 \le j \le S \\ a \in \Gamma}} X_{T,j,\langle q_{\text{acc}}, a \rangle}. \tag{4.19}$$

If this disjunction is satisfied, at least one of its literals is true. Therefore, there are a column j and a symbol $a \in \Gamma$ such that

$$X_{T,j,\langle q_{\text{acc}}, a \rangle} = 1.$$

By the intended meaning in Equation (4.12), row T then contains the accepting state.

4.5.1.3 Correctness of the Construction

We now prove Equation (4.9). Because this is an if-and-only-if statement, we prove its two directions separately. For the forward direction, we encode an accepting computation branch as a truth assignment. For the reverse direction, we use a satisfying assignment to recover an accepting tableau and hence an accepting computation branch.

Forward direction. Assume that $x \in L_A$. Then N has an accepting computation branch on x, which we pad to T steps if necessary. Assign $X_{i,j,s} = 1$ exactly when cell $[i, j]$ of its computation tableau contains s. By construction, every cell contains exactly one symbol, the first row is the initial configuration, the boundary markers are correct, every pair of consecutive rows represents a legal transition, and the final row is accepting. Hence all components of φ_x are satisfied, and therefore

$$x \in L_A \implies \varphi_x \text{ is satisfiable.}$$

Reverse direction. Assume that φ_x is satisfiable. The cell constraints determine a unique symbol in every tableau cell, while the start and boundary constraints ensure that the first row and boundary markers are correct. The move constraints ensure that every local window is legal, so Fact 4.5.1 implies that each row is a legal successor of the preceding row. Finally, the acceptance constraint ensures that the last row is accepting. Thus, the tableau describes an accepting computation of N on x, and therefore

$$\varphi_x \text{ is satisfiable} \implies x \in L_A.$$

Consequently,

$$x \in L_A \iff \varphi_x \text{ is satisfiable.}$$

4.5.1.4 Size and Construction Time

It remains to prove that the mapping

$$f_N(x) \coloneqq \langle \varphi_x \rangle$$

is computable in polynomial time. This requirement is essential: proving only that a suitable formula exists would not establish a polynomial-time reduction.

The argument has two parts. We first bound the size of the formula and then describe how a deterministic machine generates it within a comparable time bound.

Polynomial size.

Number of Variables. The tableau has

$$(T + 1)S = (T + 1)(T + 3) = O(T^2)$$

cells. For each cell and each symbol $s \in D$, there is one variable $X_{i,j,s}$. Since D is a fixed finite set depending only on N, the number of variables of this form is

$$(T + 1)S|D| = O(T^2).$$

Number of Clauses. We analyze the number of *clauses* of φ_x separately (recall the definition of a clause in Definition 3.3.2):

- For each of the $O(T^2)$ cells, φ_{cell} (shown in Equation (4.14)) contains one "at-least-one symbol" clause and a constant number of "at-most-one symbol" clauses. Thus, φ_{cell} has $O(T^2)$ clauses.
- The formula $\varphi_{\text{boundary}}$ (shown in Equation (4.15)) contains two unit clauses for each of the $T + 1$ rows, so it contains $O(T)$ clauses.

- The formula φ_{start} (shown in Equation (4.16)) contains one unit clause for each of the $S = T + 3 = O(T)$ columns, so it contains $O(T)$ clauses.
- For the formula φ_{move} Equation (4.18), first recall that there are

$$T(S-2) = T(T+1) = O(T^2)$$

  window positions. Since $D^6 \setminus W_N$ is a fixed finite set, φ_{move} contains a constant number of six-literal clauses per window and hence $O(T^2)$ clauses in total.
- The formula φ_{accept} (shown in Equation (4.19)) contains $S|\Gamma| = O(T)$ literals. If it is converted to 3-CNF using Lemma 4.6.1, the conversion introduces only $O(T)$ clauses and auxiliary variables.

Writing a row or column index requires $O(\log T)$ bits. Thus, under an explicit binary encoding of variable names, the total encoded length of the formula is at most

$$O(T^2 \log T).$$

Because $T(n) = n^k$, this quantity is polynomial in $n = |x|$.

Polynomial-time construction

The formula φ_x has size polynomial in $|x|$, as shown above. Moreover, the preceding construction is explicit: each of its clauses can be generated directly in time polynomial in its length. Therefore, φ_x can be constructed in polynomial time. Together with Equation (4.9), this proves

$$L_A \leq_{\text{P}} \text{SAT}.$$

This completes the proof of Theorem 4.5.1.

4.6 Consequences of the Cook–Levin Theorem

The Cook–Levin theorem gives us the first NP-complete language. We may now use polynomial-time reductions to prove that other languages are NP-complete.

4.6.1 3SAT Is NP-Complete

The Cook–Levin theorem shows that SAT is NP-complete. We now show that the restriction of SAT to formulas in 3-CNF is also NP-complete. This restriction is called 3SAT. Formally, we show that

Proposition 4.6.1. *It holds that SAT $\leq_p$ 3SAT. Consequently,* 3SAT *is NP-complete.*

Recall that a SAT formula is a conjunction of clauses, where each clause is a disjunction of literals. A formula is in 3-CNF if every clause contains at most three literals. Therefore, to prove Proposition 4.6.1, it suffices to show that every CNF formula can be converted into an equisatisfiable 3-CNF formula in polynomial time. We do this in two cases, according to the number of literals in each clause.

Clauses with one or two literals

We may repeat literals:

$$(a) \quad \rightsquigarrow \quad (a \vee a \vee a),$$

and

$$(a \vee b) \quad \rightsquigarrow \quad (a \vee b \vee b).$$

These replacements are logically equivalent to the original clauses.

Clauses with more than three literals

Suppose a clause contains more than three literals:

$$a_1 \vee a_2 \vee \cdots \vee a_r, \qquad r > 3. \tag{4.20}$$

We can replace this clause by a sequence of clauses, each containing exactly three literals.

Introduce fresh variables

$$z_1, z_2, \ldots, z_{r-3},$$

and do the following replacements:

1. The first clause contains a_1 and a_2:

$$a_1 \vee a_2 \vee z_1.$$

2. Each intermediate clause contains one of the original literals and connects two consecutive auxiliary variables:

$$\neg z_k \vee a_{k+2} \vee z_{k+1}, \qquad 1 \leq k \leq r-4.$$

3. Finally, the last clause contains the last two original literals:

$$\neg z_{r-3} \vee a_{r-1} \vee a_r.$$

Thus, we replace Equation (4.20) by

$$\begin{aligned} &(a_1 \vee a_2 \vee z_1) \\ &\quad \wedge \bigwedge_{k=1}^{r-4} (\neg z_k \vee a_{k+2} \vee z_{k+1}) \\ &\quad \wedge (\neg z_{r-3} \vee a_{r-1} \vee a_r). \end{aligned} \tag{4.21}$$

When $r = 4$, the middle conjunction is empty, so the construction consists only of the first and last clauses.

The auxiliary variables pass information from one clause to the next. If the literals considered so far are all false, the current clause forces the next auxiliary variable to be true. Once a true original literal is encountered, the remaining auxiliary variables can be set to false.

The formulas in Equations (4.20) and (4.21) are not equivalent because Equation (4.21) contains additional variables. They are, however, *equisatisfiable*: an assignment to the original variables satisfies Equation (4.20) if and only if it can be extended to the auxiliary variables so that Equation (4.21) is satisfied.

Example 4.6.1. *Consider the five-literal clause*

$$a_1 \vee a_2 \vee a_3 \vee a_4 \vee a_5.$$

Introduce two fresh variables z_1 and z_2. The resulting formula is

$$(a_1 \vee a_2 \vee z_1) \wedge (\neg z_1 \vee a_3 \vee z_2) \wedge (\neg z_2 \vee a_4 \vee a_5).$$

For example, suppose that a_3 is true and all the other a_i's are false. Setting

$$z_1 = 1 \qquad \textit{and} \qquad z_2 = 0$$

satisfies all three clauses:

$$(0 \vee 0 \vee 1) \wedge (0 \vee 1 \vee 0) \wedge (1 \vee 0 \vee 0).$$

By contrast, if all five original literals are false, the first clause forces $z_1 = 1$, the second then forces $z_2 = 1$, and the last clause becomes

$$\neg 1 \vee 0 \vee 0,$$

which is false.

Lemma 4.6.1 (Long clause to 3-CNF)**.** *A fixed assignment to $a_1, \ldots, a_r$ satisfies the long clause in Equation (4.20) if and only if it can be extended to $z_1, \ldots, z_{r-3}$ so that Equation (4.21) is satisfied.* ◊

Proof. We prove the two directions separately.

For the forward direction: Assume that the long clause Equation (4.20) is satisfied. Note that this means that at least one of the original literals $a_1, \ldots, a_r$ is true. We must then choose values for the auxiliary variables in Equation (4.21) so that every clause in the converted formula is satisfied. The choice depends on where the first true original literal appears: it may occur in the first clause, in one of the intermediate clauses, or in the last clause. These three cases cover all possibilities.

The auxiliary variables can be viewed as passing a signal along the chain. Before the first true literal is reached, we set the relevant auxiliary variables to 1, which satisfies each preceding clause through its positive auxiliary literal. After the first true literal is reached, we set the remaining auxiliary variables to 0, which satisfies each subsequent clause through a negated auxiliary literal.

We now carry out this construction. Suppose that a_j is the first true literal. There are three possibilities:

- **Case 1:** $j \in \{1, 2\}$**:** Set every auxiliary variable to 0. The first clause is satisfied by a_j (which is either a_1 or a_2 in this case), while every later clause is satisfied by a literal of the form $\neg z_k$.
- **Case 2:** $3 \leq j \leq r - 2$**:** Set
$$z_1 = \cdots = z_{j-2} = 1$$
and set every remaining auxiliary variable z_k to 0. Each clause before the one containing a_j is satisfied by the positive auxiliary variable that passes the signal to the next clause. The clause containing a_j, i.e.,
$$(\neg z_{j-2} \vee a_j \vee z_{j-1}),$$
is satisfied by a_j itself. Finally, each clause after it is satisfied by a negated auxiliary variable.
- **Case 3:** $j \in \{r - 1, r\}$**.** In this case, set every auxiliary variable to 1. Each clause before the last one is satisfied by its positive auxiliary literal, and the last clause is satisfied by a_j.

This completes the forward direction: whenever the long clause Equation (4.20) is satisfied, the assignment can be extended to satisfy the converted formula Equation (4.21).

For the reverse direction: We now proceed to prove the reverse direction. However, it is a little tricky to prove this direction directly. Instead, we choose to prove the contrapositive, which is equivalent to the original statement. That is, we show that if the long clause Equation (4.20) is false, then no choice of the auxiliary variables can satisfy the converted formula Equation (4.21).

First, notice that if the long clause is false, then every original literal a_i is false. Under this assumption, the first clause

$$a_1 \vee a_2 \vee z_1$$

forces $z_1 = 1$. Once $z_1 = 1$ has been forced, the next clause

$$\neg z_1 \vee a_3 \vee z_2$$

will force $z_2 = 1$ because both a_3 and $\neg z_1$ are false.

This observation can be applied repeatedly to the intermediate clauses. That is, for each k from 1 to $r - 4$, once $z_k = 1$ has been forced, the corresponding intermediate clause

$$\neg z_k \vee a_{k+2} \vee z_{k+1}$$

has its first two literals false and therefore forces $z_{k+1} = 1$. Proceeding through the chain gives

$$z_1 = z_2 = \cdots = z_{r-3} = 1.$$

The last clause then becomes

$$\neg z_{r-3} \vee a_{r-1} \vee a_r = \neg 1 \vee 0 \vee 0,$$

which is false. Thus, if all the original literals are false, the converted formula cannot be satisfied.

Combining the two directions, the long clause is satisfied if and only if the assignment to its original literals can be extended to the auxiliary variables so that the converted formula is satisfied.

■

The conversion introduces $r - 3$ new variables and $r - 2$ clauses. Consequently, it increases the size of the formula by only a linear factor.

Corollary 4.6.1. *The language* 3SAT *is NP-complete.* ◊

Proof. Clearly, 3SAT $\in$ NP, since a proposed truth assignment can be checked in polynomial time.

For NP-hardness, the Cook–Levin reduction constructs, for every $L_A \in \mathsf{NP}$ and input x, a CNF formula φ_x such that

$$x \in L_A \iff \varphi_x \text{ is satisfiable.}$$

Convert φ_x into a 3-CNF formula ψ_x: clauses with fewer than three literals are padded by repeating literals, and longer clauses are converted using Lemma 4.6.1. This conversion runs in polynomial time, increases the formula size by only a polynomial factor, and preserves satisfiability. Hence

$$x \in L_A \iff \psi_x \in \text{3SAT}.$$

Thus, 3SAT is NP-hard and therefore NP-complete. ■

4.6.2 The 2SAT problem

In contrast to 3SAT, the problem 2SAT can be solved efficiently. Given a 2-CNF formula φ, construct its *implication graph* G_φ. The graph has one vertex for each literal x and $\neg x$. A clause

$$(\ell_1 \vee \ell_2)$$

is logically equivalent to the pair of implications

$$\neg \ell_1 \implies \ell_2 \qquad \text{and} \qquad \neg \ell_2 \implies \ell_1,$$

so the corresponding directed edges are added to G_φ.

For example, consider

$$\varphi = (x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y).$$

Its implication graph is shown in Figure 4.1.

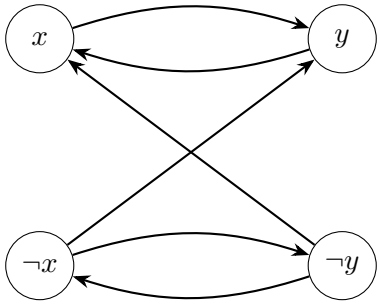


Figure 4.1: The implication graph of $\varphi = (x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y)$.

2SAT and Strongly Connected Components. A *strongly connected component* (SCC) of a directed graph is a maximal set C of vertices such that, for every $u, v \in C$, there is a directed path from u to v and a directed path from v to u. The *strongly connected components problem* asks us to partition the vertices of a directed graph into all such maximal sets. Efficient linear-time algorithms for this problem are covered in CSCI3160.

In Figure 4.2, the implication graph above has two SCCs:

$$C_1 = \{\neg x, \neg y\} \qquad \text{and} \qquad C_2 = \{x, y\}.$$

Within each component, every vertex is reachable from every other vertex. Contracting each SCC to a single vertex produces the *condensation graph*, which is always a directed acyclic graph.

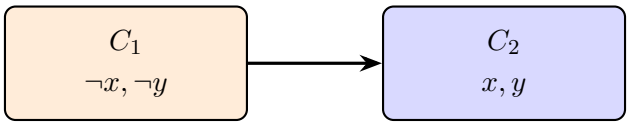


Figure 4.2: The condensation graph of Figure 4.1. Each box represents one strongly connected component.

The formula φ is satisfiable if and only if no variable x and its negation $\neg x$ lie in the same SCC of G_φ. Indeed, if they belonged to the same SCC, the graph would contain paths representing both

$$x \implies \neg x \qquad \text{and} \qquad \neg x \implies x,$$

making either truth value for x impossible. Conversely, if no such pair belongs to the same

SCC, a satisfying assignment can be recovered from a topological ordering of the condensation graph. Since the implication graph has $2n$ vertices and two edges per clause, computing its SCCs gives an $O(n+m)$-time algorithm for 2SAT, where n is the number of variables and m is the number of clauses.

"3" is special! The contrast is remarkable: 2SAT has a linear-time algorithm, but allowing just one additional literal per clause makes 3SAT NP-complete. This "jump at three" appears throughout theoretical computer science. Graph 2-colorability is simply bipartiteness and is easy to test, whereas graph 3-colorability is NP-complete. Two-dimensional matching is ordinary bipartite matching and is polynomial-time solvable, whereas 3-dimensional matching is NP-complete. Likewise, packing sets of size two is a matching problem, while packing sets of size three is NP-complete. Adding just one more choice may look harmless, yet it can completely transform the computational landscape—a small change in the problem statement can mark the boundary between an efficient algorithm and NP-completeness.

4.6.3 CLIQUE Is NP-Complete

We can now combine the NP-completeness of 3SAT with the reduction established in Theorem 4.3.1.

Corollary 4.6.2. *The language* CLIQUE *is NP-complete.* ◊

Proof. We first show that CLIQUE $\in$ NP. Given an instance $\langle G, k\rangle$, a certificate is a list

$$v_1, v_2, \ldots, v_k$$

of vertices. The verifier checks that:

1. all listed vertices belong to G;
2. the listed vertices are distinct;
3. for every $1 \le i < j \le k$, the edge $\{v_i, v_j\}$ belongs to G.

There are

$$\binom{k}{2} = O(k^2)$$

pairs to check, so the verification runs in polynomial time. Therefore, CLIQUE $\in$ NP.

By Corollary 4.6.1, 3SAT is NP-complete, and by Theorem 4.3.1,

$$\text{3SAT} \le_{\text{P}} \text{CLIQUE}.$$

Applying the NP-completeness transfer theorem, Theorem 4.2.2, shows that CLIQUE is NP-complete. ■

Chapter 5
Exemplary NP-Complete Problems

Having proved that 3SAT and CLIQUE are NP-complete, we can use them as starting points for proving the NP-completeness of many other problems. The reductions in this section follow the general pattern

$$\text{3SAT} \longrightarrow \text{CLIQUE} \longrightarrow \text{INDEPENDENT-SET} \longrightarrow \text{VERTEX-COVER},$$

together with the additional branches

$$\text{3SAT} \longrightarrow G3C, \qquad \text{3SAT} \longrightarrow \text{HAMILTONIAN-PATH},$$

and

$$\text{3SAT} \longrightarrow \text{SUBSET SUM} \longrightarrow \text{KNAPSACK}.$$

5.1 Independent Set

Recall that an *independent set* in an undirected graph $G = (V, E)$ is a set $S \subseteq V$ such that no two vertices in S are adjacent. Also recall the deinifion of the language INDEPENDENT-SET:

$$\text{INDEPENDENT-SET} \coloneqq \{\langle G, k\rangle \mid G \text{ contains an independent set of size at least } k\}.$$

The relationship between cliques and independent sets is particularly simple. Recall that the *complement* of $G = (V, E)$ is the graph $\overline{G} = (V, \overline{E})$, where, for distinct vertices u, v,

$$\{u, v\} \in \overline{E} \iff \{u, v\} \notin E.$$

Thus, every edge between vertices of a set S is present in G exactly when no such edge is present in $\overline{G}$. Consequently,

$$S \text{ is a clique in } G \iff S \text{ is an independent set in } \overline{G}.$$

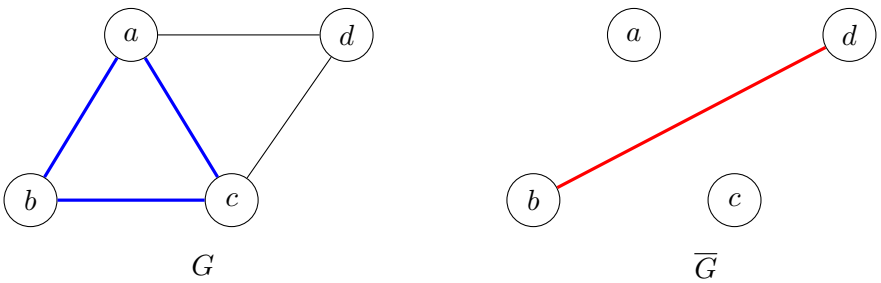


Figure 5.1: The vertices $\{a, b, c\}$ form a clique in G, and therefore form an independent set in $\overline{G}$.

Theorem 5.1.1. INDEPENDENT-SET *is NP-complete.* ◊

Proof. First, INDEPENDENT-SET $\in$ NP. A certificate is a set $S \subseteq V$ of k vertices. A verifier

checks that no pair of vertices in S is joined by an edge.

For NP-hardness, reduce from CLIQUE. Given an instance $\langle G, k\rangle$, construct

$$f(\langle G, k\rangle) = \langle \overline{G}, k\rangle.$$

The complement graph can be constructed in polynomial time, and

$$G \text{ has a clique of size } k \iff \overline{G} \text{ has an independent set of size } k.$$

Therefore,

$$\text{CLIQUE} \leq_{\text{P}} \text{INDEPENDENT-SET}.$$

Since CLIQUE is NP-complete, INDEPENDENT-SET is NP-complete. ■

5.2 Vertex Cover

A *vertex cover* of an undirected graph $G = (V, E)$ is a set $C \subseteq V$ such that every edge has at least one endpoint in C.

Definition 5.2.1 (Vertex Cover)**.** *The language* VERTEX-COVER *consists of all pairs* $\langle G, k\rangle$ *such that* G *has a vertex cover of size at most* k*.* ◊

Independent sets and vertex covers are complementary notions within the same graph.

Lemma 5.2.1. *For every graph* $G = (V, E)$ *and every* $S \subseteq V$*,*

$$S \text{ is an independent set} \iff V \setminus S \text{ is a vertex cover.}$$

◊

Proof. Suppose first that S is independent. No edge can have both endpoints in S, so every edge has at least one endpoint in $V \setminus S$. Hence $V \setminus S$ is a vertex cover.

Conversely, suppose that $V \setminus S$ is a vertex cover. If two vertices of S were adjacent, the edge between them would have no endpoint in $V \setminus S$, contradicting the assumption that $V \setminus S$ covers every edge. Therefore, S is independent. ■

Theorem 5.2.1. VERTEX-COVER *is NP-complete.* ◊

Proof. The problem belongs to NP: given a set C of at most k vertices, a verifier checks that every edge has at least one endpoint in C.

For NP-hardness, reduce from INDEPENDENT-SET. Given $\langle G, k\rangle$, where $G = (V, E)$, output

$$\langle G, |V| - k\rangle.$$

By Lemma 5.2.1,

$$G \text{ has an independent set of size at least } k$$

if and only if

$$G \text{ has a vertex cover of size at most } |V| - k.$$

The reduction changes only the integer parameter and therefore runs in polynomial time. Hence

$$\text{INDEPENDENT-SET} \leq_{\text{P}} \text{VERTEX-COVER},$$

and VERTEX-COVER is NP-complete. ■

5.2.1 Minimum Vertex Cover: From Decision to Search

The decision problem VERTEX-COVER asks whether a vertex cover of a given size exists. Often, however, we want to *find* a smallest vertex cover.

Definition 5.2.2 (Minimum Vertex Cover)**.** *Given an undirected graph $G = (V, E)$, the* Minimum Vertex Cover *problem asks us to output a vertex cover $C \subseteq V$ of minimum cardinality. We write*

$$\tau(G) = \min\{|C| : C \text{ is a vertex cover of } G\}$$

for its size. ◊

A minimum vertex cover immediately answers the decision question: the answer to $\langle G, k\rangle$ is yes exactly when $\tau(G) \leq k$. Conversely, a decision algorithm also lets us find a minimum vertex cover.

Lemma 5.2.2. *Minimum Vertex Cover can be solved using polynomially many calls to an algorithm for* VERTEX-COVER*, together with polynomial-time additional computation.* ◊

The proof of Lemma 5.2.2 is left as a homework exercise.

5.2.2 Maximum Matching and Minimum Vertex Cover

Not every natural-looking graph optimization problem is NP-hard. Maximum Matching provides an important contrast.

A *matching* in a graph $G = (V, E)$ is a set of edges no two of which share an endpoint.

Definition 5.2.3 (Maximum Matching)**.** *The decision version of* MAXIMUM MATCHING *asks whether a graph contains a matching with at least k edges.* ◊

Unlike the problems above, Maximum Matching belongs to P. There are polynomial-time algorithms for maximum matching in both bipartite and general graphs. In particular, maximum matching in a bipartite graph can be found using augmenting paths.

Minimum Vertex Cover behaves differently:

- In general graphs, its decision version is NP-complete, as proved above.
- In bipartite graphs, Minimum Vertex Cover can be solved in polynomial time.

The connection between matching and vertex cover in bipartite graphs is given by Kőnig's theorem.

Theorem 5.2.2 (Kőnig's theorem)**.** *In every bipartite graph, the size of a maximum matching is equal to the size of a minimum vertex cover.* ◊

Proof. Let $G = (L \cup R, E)$ be a finite bipartite graph, and let M be a maximum matching. A vertex is called *matched* if it is an endpoint of an edge in M, and *unmatched* otherwise. Because M is a matching, every matched vertex has exactly one *matching partner*: the vertex at the other endpoint of its edge in M.

Every vertex cover must contain at least one endpoint of each edge in M. Since these edges have no endpoints in common, every vertex cover has size at least $|M|$. We will prove the reverse inequality by constructing a vertex cover with exactly $|M|$ vertices.

1. **Explore the graph.**

   Initially, mark every unmatched vertex in L. Then repeatedly apply the following rules:

- If a marked vertex $u \in L$ has an unmarked neighbor $v \in R$ with $uv \notin M$, mark v.
- If a marked vertex $v \in R$ has an unmarked matching partner $u \in L$, mark u.

Continue until neither rule can mark a new vertex. If several moves are possible, choose any one of them, but do not stop while a move remains possible. In particular, if a left vertex has several neighbors connected to it by edges outside M, all of those neighbors will eventually be marked.

This process terminates because the graph is finite and each move marks a previously unmarked vertex. Let $Z_L \subseteq L$ and $Z_R \subseteq R$ be the sets of marked vertices.

Whenever a new vertex is marked, record the vertex from which it was reached. Tracing these records backward gives a path from an unmatched vertex in L to the newly marked vertex. The path has no repeated vertices, since each predecessor was marked earlier. Its edges alternate between edges outside M and edges in M.

2. **Show that every vertex in Z_R is matched.**

   Suppose, for a contradiction, that some vertex $v \in Z_R$ is unmatched. By the exploration in Item 1, there is a path P from an unmatched vertex in L to v whose edges alternate between edges outside M and edges in M.

   Since P starts in L and ends in R, and each edge crosses between the two parts, P has an odd number of edges. Let us denote the total number of edges in P as $2k+1$ for some $k \geq 0$. Moreover, its edges alternate between edges outside and inside M, starting and ending outside M, so it contains $k+1$ edges outside M and k edges in M for some $k \geq 0$.

   Construct a new set of edges M' as follows:

   - Keep every edge of M that is not on P.
   - Remove the k edges of M that lie on P.
   - Add the $k+1$ edges of P that are not in M.

   To check that M' is a matching, note that each internal vertex of P is incident to two consecutive path edges, exactly one of which belongs to M. This is its only matching edge, so the change removes one incident matching edge and adds one. Each endpoint was unmatched and is incident to exactly one path edge, which is added. Vertices outside P are unaffected. Thus every vertex is incident to at most one edge of M'.

   However,

   $$|M'| = |M| - k + (k+1) = |M| + 1,$$

   contradicting the assumption that M is maximum. Therefore every vertex in Z_R is matched.

3. **Relate the endpoints of each matching edge.**

   For every edge $uv \in M$, with $u \in L$ and $v \in R$, we have

   $$u \in Z_L \quad \Longleftrightarrow \quad v \in Z_R. \tag{5.1}$$

   Indeed, if v is marked, the second exploration rule ensures that its matching partner u is marked as well.

   Conversely, suppose u is marked. Since u is matched, it was not one of the initially marked vertices. The only way to mark a vertex in L afterward is to reach it from its matching partner. Thus v must already have been marked.

4. **Construct a vertex cover.**

   Define
   $$C = (L \setminus Z_L) \cup Z_R.$$
   In words, select the *unmarked* vertices on the left and the *marked* vertices on the right.

   Consider any edge $uv \in E$, where $u \in L$ and $v \in R$. If $u \notin Z_L$, then $u \in C$, so this edge is covered. Otherwise, $u \in Z_L$, and there are two cases:

   - If $uv \notin M$, the first exploration rule in Item 1 ensures that $v \in Z_R$.
   - If $uv \in M$, then $v \in Z_R$ by Equation (5.1).

   In either case, $v \in C$. Hence every edge has an endpoint in C, so C is a vertex cover.

5. **Count the vertices in the cover.**

   By Equation (5.1), the endpoints of each edge in M are either both marked or both unmarked.

   - If both are marked, C contains its right endpoint but not its left endpoint.
   - If both are unmarked, C contains its left endpoint but not its right endpoint.

   Thus C contains exactly one endpoint of each matching edge.

   Moreover, C contains no unmatched vertices. Every unmatched vertex in L was marked initially in Item 1, so it is excluded from C. Every vertex of C on the right belongs to Z_R and is therefore matched by Item 2.

   Since matching edges have no endpoints in common, it follows that
   $$|C| = |M|.$$

By Items 4 and 5, we have a vertex cover of size $|M|$. Since every vertex cover has size at least $|M|$, this cover is minimum. Therefore the maximum matching size equals the minimum vertex cover size. ■

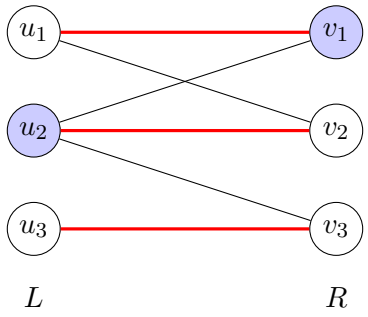


Figure 5.2: A bipartite graph. The thick red edges form a matching of size three. By Kőnig's theorem, every vertex cover has size at least three.

Theorem 5.2.2 is a striking example of how a structural restriction can transform a hard problem into an efficiently solvable one. Minimum Vertex Cover is NP-complete on general graphs, but becomes polynomial-time solvable as soon as the graph is required to be bipartite.

The broader lesson is one of the most exciting themes in algorithms: seemingly tiny changes in a problem can produce enormous changes in complexity. Two colors are easy, while three colors are NP-complete. Matching is easy, while three-dimensional matching is NP-complete. Vertex Cover is hard on general graphs, yet easy on bipartite graphs. Discovering exactly where

these transitions occur is like finding a computational fault line: on one side lies an efficient algorithm, while on the other lies the full difficulty of NP-completeness.

5.3 Graph 3-Coloring

recall that a *proper* 3-*coloring* of an undirected graph $G = (V, E)$ assigns one of three colors to each vertex so that adjacent vertices receive different colors. The language $G3C$ consists of all graphs that admit a proper coloring using at most three colors.

Theorem 5.3.1. *$G3C$ is NP-complete.* ◊

Proof. The problem belongs to NP: given a coloring, we can check that the endpoints of every edge have different colors in polynomial time.

The reduction. Given a 3SAT formula φ, we construct a graph G_φ. We use $\mathrm{NAE}(a, b, c)$ to mean that the three Boolean values a, b, c are *not all equal*.

For each clause $(\ell_1 \vee \ell_2 \vee \ell_3)$, introduce a fresh variable y and form the two constraints

$$\mathrm{NAE}(\ell_1, \ell_2, y), \qquad \mathrm{NAE}(\neg y, \ell_3, F),$$

where F is the constant false. Construct G_φ as follows:

1. Create a triangle with vertices T, F, B.
2. For every variable x, including each new variable y, create vertices $x, \neg x$. Join them to each other and join both to B.
3. For each constraint $\mathrm{NAE}(a, b, c)$, create three fresh vertices q_1, q_2, q_3 forming a triangle, and add the edges
$$\{q_1, a\}, \qquad \{q_2, b\}, \qquad \{q_3, c\}.$$
Here each input denotes its corresponding literal vertex, or the palette vertex F if the input is the constant false.

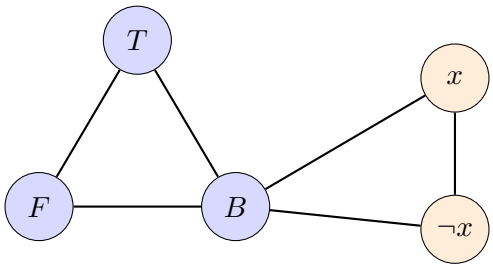


Figure 5.3: The palette triangle and a variable gadget.

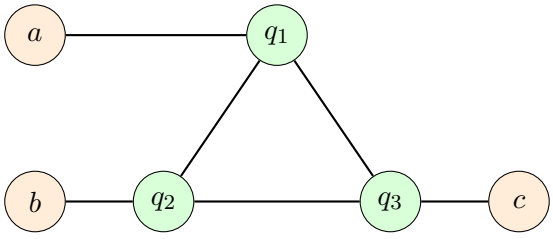


Figure 5.4: The gadget for $\mathrm{NAE}(a, b, c)$.

Correctness. In any proper 3-coloring, the palette vertices have distinct colors; name these colors $\mathsf{T}, \mathsf{F}, \mathsf{B}$, respectively. Each variable gadget forces $x, \neg x$ to receive opposite colors from $\{\mathsf{T}, \mathsf{F}\}$, so their colors represent consistent truth values.

We first check the NAE gadget. If its inputs all have the same color, each q_i must avoid that color, leaving only two colors for a triangle. Thus no extension exists. Otherwise, choose one true input and one false input. Color their adjacent triangle vertices F and T, respectively, and color the remaining vertex B. This is a proper coloring. Hence the gadget extends exactly when its NAE constraint holds.

Next, the two constraints replacing a clause can hold for some y exactly when the clause is true. If ℓ_1 or ℓ_2 is true, choose y false; both constraints hold. If both are false, the first constraint forces y true, and the second then holds exactly when ℓ_3 is true.

Therefore, a satisfying assignment of φ extends to values for all auxiliary variables satisfying every NAE constraint, and hence to a proper 3-coloring of G_φ. Conversely, a proper 3-coloring gives consistent truth values satisfying every NAE constraint, so every original clause is true. Thus

$$\varphi \text{ is satisfiable} \quad \Longleftrightarrow \quad G_\varphi \text{ is 3-colorable.}$$

The construction adds only constantly many vertices and edges per variable and clause, so it takes polynomial time. Therefore $3\mathrm{SAT} \leq_{\mathrm{P}} G3C$, and $G3C$ is NP-complete. ■

Remark 5.3.1. *The jump from 2-coloring to 3-coloring is dramatic: 2-coloring is simply the problem of testing whether a graph is bipartite and can be solved in linear time, whereas 3-coloring is NP-complete. Allowing just one additional color completely changes the computational complexity of the problem. We include a short discussion in Section 5.3.1.*

5.3.1 Three Colors, Four Colors, and a Mathematical Mystery

Nothing from Section 5.3.1 will be tested on quizzes or exams!

From three colors to four colors. A graph is *planar* if it can be drawn in the plane without edges crossing except at common endpoints. Planar graphs arise naturally from map coloring: represent each connected region by a vertex, and connect two vertices when their regions share a boundary segment. Regions meeting only at a point need not receive different colors.

Theorem 5.3.2 (Four Color Theorem)**.** *Every finite simple planar graph has a proper 4-coloring.* ◊

Four colors are sometimes necessary. For example, K_4 is planar, but its four vertices must receive distinct colors because every pair is adjacent.

Although four colors always suffice for a planar graph, determining whether three colors suffice remains difficult: the following problem is NP-complete.

$$\text{PLANAR-3COLOR} = \{\langle G \rangle \mid G \text{ is planar and has a proper 3-coloring}\}.$$

This remains NP-complete even when the maximum vertex degree is at most four.[1]

Thus, when the input is guaranteed to be planar, the answer to "Are four colors enough?" is always yes, but deciding whether three colors suffice is NP-complete. There is no contradiction: Theorem 5.3.2 guarantees a coloring with at most four colors, but does not tell us whether fewer

[1] M. R. Garey, D. S. Johnson, and L. Stockmeyer, "Some simplified NP-complete graph problems," *Theoretical Computer Science* 1 (1976), 237–267.

colors will work. The planarity assumption is essential; the theorem does not apply to arbitrary graphs.

A brief history. The four-color problem originated in 1852, when Francis Guthrie considered coloring a map of the counties of England. Alfred Kempe published a proposed proof in 1879, but Percy Heawood discovered a flaw in 1890. Nevertheless, Kempe's recoloring ideas became important ingredients in later work.

In 1976, Kenneth Appel and Wolfgang Haken established the theorem using extensive computer calculations; John Koch contributed to the computational work. The detailed proof appeared in 1977. In 1997, Neil Robertson, Daniel Sanders, Paul Seymour, and Robin Thomas published a simpler computer-assisted proof using 633 local configurations.[2]

How does a computer help prove the theorem? The computer does *not* simply color many maps and conclude that four colors probably suffice. There are infinitely many finite planar graphs, so checking a large sample would not prove the theorem.

Instead, the computer-assisted proofs reduce the problem to a finite collection of local checks. Their broad strategy is as follows:

1. **Assume a smallest counterexample exists.** Suppose some planar graph cannot be colored with four colors, and choose one with the fewest vertices.

2. **Force a configuration from a finite list.** A *configuration* is a local pattern of vertices and edges, together with specified degree information. Prove that the smallest counterexample must contain at least one configuration from a particular finite list.

3. **Rule out every listed configuration.** Show that each listed configuration is *reducible*, meaning that it cannot occur in a smallest counterexample. Roughly, one reduces the graph to a smaller planar graph, colors the smaller graph, and recovers a coloring of the original graph, possibly by recoloring.

The last two steps contradict each other: the smallest counterexample must contain a listed configuration, but cannot contain any of them. Therefore no counterexample exists. Computers carry out the extensive finite case analysis needed to establish the local claims.[3]

The second step uses a technique called *discharging*. One assigns numerical charges to vertices according to their degrees; Euler's formula determines the total charge. The charges are redistributed according to local rules, without changing their total. The rules are designed so that avoiding every listed configuration would contradict this fixed total. This forces one of the configurations to appear.

The important distinction is that the computer checks *all cases in a mathematically sufficient finite list*, not merely many examples of planar graphs.

Formal verification and the current status. In 2005, Georges Gonthier, working with Benjamin Werner, completed a formal verification of the Four Color Theorem using the Coq proof assistant. Rather than merely running a special-purpose checking program, this project formalized the mathematical argument and verified the reasoning behind the computations.[4] Formal verification does not make the proof computer-free; it makes the computer-assisted reasoning itself subject to systematic logical checking.

[2] N. Robertson, D. P. Sanders, P. D. Seymour, and R. Thomas, "The four-colour theorem," *Journal of Combinatorial Theory, Series B* 70 (1997), 2–44. For historical background, see also Robin Thomas, "The Four Color Theorem," Georgia Institute of Technology.

[3] For the proof framework and its computational checks, see Robertson, Sanders, Seymour, and Thomas, "The four-colour theorem," and their accompanying computer programs and configuration data.

[4] G. Gonthier, "Formal Proof—The Four-Color Theorem," *Notices of the American Mathematical Society* 55 (2008), 1382–1393.

As of October 2026, no generally accepted computer-free proof is known. The theorem is nevertheless established: the absence of a computer-free proof does not make it an open conjecture.

Research also continues on the algorithmic side. A preprint first posted in March 2026 by Yuta Inoue, Ken-ichi Kawarabayashi, Atsuyuki Miyashita, Bojan Mohar, Carsten Thomassen, and Mikkel Thorup gives a new computer-assisted approach yielding an $O(n \log n)$-time algorithm for finding a 4-coloring of an n-vertex planar graph, improving the earlier quadratic-time bound. The broad idea is to find many reductions that can be used together, so that each stage substantially shrinks the graph.[5] This improves the efficiency of finding a coloring, but does not eliminate computer-assisted case checking.

A connection to *Suspect X*. The Japanese film *Suspect X* (2008), adapted from Keigo Higashino's novel *The Devotion of Suspect X*, features Tetsuya Ishigami, a gifted mathematician working as a high-school teacher. The Four Color Theorem appears in the film's mathematical discussions.[6]

In the novel, Ishigami knows about the existing computer-assisted proof but seeks a more elegant proof of his own.[7] He is therefore not trying to settle an unsolved conjecture; he is searching for a different explanation of a theorem already known to be true.

Without revealing the mystery, this gives us a question to consider while watching the film:

> Once we know that a theorem is true, what can we gain from finding another proof?

A different proof might explain the result more clearly, connect it to other ideas, or yield a faster algorithm. The Four Color Theorem illustrates why proving a theorem need not be the end of its mathematical story.

Here is a link[8] to 3 scenes from the film that are relevant to the Four Color Theorem.

5.4 Hamiltonian Path and Subset Sum

Recall that a *Hamiltonian path* in an undirected graph is a path that visits every vertex exactly once. The language HAMILTONIAN-PATH consists of all undirected graphs that contain a Hamiltonian path.

Theorem 5.4.1. *Both* HAMILTONIAN-PATH *and* SUBSET-SUM *are NP-complete.* ◊

Theorem 5.4.1 can be established by reducing 3SAT to both languages. But these two reductions are complicated and boring. So, we decide to not cover them in this undergraduate course. But you need to memorize this Theorem 5.4.1.

[5] Y. Inoue et al., "The Four Color Theorem with Linearly Many Reducible Configurations and Near-Linear Time Coloring," arXiv:2603.24880, first posted March 25, 2026; revised October 1, 2026.

[6] See Oliver Knill's *Mathematics in Movies* entry for *Suspect X* (2008), Harvard University.

[7] See Alex Kasman's *MathFiction* entry for Keigo Higashino's *The Devotion of Suspect X*, College of Charleston.

[8] This link is password-protected so that only the instructor can play it during the lecture.

Bibliography

[AB09] Sanjeev Arora and Boaz Barak. *Computational complexity: a modern approach*. Cambridge University Press, 2009. 14, 18

[ACM12] ACM Special Interest Group on Algorithms and Computation Theory. 2012 knuth prize citation: Leonid levin. ACM SIGACT Knuth Prize citation, 2012. 91

[AKS04] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. Primes is in p. *Annals of mathematics*, pages 781–793, 2004. 47, 73

[Ber57] Claude Berge. Two theorems in graph theory. *Proceedings of the National Academy of Sciences of the United States of America*, 43(9):842–844, 1957. doi:10.1073/pnas.43.9.842. 38

[Coo71] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, STOC '71, pages 151–158. Association for Computing Machinery, 1971. doi:10.1145/800157.805047. 90

[Edm65] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965. doi:10.4153/CJM-1965-045-4. 40

[HK73] John E. Hopcroft and Richard M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 2(4):225–231, 1973. doi:10.1137/0202019. 40

[Lev73] Leonid A. Levin. Universal search problems. *Problems of Information Transmission*, 9(3):265–266, 1973. English translation of "Universal'nye zadachi perebora," originally published in Problemy Peredachi Informatsii, volume 9, number 3, pages 115–116. 90

[Mil75] Gary L Miller. Riemann's hypothesis and tests for primality. In *Proceedings of the seventh annual ACM symposium on Theory of computing*, pages 234–239, 1975. 49

[MV80] Silvio Micali and Vijay V. Vazirani. An $O(\sqrt{|V|}\,|E|)$ algorithm for finding maximum matching in general graphs. In *Proceedings of the 21st Annual Symposium on Foundations of Computer Science*, pages 17–27. IEEE, 1980. doi:10.1109/SFCS.1980.12. 40

[Rab80] Michael O Rabin. Probabilistic algorithm for testing primality. *Journal of number theory*, 12(1):128–138, 1980. 49